راهنمای توسعه‌دهندهٔ هم‌رسان

سند کامل معماری، تکنولوژی و کد پروژه، بخش به بخش

نسخهٔ سند: ۱٫۰ (بر پایهٔ کامیت `00c4bc2`، build ۱۹۴)

تاریخ: ۱۴۰۵/۰۸/۱۳

توسعه‌دهنده: علی رودی

شناسهٔ بسته: `ir.hamrasan.app`

فهرست

پیش از شروع: این سند را چطور بخوانیم

بخش ۱. معرفی پروژه

بخش ۲. راه‌اندازی محیط و دستورهای روزانه

بخش ۳. تکنولوژی‌ها و کتابخانه‌ها

بخش ۴. معماری کلی

بخش ۵. مدیریت وضعیت با BLoC

بخش ۶. پایگاه‌داده و ذخیره‌سازی

بخش ۷. پل Dart و Kotlin

بخش ۸. پیامک: دریافت، ارسال و همگام‌سازی

بخش ۹. صندوق پیام، جستجو، گروه و مسدودی

بخش ۱۰. صفحهٔ گفتگو و نوشتن پیام

بخش ۱۱. پیش‌نویس و قالب آماده

بخش ۱۲. پیام زمان‌بندی‌شده

بخش ۱۳. تماس و شماره‌گیر

بخش ۱۴. «اخیر»: فهرست تماس و یادآور

بخش ۱۵. دو سیم‌کارت

بخش ۱۶. مخاطبین و موردعلاقه‌ها

بخش ۱۷. جستجوی مخاطب، T9 و عادی‌سازی شماره

بخش ۱۸. تنظیمات

بخش ۱۹. احراز هویت و قفل برنامه

بخش ۲۰. نسخه‌ها و فعال‌سازی

بخش ۲۱. بخش امن

بخش ۲۲. رمزنگاری پیامک

بخش ۲۳. بانک کلید و گواهی عضو

بخش ۲۴. موتور پیام رمز و گروه رمز

بخش ۲۵. دفترچهٔ مخفی

بخش ۲۶. امکانات امنیتی دیگر

بخش ۲۷. ابزار و پنل ویندوزی

بخش ۲۸. ظاهر، RTL و ویجت‌های مشترک

بخش ۲۹. سرعت

بخش ۳۰. شبکه و گزارش خطا

بخش ۳۱. آزمون‌ها

بخش ۳۲. ساخت و انتشار

بخش ۳۳. قوانین طلایی و چک‌لیست تغییر

بخش ۳۴. مشکلات شناخته‌شده و کارهای باقی

پیوست الف: فهرست فایل‌به‌فایل کد

پیوست ب: تصمیم‌های طراحی

پیوست پ: واژه‌نامه

پیوست ت: برای هر تغییر سراغ کجا بروید

پیش از شروع: این سند را چطور بخوانیم

این سند برای توسعه‌دهنده‌ای نوشته شده که هم‌رسان را تا حالا ندیده و می‌خواهد بدون کمک کسی روی کل پروژه مسلط شود. فرض سند این است که Flutter و کمی Android را می‌شناسید. رمزنگاری، Telecom و نقش‌های پیش‌فرض اندروید را لازم نیست از قبل بدانید؛ هر جا لازم باشد از پایه توضیح داده شده‌اند.

ساختار سند

هر بخش راهنمای مستقل یک قسمت از پروژه است. مثلاً اگر فقط با رمزنگاری کار دارید، بخش‌های ۲۱ تا ۲۴ را می‌خوانید و به بقیه نیاز ندارید. هر بخش این چیزها را می‌گوید:

- این قسمت چه کاری می‌کند و کاربر آن را کجا می‌بیند.
- کد آن کجاست و فایل‌های اصلی‌اش کدام‌اند.
- داده چطور از لایه‌ها می‌گذرد.
- چه قانون‌هایی نباید شکسته شوند و چرا. بیشتر این قانون‌ها از یک خطای واقعی روی گوشی آمده‌اند.

مسیر پیشنهادی یادگیری

مرحله	چه بخوانید	نتیجه
۱. روز اول	بخش‌های ۱ تا ۷	برنامه را می‌سازید و اجرا می‌کنید و نقشهٔ کلی را می‌فهمید
۲. هستهٔ برنامه	بخش‌های ۸ تا ۱۸	پیامک، تماس، مخاطبین و تنظیمات را می‌فهمید
۳. بخش امن	بخش‌های ۱۹ تا ۲۷	قفل برنامه، نسخه‌ها، رمزنگاری، دفترچهٔ مخفی و ابزار ویندوزی را می‌فهمید
۴. کار روزانه	بخش‌های ۲۸ تا ۳۴	ظاهر، سرعت، آزمون، انتشار و چک‌لیست تغییر را یاد می‌گیرید
۵. مرجع	پیوست‌ها	هر فایل کد چه کاری می‌کند و برای هر تغییر سراغ کدام فایل بروید

نشانه‌ها و قراردادها

- متن‌های داخل «گیومه» نوشته‌هایی‌اند که کاربر روی صفحه می‌بیند، مثل «اخیر» یا «پیام‌های رمز».
- نام فایل، کلاس و تابع همیشه به انگلیسی و با قلم کد است، مثل `MessageBloc`.
- مسیر فایل‌های Dart نسبت به `lib/` است و برای کوتاهی، `lib/features/` هم از اول مسیر امکانات حذف شده است؛ پس `contacts/models/vcf_file.dart` یعنی `lib/features/contacts/models/vcf_file.dart`. مسیر فایل‌های Kotlin از `android/app/src/main/kotlin/com/example/communication_super_app/` شروع می‌شوند. برای کوتاهی، این مسیر دوم را در سند `kt/` می‌نویسیم.
- کادر آبی یعنی توضیح مهم. کادر زرد یعنی قانونی که شکستنش برنامه را خراب می‌کند.

سندهای دیگر پروژه

این سند همهٔ پروژه را یک‌جا جمع می‌کند، اما سند زندهٔ پروژه نیست. جزئیات تازه همیشه در این فایل‌ها نوشته می‌شوند و هنگام تغییر کد باید آن‌ها را هم به‌روز کنید:

فایل	محتوا
`docs/DEVELOPER_GUIDE.md`	قانون‌های سراسری و فهرست یادداشت‌ها
`docs/architecture/*.md`	یک یادداشت برای هر قسمت: تماس، پیامک، مخاطبین، صندوق و گروه، گفتگو، زمان‌بندی، دو سیم‌کارت، پیش‌نویس و قالب، تنظیمات، نسخه‌ها
`docs/ARCHITECTURE.md` ، `PROJECT_STRUCTURE.md` ، `STATE_MANAGEMENT.md` ، `NETWORK_LAYER.md`	معماری و ساختار
`docs/DESIGN_DECISIONS.md`	تصمیم‌های طراحی D1 تا D17
`docs/PERFORMANCE_NOTES.md` ، `KNOWN_ISSUES.md` ، `CONTRIBUTING.md`	سرعت، مشکلات باز و روش کار
`docs/FINAL_DEVICE_TESTS.md`	آزمون‌هایی که فقط با دو گوشی واقعی انجام می‌شوند
`docs/publishing/store-release.md`	وضعیت زندهٔ انتشار در بازار و مایکت
`tools/keybank/README.md`	ابزار ویندوزی

بخش ۱. معرفی پروژه

۱.۱ هم‌رسان چیست

هم‌رسان یک برنامهٔ اندروید فارسی برای **پیامک و تلفن** است. برنامه جای دو برنامهٔ اصلی گوشی را می‌گیرد: برنامهٔ پیام‌رسان پیش‌فرض (نقش `ROLE_SMS`) و تلفن پیش‌فرض (نقش `ROLE_DIALER`). یعنی همهٔ پیامک‌ها و همهٔ تماس‌های گوشی از این برنامه می‌گذرند و صفحهٔ تماس هم صفحهٔ خود برنامه است.

برنامه **سرور ندارد**. همهٔ داده روی خود گوشی است و تنها ارتباط با بیرون، پیامک و تماس از طریق اپراتور است. تنها استثنا گزارش خطاست که به‌طور پیش‌فرض خاموش است و در نسخهٔ سازمانی اصلاً در برنامه وجود ندارد.

۱.۲ چهار تب برنامه

تب	نام داخلی	کار
۰	«اخیر»	فهرست تماس‌ها، یادآورها، و دکمهٔ باز کردن شماره‌گیر
۱	«موردعلاقه‌ها»	مخاطبان ستاره‌دار و شماره‌گیری سریع
۲	«مخاطبین»	دفترچهٔ تلفن گوشی با برچسب، ادغام، فایل `.vcf` و سیم‌کارت
۳	«پیام‌ها»	صندوق پیامک، گفتگو، پیش‌نویس، قالب، زمان‌بندی و گروه

شماره‌گیر تب جدا ندارد و به‌صورت یک برگهٔ پایین‌صفحه (`showDialerBottomSheet`) باز می‌شود.

۱.۳ سه نسخهٔ برنامه

پروژه یک کد دارد و سه نسخه از آن به دست کاربر می‌رسد:

نسخه	چطور به دست کاربر می‌رسد	چه دارد
تجاری	APK فلیور `commercial`، از بازار و مایکت	پیامک و تلفن کامل، با قفل برنامه و فهرست سیاه و سفید
بین‌سازمانی	همان APK تجاری بعد از وارد کردن کد فعال‌سازی	نسخهٔ تجاری به‌علاوهٔ بخش امن: پیام رمز، گروه رمز، بانک کلید، دفترچهٔ مخفی، کنترل امنیت گوشی، عکس ورود ناموفق
سازمانی	APK فلیور `organization`، تحویل مستقیم، هرگز از فروشگاه	همهٔ امکانات بین‌سازمانی به‌علاوهٔ متن پوششی. تا کد فعال‌سازی وارد نشود هیچ چیز باز نمی‌شود. گزارش خطا در آن اصلاً کامپایل نمی‌شود

بخش ۲۰ این موضوع را کامل توضیح می‌دهد.

۱.۴ اعداد پروژه

مورد	مقدار
کامیت‌ها	۲۰۱ (از ۱۴۰۴/۰۹/۱۴)
Dart در `lib/`	۷۷,۲۶۱ خط در ۲۸۹ فایل
Kotlin برنامه	۱۸,۸۳۲ خط در ۸۱ فایل
ابزار و پنل ویندوزی	حدود ۲,۸۵۰ خط

مورد	مقدار
BLoC	۱۹ عدد
جدول پایگاه‌دادهٔ اصلی	۲۱ جدول
جدول `secure.db`	۱۵ جدول
کانال Kotlin و Dart	۲۵ کانال
آزمون‌ها	۱۴۲ JVM، ۹۱۰ Dart، ابزار ۲۵، پنل ۱۳
مجوزهای مانیفست	۲۶ مجوز

حجم کد به این شکل تقسیم شده است:

قسمت	خط	فایل
`lib/core`	۱۰,۸۷۰	۶۰
`features/messages`	۲۴,۷۰۹	۷۲
`features/contacts`	۸,۸۲۶	۳۳
`features/secure_sms`	۵,۵۳۸	۱۲
`features/dialer`	۵,۵۳۲	۱۶
`features/secure`	۴,۸۸۲	۱۴
`features/settings`	۴,۷۸۲	۱۷
`features/call_history`	۳,۴۱۲	۱۲
`features/authentication`	۲,۴۲۸	۱۷
`features/hidden`	۲,۲۵۳	۱۰
`features/keybank`، `security`، `edition`، `favorites`، `search`	۳,۸۷۹	۲۵
Kotlin ریشه	۴,۵۰۱	۱۵
Kotlin `call/`	۳,۲۳۶	۱۰
Kotlin `smscrypto/`	۲,۵۶۲	۱۴ به‌علاوهٔ ۸ فایل `keybank/`
Kotlin `contacts/`	۲,۰۸۹	۵
بقیهٔ Kotlin	۶,۴۴۴	۲۹

۱.۵ چند نکته که باید از اول بدانید

- **نام برنامه:** هم‌رسان. نام کلاس‌ها و سندها گاهی `Hamresan` است و نام بسته `ir.hamrasan.app`. هر دو املا به همین برنامه برمی‌گردند.
- **پکیج Kotlin هنوز `com.example.communication_super_app` است.** این عمداً عوض نشده. `applicationId` نام نصب‌شدهٔ برنامه است و `namespace` فقط نام پکیج کد. نام کانال‌ها هم با همین پیشوند شروع می‌شوند و باید در Dart و Kotlin حرف‌به‌حرف یکی بمانند.
- **هیچ نسخه‌ای هنوز منتشر نشده.** به همین دلیل پروژه مهاجرت پایگاه‌داده ندارد و هیچ کدی برای دادهٔ نسخه‌های قبلی نوشته نمی‌شود (بخش ۶).

- **بسیاری از رفتارها فقط روی گوشی واقعی دیده می‌شوند.** سامسونگ، شیائومی و نسخه‌های مختلف اندروید رفتار متفاوتی دارند. قانون‌هایی که در این سند می‌بینید اغلب نتیجهٔ همین خطاهای واقعی‌اند.

بخش ۲. راه‌اندازی محیط و دستورهای روزانه

۲.۱ چیزهایی که لازم دارید

ابزار	توضیح
Flutter	کانال stable (پروژه با نسخهٔ 3.47.1 ساخته می‌شود)، با Dart `^3.10.1`
Android Studio	جاوای داخلی آن (`jbr`) برای Gradle و ابزار ویندوزی کافی است
گوشی واقعی	برای پیامک و تماس لازم است. شبیه‌ساز برای بیشتر صفحه‌ها و آزمون پیامک تزریقی کافی است، اما رفتار واقعی اپراتور را نشان نمی‌دهد

در ویندوز، پیش از اجرای مستقیم `gradlew` مسیر جاوا را بدهید:

```
export JAVA_HOME="/c/Program Files/Android/Android Studio/jbr"
```

۲.۲ اجرای برنامه

```
flutter pub get
flutter run                          # فلیور commercial (پیش‌فرض pubspec)
flutter run --flavor organization    # نسخهٔ سازمانی
flutter run --release
```

> نسخهٔ سازمانی را روی گوشی اصلی‌تان نصب نکنید. versionCode آن دو میلیارد بالاتر است و بعد از نصب آن، نسخهٔ تجاری بدون حذف برنامه روی آن نصب نمی‌شود. برای آن از شبیه‌ساز یا گوشی یدکی استفاده کنید.

بعد از اجرا، برنامه می‌خواهد پیام‌رسان پیش‌فرض و تلفن پیش‌فرض شود. برای کار با بیشتر امکانات این دو نقش لازم‌اند.

۲.۳ دستورهای روزانه

کار	دستور
بررسی ایستا (باید صفر خطا بدهد)	`flutter analyze`
قالب‌بندی	`dart format lib test`
همهٔ آزمون‌های Dart	`flutter test`
یک فایل آزمون	`flutter test test/unit/phone_normalizer_test.dart`
آزمون‌های Kotlin	`cd android && ./gradlew :app:testCommercialDebugUnitTest`
آزمون ابزار ویندوزی	`android/gradlew -p tools/keybank :test`
آزمون پنل	`android/gradlew -p tools/keybank :panel:test`
APK فروشگاه	`scripts/release_build.sh`
APK سازمانی	`scripts/release_build.sh --flavor organization`

۲.۴ پرچم‌های زمان ساخت (`--dart-define`)

پرچم	کار
`SENTRY_DSN=…`	گزارش خطا را کامپایل می‌کند؛ کاربر هم باید در تنظیمات روشنش کند
`HAMRESAN_DEV_ANCHOR=true`	مرجع بانک کلید آزمایشی را معتبر می‌شمارد. فقط برای توسعه؛ اسکریپت انتشار APK دارای آن را رد می‌کند
`CARET_TRACE=true`	برای هر حرکت نشانگر متن یک خط `[caret]` در logcat می‌نویسد

برای دیدن لاگ‌های تشخیصی:

```
adb logcat -s flutter | grep -E '\[caret\]|\[redial\]'
```

۲.۵ نکته‌های ساخت

- **فقط `arm64-v8a`.** `abiFilters` و بلوک `packaging` فقط همین معماری را در APK نگه می‌دارند.
- **minSdk و targetSdk و compileSdk** از Flutter می‌آیند (minSdk اکنون ۲۴ است) و جاوا و Kotlin هر دو روی JVM 17 هستند. این دو باید با هم یکی بمانند، چون عدم تطابق جایی خطا می‌دهد که Kotlin و Java کلاس مشترک دارند.
- **`android/third_party/maven`** نسخه‌ای بررسی‌شده از بسته‌های `androidx.sqlite` است که `sqflite_sqlcipher` لازم دارد و Google Maven روی شبکهٔ توسعه نمی‌دهد. Gradle از این پوشه فقط همین گروه را می‌گیرد و هش فایل‌ها در `SHA256SUMS` هست.
- **`android/key.properties` و فایل keystore در git نیستند.** اگر نباشند، ساخت release با کلید debug امضا می‌شود و Gradle هشدار می‌دهد. چنین APKای نباید پخش شود. پشتیبان‌گیری از کلید با `scripts/backup_keystore.*` است.
- **R8 روشن است** (`isMinifyEnabled` و `isShrinkResources`). قانون‌ها در `android/app/proguard-rules.pro` هستند. هر کلاسی که با reflection صدا زده می‌شود باید آنجا نگه داشته شود.
- **امضا فقط v2 و v3** است. v1 و v4 عمداً خاموش‌اند.

۲.۶ ساختار پوشه‌ها در یک نگاه

```
lib/
  main.dart            نقطهٔ شروع
  core/                کد مشترک همهٔ بخش‌ها
  features/            یک پوشه برای هر قابلیت
android/app/src/main/  بخش بومی: Kotlin و مانیفست
android/app/src/test/  آزمون‌های JVM
test/unit, test/widget آزمون‌های Dart
tools/keybank/         ابزار خط فرمان و پنل صدور ویندوزی
scripts/               ساخت نسخهٔ انتشار و پشتیبان کلید
docs/                  سندها
assets/fonts/          قلم Vazirmatn
```

بخش ۳. تکنولوژی‌ها و کتابخانه‌ها

۳.۱ زبان‌ها

زبان	کجا	چرا
Dart / Flutter	همهٔ رابط کاربری و منطق برنامه	یک کد برای صفحه‌ها، سریع برای ساخت رابط فارسی
Kotlin	هر کاری که باید بدون باز بودن برنامه انجام شود یا API آن فقط بومی است: دریافت پیامک، تماس، رمزنگاری، Keystore، آلارم	Flutter در پس‌زمینه و هنگام بسته بودن برنامه اجرا نمی‌شود
SQL	پایگاه‌داده	پرس‌وجوهای صندوق پیام و جستجو

برنامه فقط اندروید است و نسخهٔ iOS ندارد. iOS اصلاً اجازهٔ جایگزینی برنامهٔ پیامک و تلفن را نمی‌دهد.

۳.۲ کتابخانه‌های Dart

کتابخانه	نسخه	کار
`flutter_bloc`	۹	مدیریت وضعیت
`equatable`	۲	مقایسهٔ eventها و stateها؛ stateهای برابر دوباره رسم نمی‌شوند
`sqflite`	۲	پایگاه‌دادهٔ اصلی
`sqflite_sqlcipher`	۳	پایگاه‌دادهٔ رمزشدهٔ بخش امن. فقط `lib/features/secure/` آن را import می‌کند
`flutter_contacts`	۱	خواندن و نوشتن دفترچهٔ تلفن گوشی
`permission_handler`	۱۲	گرفتن مجوزها، فقط از داخل `PermissionService`
`flutter_secure_storage`	۹	هش PIN، هش کد بازیابی، کد فعال‌سازی
`flutter_local_notifications`	۱۸	اعلان‌هایی که از Dart فرستاده می‌شوند. بیشتر اعلان‌های پیامک بومی‌اند
`shared_preferences`	۲	تنظیمات و وضعیت‌های کوچک
`crypto`	۳	SHA-256 برای کشیدن PIN و HMAC
`uuid`	۴	شناسهٔ ردیف‌ها
`url_launcher`	۶	باز کردن لینک در مرورگر
`path_provider`، `path`		مسیر فایل‌ها
`package_info_plus`	۸	شماره نسخه در «درباره برنامه»
`intl`، `flutter_localizations`		تاریخ و locale فارسی
`sentry`	۹	گزارش خطا، نسخهٔ خالص Dart

برای آزمون هم `flutter_test`، `bloc_test`، `mocktail` و `sqflite_common_ffi` (پایگاه‌داده در حافظه) استفاده می‌شوند.

۳.۳ کتابخانه‌های Kotlin

کتابخانه	کار
`org.bouncycastle:bcprov-jdk18on:1.86`	ML-KEM-768، ML-DSA-65، X25519، Ed25519، HKDF و Argon2id. خالص جاواست و روی همهٔ نسخه‌های اندروید یکسان کار می‌کند. نسخه‌اش ثابت است و هر ارتقا باید آزمون‌های KAT را دوباره بگذراند
`androidx.core:core-ktx`	کمکی‌های اندروید
`kotlinx-coroutines`	کار پس‌زمینه
`junit`	آزمون‌های JVM

۳.۴ کتابخانه‌هایی که عمداً استفاده نمی‌شوند

این فهرست مهم است، چون کسی که پروژه را نمی‌شناسد ممکن است دوباره آن‌ها را اضافه کند.

کتابخانه	چرا حذف شد	جایگزین
`call_log`	خودش مجوز می‌گیرد و کرش می‌کند	کد بومی `calllog/` و کانال `call_log`
`another_telephony`	همان مشکل	`SmsHandler.kt` و گیرنده‌های بومی
`image_picker`، `image_cropper`	همان مشکل، و نیاز به مجوز دوربین از داخل افزونه	`media/PhotoHandler.kt`
`sentry_flutter`	افزونهٔ Gradle آن به AGP 7.4.2 قفل است و ساخت خراب می‌شود	`sentry` خالص Dart، با هزینهٔ ندیدن کرش‌های Kotlin
`local_auth`، `androidx.biometric`	به `FragmentActivity` نیاز دارند، اما `MainActivity` یک `FlutterActivity` است	`BiometricPrompt` مستقیم در `security/BiometricGate.kt`
`camera`	عکس ورود ناموفق بدون پیش‌نمایش گرفته می‌شود	Camera2 در `security/`

> **هیچ افزونه‌ای در این برنامه حق گرفتن مجوز زمان اجرا را ندارد.** این افزونه‌ها فقط یک `MethodChannel.Result` نگه می‌دارند. اگر دو درخواست هم‌زمان شوند، خطای `IllegalStateException: Reply already submitted` رخ می‌دهد که در حلقهٔ اصلی است و گرفته نمی‌شود، پس برنامه کرش می‌کند. پیش از اضافه کردن هر افزونه این را در کد آن بررسی کنید.

۳.۵ تکنیک‌های رمزنگاری به زبان ساده

نام	برای چه استفاده می‌شود
X25519	توافق کلید کلاسیک بین دو عضو
ML-KEM-768	توافق کلید مقاوم در برابر رایانهٔ کوانتومی (FIPS 203)
Ed25519 و ML-DSA-65	امضای مرجع روی گواهی اعضا
AES-256-GCM	رمز کردن پیام و هر داده‌ای که ذخیره می‌شود
HKDF	ساختن کلیدهای مشتق‌شده
Argon2id	ساختن کلید از گذرواژهٔ فایل کلید
PBKDF2	ساختن کلید از PIN بخش امن (۲۱۰٬۰۰۰ دور)

نام	برای چه استفاده می‌شود
SHA-256 با ۶۰٬۰۰۰ دور	ذخیرهٔ PIN برنامه
BLAKE3	کد فعال‌سازی. پیاده‌سازی Dart آن در `core/utils/blake3.dart` است
SQLCipher	رمز کردن کامل فایل `secure.db`
Android Keystore	نگهداری کلیدی که از گوشی بیرون نمی‌آید

بخش ۲۲ این‌ها را به‌ترتیب کاربرد توضیح می‌دهد.

۳.۶ قلم و زبان

قلم برنامه Vazirmatn است و چهار وزن آن در `assets/fonts/` قرار دارد. متن‌ها مستقیم و به فارسی در کد نوشته شده‌اند و لایهٔ ترجمه وجود ندارد. برنامه `MaterialLocalizations` فارسی ندارد، پس برچسب بعضی ابزارهای Flutter، مثل نوار انتخاب متن، دستی داده می‌شوند.

بخش ۴. معماری کلی

۴.۱ تصویر بزرگ

هم‌رسان یک برنامهٔ Flutter تک‌ماژولی است که قابلیت‌هایش از هم جدا هستند و نیمهٔ بومی بزرگی دارد. داده از سه جا می‌آید:

۱. **SQLite:** پایگاه‌دادهٔ اصلی، و `secure.db` رمزشدهٔ بخش امن.

۲. **کانال‌های پلتفرم:** Kotlin برای پیامک، تماس، فهرست تماس، سیم‌کارت، بخشی از دفترچهٔ تلفن، Keystore و رمزنگاری.

۳. **افزونه‌های گوشی:** مثل `flutter_contacts` و `flutter_secure_storage`.

۴.۲ لایه‌ها

```
Widget ──event──▶ BLoC ──▶ Repository / Service ──┬──▶ SQLite
   ▲                │                             ├──▶ کانال ──▶ Kotlin
   └────state───────┘                             └──▶ افزونه
```

قانون این لایه‌ها ساده است:

- ویجت هیچ‌وقت مستقیم سراغ repository یا کانال نمی‌رود.
- BLoC هیچ‌وقت رابط نمی‌سازد.
- Repository دسترسی به داده است و Service پل به سمت بومی یا هماهنگ‌کنندهٔ چند کار.

هر قابلیت در `lib/features/<name>/` این ساختار را دارد:

```
bloc/          event، state و خود bloc
models/        مدل‌های تغییرناپذیر با Equatable
repositories/  دسترسی به SQLite، SecureStore یا افزونه
screens/       صفحه‌ها (و widgets/ برای تکه‌های جداشده)
services/      پل بومی و هماهنگی، در صورت نیاز
```

نام‌گذاری هم ثابت است: `<f>_bloc.dart`، `<f>_event.dart`، `<f>_state.dart`، `<thing>_screen.dart`، `_repository.dart` و `_service.dart`.

۴.۳ مسیر شروع برنامه

```
main()
 ├─ AppEdition.verifyBuildFlavor()      فلیور ناشناخته: برنامه بالا نمی‌آید
 └─ CrashReporting.runApp
     ├─ NotificationService().initialize()
     ├─ قفل جهت صفحه روی عمودی
     └─ runApp(MyApp)
         └─ AppBlocProviders            همهٔ BLoCها
             └─ BlocBuilder<ThemeBloc>  پوسته
                 └─ BlocBuilder<SettingsBloc>  نوع تقویم
                     └─ MaterialApp (navigatorKey: appNavigatorKey)
                         ├─ builder: ReturnToCallBar → SecureSessionGuard → AppLockWrapper
                         └─ home: CallUiCoordinator
                               └─ OrganizationActivationGate
                                   └─ AuthWrapperScreen
```

```
                    └ PermissionGate
                        └ DefaultAppGate
                            └ MainNavigation
```

هر حلقه دلیل مشخصی دارد:

حلقه	کار	چرا در این جاست
`ReturnToCallBar`	نوار «بازگشت به تماس» بالای هر صفحه	روی همهٔ صفحه‌ها دیده می‌شود، پس بیرون navigator است
`SecureSessionGuard`	بستن بخش امن هنگام رفتن به پس‌زمینه	باید همیشه زنده باشد
`AppLockWrapper`	قفل دوبارهٔ برنامه بعد از مدت «قفل خودکار»	قفل یک route است و روی همهٔ صفحه‌ها می‌آید
`CallUiCoordinator`	باز کردن صفحهٔ تماس ورودی و جاری	بالای احراز هویت است تا روی صفحهٔ PIN هم بشود تماس را جواب داد
`OrganizationActivationGate`	در نسخهٔ سازمانی، پیش از کد فعال‌سازی هیچ چیز باز نمی‌شود	حتی PIN و مجوزها هم پشت آن‌اند. در نسخهٔ تجاری فقط رد می‌شود
`AuthWrapperScreen`	ساختن یا وارد کردن PIN	بر اساس state در `AuthBloc`
`PermissionGate`	گرفتن همهٔ مجوزها یک‌جا	تنها جای گرفتن مجوز. چند تب هم‌زمان مجوز می‌خواستند و برنامه کرش می‌کرد
`DefaultAppGate`	گرفتن نقش پیام‌رسان و تلفن پیش‌فرض	پیش از رسیدن به تب‌ها
`MainNavigation`	چهار تب در یک `IndexedStack`	همهٔ تب‌ها زنده می‌مانند

۴.۴ چند قانون برای شروع سریع

- **هیچ چیز در مسیر ورود چرخانک نمی‌کشد.** دروازه‌ها یک `Scaffold` خالی نشان می‌دهند و کارشان در چند فریم تمام می‌شود. چرخانک باعث می‌شد هر اجرا «در حال بارگذاری» به نظر برسد.
- `hasAllRequiredPermissions` **چهار خواندن را با** `Future.wait` **انجام می‌دهد**، نه یکی‌یکی.
- **«اخیر» و صندوق از SQLite نقاشی می‌شوند.** اولین نقاشی نام‌ها را از جدول `contact_name_cache` می‌گیرد. همگام‌سازی با گوشی جدا و بعد از آن اجرا می‌شود.
- **یک صندوق خالی هنگام اولین ورود پیام‌ها این را می‌گوید** (`ThreadsLoaded.syncing` و `MessagesImportingState`)، چون «پیامکی نیست» روی گوشی پر از پیام مثل از بین رفتن داده به نظر می‌رسد.
- روی گوشی آزمون (سامسونگ A33)، فهرست «اخیر» نام‌دار حدود یک ثانیه بعد از `am start` آماده است.

۴.۵ انتخاب تب هنگام باز شدن

`MainNavigation` هنگام باز شدن برنامه تب را انتخاب می‌کند: اگر در نبود کاربر پیامک رسیده «پیام‌ها»، اگر تماس بی‌پاسخ داشته «اخیر»، و اگر هر دو باشد هرکدام که تازه‌تر است. نقطهٔ مقایسه `last_foreground_at` است که در هر رویداد خروج از حالت resumed ذخیره می‌شود. پس «تازه» یعنی بعد از آخرین باری که کاربر برنامه را دیده، نه «خوانده‌نشده». این کار پرس‌وجوی جدیدی ندارد: به stateهای `MessageBloc` و `CallLogBloc` گوش می‌دهد، مدت کوتاهی منتظر پاسخ هر دو می‌ماند و بعد قطع می‌کند. اگر کاربر از یک اعلان وارد شده باشد، تصمیم بسته می‌شود تا تب زیر گفتگوی باز عوض نشود.

۴.۶ ناوبری

کلید سراسری `appNavigatorKey` در `core/navigation/` است و هر چیزی که باید خارج از درخت ویجت صفحه‌ای باز کند از آن استفاده می‌کند. لینک‌های عمیق اعلان‌ها از `DeepLinkService` می‌گذرند و تا پایین آمدن قفل منتظر می‌مانند (`AppLock.whenDown`)، تا پیام از پشت PIN خوانده نشود.

۴.۷ جایی برای هر چیز تازه

چه می‌خواهید اضافه کنید	کجا
صفحه‌ای برای قابلیت موجود	`features/<f>/screens/`، ریشهٔ RTL، همان BLoC
دادهٔ ماندگار	جدول در `DatabaseHelper._createDB` (اگر محرمانه است در `secure.db`)، مدل، repository
هر کار بومی	اول فهرست آینه‌ها در بخش ۷ را ببینید
کد مشترک	`lib/core/` فقط وقتی دو قابلیت از آن استفاده کنند
شبکه و سرور	وجود ندارد. اگر روزی اضافه شود، `core/network/` و تزریق به repositoryها؛ و توجه کنید که نسخهٔ سازمانی با این قول تحویل می‌شود که با سرور شخص ثالث حرف نمی‌زند

بخش ۵. مدیریت وضعیت با BLoC

۵.۱ قانون کلی

برنامه فقط از `flutter_bloc` استفاده می‌کند. Riverpod، Provider و GetX وجود ندارند. وضعیت موقت یک ویجت، مثل کنترلر متن یا باز بودن یک برگه، در `StatefulWidget` می‌ماند. هر چیزی که مشترک یا ماندگار است از BLoC می‌گذرد.

همهٔ BLoCها یک بار و در `core/bloc_providers/app_bloc_providers.dart` ساخته می‌شوند. **هیچ صفحه‌ای `BlocProvider` خودش را نمی‌سازد.** دلیلش این است که تب‌ها در `IndexedStack` زنده می‌مانند و یک داده در چند تب لازم است. هزینه‌اش هم این است که همهٔ BLoCها تا پایان اجرا در حافظه‌اند، که برای این اندازهٔ برنامه پذیرفتنی است.

```
context.read<XBloc>()     // فرستادن event
context.watch<XBloc>()    // بازسازی با هر تغییر state
BlocBuilder<XBloc, XState>(buildWhen: …, builder: …)   // بازسازی محدود
BlocListener<XBloc, XState>(listenWhen: …, listener: …) // ناوبری و پیام کوتاه
```

ناوبری و نوار پیام همیشه در `BlocListener` است، نه در `builder`.

۵.۲ فهرست BLoCها

ترتیب این جدول همان ترتیب ساخت است و اهمیت دارد: هر BLoC فقط می‌تواند BLoCهای بالای خودش را بخواند. `test/widget/app_bloc_providers_test.dart` همین ترتیب را می‌سازد و آزمایش می‌کند. `lazy: false` یعنی BLoC از همان ابتدا ساخته می‌شود و منتظر اولین خواندن نمی‌ماند.

BLoC	زودهنگام	event اولیه	کار
`ThemeBloc`		`LoadTheme`	پوستهٔ روشن و تیره
`EditionBloc`	بله	`LoadEdition`	نسخه و وضعیت فعال‌سازی
`AuthBloc`		`CheckAuthStatus`	PIN برنامه، کد بازیابی
`SecureSessionBloc`	بله	`SecureSessionRefresh`	باز و بسته بودن بخش امن
`KeyBankBloc`			فایل‌های کلید
`SecureMessagesBloc`			پیام و گروه رمز
`SimBloc`		`LoadSims`	فهرست سیم‌کارت‌ها
`ContactBloc`			دفترچهٔ تلفن
`MessageBloc`		با اولین `LoadThreads`	صندوق و گفتگو
`DraftBloc`			پیش‌نویس‌ها و دسته‌ها
`TemplateBloc`			قالب‌ها
`ScheduledMessageBloc`	بله	`LoadScheduled`	زمان‌بندی؛ آلارم را دوباره تنظیم می‌کند
`CallLogBloc`			«اخیر»
`HiddenBloc`	بله		دفترچهٔ مخفی؛ همراه بخش امن باز و بسته می‌شود
`DialerBloc`		مخاطبین در سازنده	شماره‌گیر و وضعیت تماس

BLoC	زودهنگام	event اولیه	کار
`FavoritesBloc`		`LoadFavorites`	موردعلاقه‌ها
`SearchBloc`			جستجوی یکپارچه
`SettingsBloc`		`LoadSettings`	تنظیمات
`BlockedNumbersBloc`		`LoadBlocked`	شماره‌های مسدود

`HiddenBloc` سه تابع هم می‌گیرد (`importHistory`، `renamePeer`، `onHistoryMoved`)، چون وقتی شماره‌ای مخفی می‌شود باید تاریخچه‌اش به بخش امن منتقل شود و `SecureMessagesBloc` و `MessageBloc` خبردار شوند.

۵.۳ eventها

برای پیدا کردن کار هر BLoC کافی است eventهایش را ببینید:

BLoC	eventها
`MessageBloc`	`LoadThreads`، `RefreshContactNames`، `LoadMoreThreads`، `SyncDeviceMessages`، `DeviceSyncFinished`، `ThreadChangedExternally`، `LoadMessages`، `LoadMoreMessages`، `SendMessage`، `SendGroupMessage`، `ReceiveMessage`، `MessageSentExternally`، `MessageStatusChanged`، `RetryMessage`، `DeleteMessage`، `DeleteThread`، `DeleteThreads`، `DeleteMessages`، `ArchiveThreads`، `PinThread`، `SetThreadRead` و چند event دیگر
`DialerBloc`	`DialerNumberPressed`، `DialerNumberSet`، `DialerNumberCleared`، `DialerNumberDeleted`، `DialerCaretMoved`، `DialerLoadContacts`، `DialerFilterContacts`، `MakeCall`، `EndCall`، `AnswerCall`، `RejectCall`، `ToggleMute`، `ToggleSpeaker`، `HoldCall`، `DtmfKeyDown`، `DtmfKeyUp`، `MergeCalls`، `SwapCalls`، `SelectAudioRoute`، `SyncCallState`، `CancelAutoRedial`، `AutoRedialDue`، `CallEventReceived`
`CallLogBloc`	`LoadCallLogs`، `RefreshCallLogs`، `LoadMoreCallLogs`، `SyncCallLogs`، `RefreshCallLogContactNames`، `DeleteCallLog`، `DeleteCallLogs`، `ClearCallLogs`
`ScheduledMessageBloc`	`LoadScheduled`، `SaveScheduled`، `CancelScheduled`، `DeleteScheduled`، `DeliverDueScheduled`، `SendScheduledNow`
`AuthBloc`	`CheckAuthStatus`، `SetPin`، `ValidatePin`، `Authenticate`، `BiometricUnlocked`، `Logout`، `ClearAuth`، `SkipAuthSetup`، `DisableAuth`، `LockApp`، `RecoverWithCode`، `RegenerateRecoveryCode`
`SettingsBloc`	`LoadSettings`، `SetBoolSetting`، `SetCalendarType`، `SetAutoRedialAttempts`، `SetAdPrefixes`، `SetNumberLists`، `SetMessageTextScale`
`DraftBloc`	`LoadDrafts`، `SaveDraft`، `DeleteDraft`، `DeleteDrafts`، `PinDrafts`، `MoveDraftsToCategory`، `AddCategory`، `RenameCategory`، `DeleteCategory`، `DeleteCategories`، `PinCategories`

BLoC	eventها
`SecureSessionBloc`	`SecureSessionRefresh`، `SecureCreateRequested`، `SecureUnlockRequested`، `SecureBiometricToggleRequested`، `SecureLockRequested`، `SecureAppBackgrounded`، `SecureAppResumed`، `SecureBiometricUnlockRequested`، `SecureVerifyPinRequested`، `SecureChangePinRequested`، `SecureResetRequested`
`SecureMessagesBloc`	`SecureOpenThread`، `SecureCloseThread`، `SecureSendText`، `SecureRetry`، `SecureDeleteForBoth`، `SecureDeleteLocally`، `SecureDeleteConversation`، `SecureLoadPeers`، `SecureLookupNumber`، `SecureRetryChannel`، `SecureStartEncrypted`، `SecureStartConversation`، `SecureOpenWith`، `SecureImportHistory`، `SecureRenamePeer`، `SecureOpenGroup`، `SecureCloseGroup`، `SecureSendGroupText`، `SecureCreateGroup`، `SecureEditGroup`، `SecureDeleteGroup`، `SecureGroupDeleteForAll`، `SecureGroupDeleteLocally`، `SecureRetryGroupMessage`، `SecureLoadGroupCandidates`، `SecureAddGroupCandidates`، `SecureSetCover`، `SecureSetSim`، `SecureSetReceipts`، `SecureSetPinned`، `SecureLoadOlder`
`HiddenBloc`	`HiddenRefresh`، `HiddenSaveContact`، `HiddenDeleteContact`، `HiddenDeleteCalls`، `HiddenClearCalls`، `HiddenLoadMoreCalls`
`KeyBankBloc`	`KeyBankRefresh`، `KeyBankImportFile`، `KeyBankRemoveKey`

۵.۴ قرارداد event و state

- eventها و stateها از `Equatable` ارث می‌برند تا stateهای یکسان دوباره رسم نشوند.
- stateها معمولاً سلسله‌مراتبی‌اند: `Initial`، `Loading`، یک یا چند حالت بارگذاری‌شده و `Error`. مثلاً: `MessageInitial → MessageLoading → ThreadsLoaded | MessagesLoaded | MessageSent | MessageSendFailed | MessageError`.
- eventها فعل امری‌اند: `LoadThreads`، `SendMessage`، `ArchiveThreads`.
- هر BLoC وابستگی‌هایش را از سازنده می‌گیرد و پیش‌فرض آن نسخهٔ واقعی است. پس `AppBlocProviders` بدون آرگومان آن را می‌سازد و آزمون‌ها mock تزریق می‌کنند.

۵.۵ نگهبان‌های state

این رفتارها زائد به نظر می‌رسند ولی **نباید ساده شوند**:

نگهبان	کجا	چه می‌کند
نرفتن به حالت بارگذاری در به‌روزرسانی پس‌زمینه	`MessageBloc._onLoadThreads`	`MessageLoading` فقط وقتی فرستاده می‌شود که state فعلی `ThreadsLoaded` یا `MessagesLoaded` نباشد. وگرنه پیامک تازه گفتگو را سفید می‌کرد
ماندن گفتگو هنگام پیام گفتگوی دیگر	`MessageBloc._onReceiveMessage`	پیام گفتگوی دیگر `MessagesLoaded` را عوض نمی‌کند

نگهبان	کجا	چه می‌کند
بازسازی محدود	`MainNavigation`	نشان خوانده‌نشده با `buildWhen` فقط وقتی جمع عوض شود رسم می‌شود
قطع خوش‌بینانهٔ تماس	`DialerBloc._onEndCall`	رابط فوراً به حالت عادی برمی‌گردد و `endCall()` بومی منتظر نمی‌ماند. همین بازگشت زودهنگام باعث می‌شود تماسی که کاربر قطع کرده تماس مجدد خودکار نخورد، چون `DISCONNECTED` بعدی وضعیت را از قبل بی‌کار می‌بیند
تماس مجدد، وضعیت کنار تماس است	`DialerState.autoRedial`	در شمارش معکوس تماسی وجود ندارد و مسیرهای پایان تماس اجرا می‌شوند. `copyWith` این فیلد را فقط با `clearAutoRedial` پاک می‌کند
پرچم در حال اجرا برای صفحه‌بندی	`_loadingMoreThreads`، `_loadingMoreMessages`	محرک صفحه‌بندی در هر صفحه چند بار اجرا می‌شود

۵.۶ کش‌های هر اجرا

کش	محل	چرا
`_hasImported`	`MessageBloc` (static)	همگام‌سازی کامل یک بار در هر اجرا. فقط بعد از موفقیت تنظیم می‌شود تا رد مجوز در اجرای اول دوباره تلاش شود
`_cachedPhoneToName`	`MessageBloc`	پیدا کردن نام بدون خواندن دوبارهٔ دفترچهٔ تلفن
`_cache`	`CallLogService` (static)	پرهیز از خواندن دوبارهٔ فهرست تماس
`_listening`	`SmsService`	ثبت `EventChannel` فقط یک بار
`contact_name_cache`	جدول SQLite	نام‌دار بودن اولین نقاشی صندوق و «اخیر»

۵.۷ یک BLoC بزرگ را چطور بخوانیم

`message_bloc.dart` با ۱٬۱۷۳ خط یکی از بزرگ‌ترین‌هاست. برای خواندنش این ترتیب را پیشنهاد می‌کنیم: اول فهرست `on<…>` در سازنده را ببینید. بعد `_onLoadThreads` و `_startBackgroundSync` را بخوانید تا نقاشی اول و همگام‌سازی را بفهمید. بعد `_onSendMessage` و `_onReceiveMessage` را بخوانید. بقیه کارهای سادهٔ پایگاه‌داده‌اند.

بخش ۶. پایگاه‌داده و ذخیره‌سازی

۶.۱ چهار جای نگهداری داده

کجا	چه چیزی	چه کسی می‌خواند
`communication_app.db` (SQLite)	پیام، تماس، پیش‌نویس، زمان‌بندی و بقیه	Dart و Kotlin. Kotlin حتی وقتی برنامه بسته است و بدون هیچ کلیدی آن را باز می‌کند
`secure.db` (SQLCipher)	پیام رمز، کلیدها، دفترچهٔ مخفی، عکس ورود ناموفق	فقط Dart، و فقط وقتی بخش امن باز است
`SharedPreferences`	تنظیمات و وضعیت‌های کوچک	Dart. بخشی از آن‌ها در فایل بومی هم کپی می‌شوند
`flutter_secure_storage`	PIN کشیده‌شده، هش کد بازیابی، کد فعال‌سازی	Dart

> **هیچ دادهٔ محرمانه‌ای در پایگاه‌دادهٔ اصلی نوشته نمی‌شود.** Kotlin این فایل را بدون کلید باز می‌کند، پس رمزکردن آن ممکن نیست. `secure_queue` و `sealed_queue` فقط متن رمزشده‌ای را نگه می‌دارند که منتظر باز شدن بخش امن است.

۶.۲ مهاجرت وجود ندارد

هیچ نسخه‌ای هنوز به کسی نرسیده است. پس هر دو پایگاه‌داده نسخهٔ ۱ هستند و کل طرحشان در `DatabaseHelper._createDB` و `SecureStore._onCreate` است. اگر جدولی را عوض می‌کنید، همان‌جا عوض کنید و برنامه را روی گوشی‌های آزمون دوباره نصب کنید. هیچ کدی نباید «ردیفی که نسخهٔ قبلی نوشته» را مدیریت کند. اولین تغییر بعد از انتشار واقعی، `AppConstants.databaseVersion` را بالا می‌برد و `onUpgrade` اضافه می‌کند.

۶.۳ جدول‌های پایگاه‌دادهٔ اصلی

جدول	ستون‌های اصلی	توضیح
`messages`	`id`، `thread_id`، `contact_id`، `phone_number`، `body`، `type`، `status`، `timestamp`، `is_read`، `is_deleted`، `device_sms_id`، `is_starred`، `subscription_id`، `search_indexed`	آینهٔ پیامک‌های گوشی
`message_search`	جدول مجازی FTS5 با `trigram`	نمایهٔ جستجو. روی اندروید خاموش است (بخش ۹)
`call_logs`	`id`، `contact_id`، `phone_number`، `call_type`، `duration`، `timestamp`، `subscription_id`	آینهٔ فهرست تماس
`contacts`	`id`، `name`، `phone_number`، `email`، `is_favorite`، `ringtone_uri`، ...	از قدیم مانده و چیزی در آن نمی‌نویسد. مخاطبان در دفترچهٔ گوشی‌اند
`favorites`	`id`، `phone_number`، `normalized` (یکتا)، `name`، `contact_id`، `created_at`	موردعلاقه‌ها

جدول	ستون‌های اصلی	توضیح
`blocked_numbers`	`id`، `phone_number`، `normalized` (یکتا)، `created_at`، `is_spam`، `reported_at`	مسدودی و گزارش هرزنامه در یک جدول
`archived_threads`	`thread_id`، `archived_at`	گفتگوهای بایگانی
`pinned_threads`	`thread_id`، `pinned_at`	سنجاق، حداکثر ۵ (`AppConstants.maxPinned`)
`unread_marks`	`thread_id`، `marked_at`	«علامت‌گذاری نخوانده»
`drafts`	`id`، `title`، `body`، `category_id`، `updated_at`، `is_pinned`	پیش‌نویس‌ها
`message_categories`	`id`، `name`، `created_at`، `is_pinned`	دسته‌های پیش‌نویس
`message_templates`	`id`، `title`، `body`، `use_contact_name`، `is_pinned`، `updated_at`	قالب‌ها، با چند قالب اولیه
`scheduled_messages`	`id`، `phone_number`، `contact_name`، `body`، `scheduled_at`، `repeat`، `repeat_every`، `weekdays`، `jitter`، `end_type`، `end_date`، `max_occurrences`، `occurrence_count`، `status`، `created_at`، `attempt_count`، `next_attempt_at`، `last_error`، `claim_token`، `claimed_at`، `subscription_id`	زمان‌بندی
`thread_sim`	`thread_id`، `subscription_id`، `updated_at`	سیم‌کارت هر گفتگو
`speed_dial`	`position` (کلید)، `phone_number`، `name`، `contact_id`، `updated_at`	شماره‌گیری سریع
`message_groups`	`id`، `title`، `created_at`، `updated_at`	گروه پیامکی
`message_group_members`	`group_id`، `normalized`، `phone_number`، `display_name`، `added_at`	اعضای گروه
`message_group_targets`	`message_id`، `normalized`، `phone_number`، `status`، `device_sms_id`، `subscription_id`، `tracking_id`	یک ردیف برای هر گیرندهٔ هر پیام گروهی
`contact_name_cache`	`normalized` (کلید)، `name`، `contact_id`	کش دفترچهٔ تلفن برای نقاشی اول

جدول	ستون‌های اصلی	توضیح
`secure_queue`	`id`، `address`، `body`، `timestamp`، `subscription_id`، `received_at`؛ یکتا روی `(address, body)`	پیامک `#E:` رسیده در انتظار بخش امن
`sealed_queue`	`id`، `kind`، `blob`، `created_at`	رکورد مهرشدهٔ دفترچهٔ مخفی

۶.۴ قانون‌های طرح پایگاه‌داده

- **`messages.thread_id` شمارهٔ عادی‌شده است.** فقط برای گروه مقدارش `'g:' || message_groups.id` است (`GroupThread`). به همین دلیل صفحه‌بندی، سنجاق، بایگانی و جستجو بدون هیچ شرطی روی گروه هم کار می‌کنند. پس هیچ کدی نباید شناسهٔ گفتگو را بدون بررسی `isGroup` به یک تابع نمایش شماره بدهد.
- **ایندکس یکتای `messages`** روی `(phone_number, body, timestamp, type)` است. همهٔ درج‌های گروهی باید `ConflictAlgorithm.ignore` داشته باشند تا تکراری‌ها بی‌صدا رد شوند. یک پیامک ممکن است دو بار برسد: زنده از گیرنده و بعد از همگام‌سازی. یک پنجرهٔ ۵ ثانیه‌ای حافظه در `SmsService` هم تحویل دوتایی سریع را پیش از پایگاه‌داده می‌گیرد.
- **`messages.is_read`:** هر ردیف ارسالی با `is_read = 1` نوشته می‌شود. پس `unread_count` همهٔ نوع‌ها را می‌شمارد و نباید `type = 'received'` را شرط کند. `markThreadAsRead` هم روی همهٔ نوع‌ها اجرا می‌شود، وگرنه گفتگویی که فقط پیام ارسالی دارد برای همیشه پررنگ می‌ماند.
- **`messages.device_sms_id`** شناسهٔ ردیف در `content://sms` است و کلید همگام‌سازی و حذف. مقدار null یعنی ردیف شناخته‌شده‌ای در گوشی نیست و همگام‌سازی آن را حذف نمی‌کند.
- **`blocked_numbers.normalized`** شناسهٔ کانونی گفتگوست (`09xxxxxxxxx`) و فقط `BlockedNumberModel.normalize` باید آن را بسازد. این ستون قبلاً رقم‌های خام بود، ردیف `989121234567` ذخیره می‌شد و همهٔ جستجوها `09121234567` را می‌پرسیدند. نتیجه این بود که مسدودی اصلاً کار نمی‌کرد.
- **`favorites.normalized`** هم همین کلید است و `FavoriteModel.normalize` برای متن بدون رقم خالی برمی‌گرداند.
- **`contact_name_cache` فقط کش است.** فقط خواندن کامل دفترچه آن را می‌نویسد و هر بار **کل مجموعه** را در یک تراکنش جایگزین می‌کند. اگر فقط upsert می‌کرد، نام مخاطب حذف‌شده می‌ماند.
- **`unread_marks` هیچ پیامی را علامت نمی‌زند.** علامت دستی و پیام واقعی دو واقعیت جدا هستند. اگر با `is_read = 0` نشان داده شوند، نشان خوانده‌نشده عدد اشتباه می‌گوید (بخش ۹).
- **`drafts.is_pinned`:** `DraftRepository.upsertDraft` ردیف را REPLACE می‌کند، پس `DraftBloc._onSave` پرچم قبلی را منتقل می‌کند. بدون این کار، ویرایش پیش‌نویس سنجاق را برمی‌داشت.
- **حذف پیام نرم است** (`is_deleted = 1`). ردیف حذف‌شده با `device_sms_id` می‌ماند تا همگام‌سازی آن را دوباره وارد نکند (بخش ۸).

۶.۵ ایندکس‌ها

ایندکس	برای چه
`messages(thread_id)`	صفحهٔ گفتگو
`messages(timestamp DESC)`	ترتیب زمانی

ایندکس	برای چه
`messages(is_read)`	شمارش خوانده‌نشده
`idx_messages_search (is_deleted, timestamp DESC)`	جستجوی پشت‌سرهم از تازه به قدیم که ردیف‌های حذف‌شده را رد می‌کند
`idx_messages_search_pending`	کار نمایه‌سازی
`idx_messages_is_starred`	پیام‌های ستاره‌دار
`call_logs(timestamp DESC)`	«اخیر»
`idx_group_targets_device`، `idx_group_targets_tracking`	همگام‌سازی و گزارش تحویل گروه
`idx_templates_sort`	ترتیب قالب‌ها

۶.۶ جدول‌های `secure.db`

فقط `SecureStore` این فایل را باز می‌کند. جزئیات کلید آن در بخش ۲۱ است.

جدول	ستون‌ها	کار
`secure_meta`	`key`، `value`	اطلاعات کلی، مثل زمان ساخت
`kb_own_keys`	`authority_id`، `key_id`، `phone`، `generation`، `secret`، `cert`، `imported_at`	کلید و گواهی خود عضو، یکی برای هر مرجع
`sm_conversations`	`phone`، `name`، `peer_key_id`، `peer_public`، `own_key_id`، `created_at`، `last_at`، `unread`، `seen_sid`، `seen_counter`، `listed`، `peer_generation` و چند ستون تنظیم	گفتگوهای رمز
`sm_sessions`	`phone`، `sid`، `pending`، `state`، `created_at`، `kind`، `offered_at`، `send_failed`	نشست‌های رمزنگاری
`sm_answered_requests`	`digest`، `answered_at`	نگهبان درخواست تکراری
`sm_messages`	`id`، `phone`، `outgoing`، `body`، `timestamp`، `status`، `sid`، `counter`، `delete_after_seen`، `seen_at`، `parts`، `plain`، `ttl`	پیام‌ها. `sm_messages_wire` روی `(phone, outgoing, sid, counter)` یکتاست
`sm_inbox`	`id`، `address`، `body`، `timestamp`، `received_at`، `attempts`	پیام رسیده که هنوز باز نشده
`sg_groups`	`id`، `name`، `mode`، `version`، `creator`، `pending_info`، `left_group`، `created_at`، `last_at`، `unread`	گروه‌های رمز
`sg_members`	`group_id`، `phone`، `key_id`، `position`، `info_version`، `removed`	اعضا
`sg_messages`	`id`، `group_id`، `sender`، `body`، `timestamp`، `status`، `sid`، `counter`، `delete_after_seen`، `seen_at`، `ttl`	پیام گروه
`sg_deliveries`	`message_id`، `phone`، `status`، `sid`، `counter`، `parts`	تحویل به هر عضو

جدول	ستون‌ها	کار
`hc_contacts`	`id` ، `name` ، `note` ، `created_at` ، `updated_at`	مخاطب مخفی
`hc_numbers`	`id` ، `contact_id` ، `phone` ، `label` ، `position`	شماره‌های مخاطب مخفی
`hc_calls`	`id` ، `phone` ، `call_type` ، `timestamp` ، `duration` ، `account`	تماس‌های مهرشده
`ip_photos`	`id` ، `taken_at` ، `camera` ، `failed_attempts` ، `jpeg` ، `seen`	عکس ورود ناموفق

۶.۷ کلیدهای `flutter_secure_storage`

کلید	محتوا
`app_pin`	PIN به شکل `v2:<salt>:<hash>` . کد Kotlin فقط **وجود** این ورودی را می‌بیند، نه مقدارش
`auth_type` ، `auth_skipped` ، `auth_pin_length`	نوع احراز هویت، رد کردن تنظیم PIN، طول PIN
`auth_recovery_code`	هش کد بازیابی
`auth_biometric_unlock` ، `auth_biometric_sentinel`	باز کردن با اثر انگشت
`inter_org_activation_code` ، `org_activation_code`	کدهای فعال‌سازی

۶.۸ کلیدهای مهم `SharedPreferences`

کلید	کار
`theme_mode`	پوسته
`last_foreground_at`	آخرین زمان حضور کاربر، برای انتخاب تب
`sim_default_sms` ، `sim_default_voice`	سیم‌کارت پیش‌فرض برنامه
`contact_save_target`	محل ذخیرهٔ پیش‌فرض مخاطب
`contact_filters_hidden`	فیلترهای پنهان‌شده
`pinned_contacts`	مخاطبان سنجاق‌شده
`blocked_system_snapshot`	عکس فوری فهرست مسدود گوشی برای همگام‌سازی سه‌طرفه
`set_list_*`	«فهرست سیاه و سفید». یک نسخه در فایل بومی `number_lists` هم هست
`set_spam_prompt_dismissed`	پاسخ «نه» به پیشنهاد گزارش هرزنامه
`group_sms_notice_seen_v1`	بنر یک‌بارهٔ گروه
`emoji_recents_v1`	ایموجی‌های اخیر
`crash_reporting_enabled`	«تشخیص خطا»

کلیدهای تنظیمات در `SettingsBloc` و `BoolSetting` تعریف شده‌اند. چند کلید در `secure_meta` هم هستند، مثل `cover_text`، `sms_sim` و `send_seen_receipts`.

۶.۹ کد دسترسی به پایگاه‌داده

- `DatabaseHelper` (`core/database/database_helper.dart`) یک singleton است. برای آزمون، `DatabaseHelper.databasePathOverride = inMemoryDatabasePath` و `resetForTesting()` دارد.
- هیچ ویجتی SQL نمی‌نویسد. پرس‌وجوها در repositoryها هستند: `MessageRepository`، `CallLogRepository`، `GroupRepository`، `ScheduledMessageRepository`، `DraftRepository`، `TemplateRepository`، `FavoritesRepository`، `BlockedNumbersRepository`، `ThreadSimRepository`، `SpeedDialRepository`.
- Kotlin با `android.database.sqlite` همان فایل را باز می‌کند و تمام نوشتن‌هایش از `withDbRetry` می‌گذرد، چون ممکن است اتصال Dart قفل نوشتن را چند ثانیه نگه داشته باشد.

بخش ۷. پل Dart و Kotlin

۷.۱ چرا این‌قدر کد بومی داریم

سه دلیل دارد. Flutter وقتی برنامه بسته است اجرا نمی‌شود، اما پیامک، آلارم و تماس باید همان موقع هم کار کنند. بعضی APIها، مثل Keystore، Telecom و `RoleManager`، در Flutter نیستند. و افزونه‌های موجود اغلب خودشان مجوز می‌گیرند که در این پروژه ممنوع است.

۷.۲ کانال‌ها

نام همهٔ کانال‌ها با `com.example.communication_super_app/` شروع می‌شود و باید در دو طرف حرف‌به‌حرف یکی باشد. این نام‌ها فقط متن‌اند و ارتباطی با نام بسته ندارند.

کانال	سمت Kotlin	سمت Dart	کار
`sms`، `sms_events`	`SmsHandler`	`NativeSmsService`	ارسال، پرس‌وجوی provider، حذف، پیامک ورودی
`call`، `call_events`	`call/CallHandler`	`CallService`	چرخهٔ تماس، فهرست سیاه و سفید
`call_log`، `call_log_events`	`calllog/CallLogSyncHandler`	`NativeCallLogService`	خواندن فهرست تماس و اعلام تغییر
`call_reminders`	`reminders/`	سرویس یادآور	یادآورهای «اخیر»
`sim`، `sim_events`	`sim/SimRegistry`	`SimService`	فهرست سیم‌کارت، پیش‌فرض‌ها، قانون تماس
`sim_contacts`	`contacts/SimContactsHandler`	`contacts`	مخاطبان سیم‌کارت
`contact_extras`	`contacts/ContactExtrasHandler`	`contacts`	زنگ اختصاصی، حساب‌ها و کارهای بیرون از افزونه
`contact_groups`	`contacts/ContactGroupsHandler`	`ContactGroupsService`	برچسب‌ها
`contact_link`	`contacts/ContactLinkHandler`	`contacts`	ادغام مخاطبان
`contact_file`	`contacts/ContactFileHandler`	`contacts`	فایل `.vcf`
`scheduled_sms`	`scheduled/`	`scheduled`	تحویل بومی و آلارم
`blocked`	`SystemBlockList`	`system_block_list.dart`	فهرست مسدود خود گوشی
`intents`	`MainActivity`	`DeepLinkService`	لینک عمیق و گفتگوی باز
`media`	`media/PhotoHandler`	`ImagePickerService`	عکس مخاطب
`location`	`LocationHandler`	`LocationService`	«ارسال موقعیت»
`app_lock`	`security/`	`BiometricUnlock` و هم‌خانواده‌ها	پنجرهٔ اثر انگشت، حریم اعلان، جلسهٔ قفل

کانال	سمت Kotlin	سمت Dart	کار
`device_identity`	`edition/DeviceIdentityHandler`	`edition`	کد دستگاه
`secure_vault`	`secure/`	`features/secure`	بخش امن Keystore
`key_file`	`smscrypto/KeyFilePicker`	`keybank`	انتخاب فایل کلید
`sms_crypto`	`smscrypto/`	`SmsCryptoService`	رمزنگاری بدون وضعیت
`hidden`	`hidden/HiddenHandler`	`hidden` و `security`	برچسب شماره‌ها، صف مهرشده، کنترل امنیت، عکس ورود ناموفق

۷.۳ قرارداد کدهای خطا

کدهای خطای ارسال پیامک بخشی از قرارداد دو طرف‌اند:

کد	معنا
`NO_SIM_CARD`	سیم‌کارت نیست
`NO_SERVICE`	آنتن نیست
`PERMISSION_DENIED`	مجوز نیست. Dart این را «نمی‌توانم بخوانم» می‌فهمد، نه «چیزی نیست»، پس داده را پاک نمی‌کند
`SMS_SEND_FAILED`	ارسال ناموفق
`HIDDEN_NUMBER`	شماره در دفترچهٔ مخفی است و نوشتن معمولی ممنوع است
`NUMBER_NOT_ALLOWED`	«فهرست سیاه و سفید» اجازه نمی‌دهد

۷.۴ شمارهٔ درخواست‌های `onActivityResult`

شماره	کار
`9002`	نقش پیام‌رسان پیش‌فرض
`9003`	نقش تلفن پیش‌فرض
`9101`	انتخاب زنگ
`9201`، `9202`	گالری و دوربین عکس مخاطب
`9301`	انتخاب فایل کلید
`9401`، `9402`	ساختن و باز کردن فایل `.vcf`

`MainActivity.kt` این نتیجه‌ها را به handler مربوط می‌رساند.

۷.۵ مؤلفه‌های مانیفست

مؤلفه	کار
`MainActivity`	تنها اکتیویتی. فیلترهای `sms:`، `smsto:`، `mms:`، `mmsto:`، `tel:`، `DIAL` و `SEND` متنی
`HeadlessSmsSendService`	پاسخ سریع از صفحهٔ تماس (لازمهٔ نقش پیامک)

مؤلفه	کار
`SmsDeliverReceiver`	`SMS_DELIVER`، نوشتن در provider
`IncomingSmsReceiver`	`SMS_RECEIVED` با اولویت ۹۹۹، وقتی برنامه بسته است
`MmsReceiver`	`WAP_PUSH_DELIVER`، فقط برای نقش. MMS پشتیبانی نمی‌شود
`SmsNotificationActionReceiver`	پاسخ، خواندم، مسدودی و لغو آن از اعلان
`call.CallInCallService`	سرویس تماس هنگام پیش‌فرض بودن
`call.CallConnectionService`	مسیر self-managed غیرفعال برای VoIP احتمالی
`call.MissedCallNotificationReceiver`	`SHOW_MISSED_CALLS_NOTIFICATION`
`call.CallActionReceiver`	پاسخ، رد و تماس برگشتی از اعلان
`scheduled.ScheduledSmsAlarmReceiver`، `scheduled.BootReceiver`	آلارم زمان‌بندی و تنظیم دوباره بعد از روشن شدن
`reminders.CallReminderReceiver`	آلارم یادآور
`security.AppLockSession$DeadlineReceiver`	پایان پنجرهٔ قفل خودکار

۷.۶ مجوزهای مانیفست

هر دو فلیور یک مانیفست دارند. اگر روزی مجوزی فقط برای یک نسخه لازم شد، باید به مانیفست همان فلیور برود.

گروه	مجوزها
پیامک	`SEND_SMS`، `RECEIVE_SMS`، `READ_SMS`، `RECEIVE_MMS`، `RECEIVE_WAP_PUSH`
تلفن	`CALL_PHONE`، `READ_PHONE_STATE`، `READ_CALL_LOG`، `WRITE_CALL_LOG`، `MANAGE_OWN_CALLS`، `PROCESS_OUTGOING_CALLS`، `READ_PHONE_NUMBERS`
مخاطب	`READ_CONTACTS`، `WRITE_CONTACTS`
اعلان و زمان	`POST_NOTIFICATIONS`، `VIBRATE`، `RECEIVE_BOOT_COMPLETED`، `USE_FULL_SCREEN_INTENT`، `WAKE_LOCK`، `SCHEDULE_EXACT_ALARM`، `USE_EXACT_ALARM`
اختیاری	`ACCESS_FINE_LOCATION`، `ACCESS_COARSE_LOCATION`، `CAMERA`، `USE_BIOMETRIC`
شبکه	`INTERNET` فقط برای گزارش خطای اختیاری

مجوزهای زمان اجرا را فقط `PermissionGate` می‌گیرد. دو مجوز اختیاری، یعنی دوربین و موقعیت، را خود برنامه همان لحظهٔ نیاز می‌خواهد. برنامه مجوز حافظه نمی‌خواهد، چون فایل‌ها از انتخابگر سیستم می‌گذرند.

۷.۷ آینه‌ها: فایل‌هایی که باید با هم عوض شوند

این مهم‌ترین جدول این بخش است. عدم تطابق این فایل‌ها خطای آشکار نمی‌دهد و برنامه فقط بی‌صدا کار دیگری می‌کند.

Kotlin یا ساخت	Dart یا طرف دیگر
`ScheduledSmsWorker.kt`	`scheduled_message_model.dart`
`TemplateWire.kt`	`message_template_model.dart` و `TinyImage` (`sigil`، شکل `isWire`، `label`)
`BlockedNumbers.kt`	`PhoneNormalizer`

Kotlin یا ساخت	Dart یا طرف دیگر
`SecureSmsInbox.kt`	`DatabaseHelper._createSecureQueueTable` و `AppConstants.secureQueueTable`
`hidden/SealedInbox.kt`	`DatabaseHelper._createSealedQueueTable` و `AppConstants.sealedQueueTable`
`SmsNotificationActionReceiver.chosenThreadSim`	`ThreadSimRepository`
`SimRegistry.setAppDefaults` ، `APP_ASK`	`SimService.setAppDefault` ، `appAsk`
`SimRegistry.routeFor`	`SimService.callRoute` فقط می‌پرسد
`SystemBlockList.kt`	`system_block_list.dart` (`remove` با پاسخ ۱- یعنی اجازهٔ نوشتن نیست)
`Wire.PREFIX`	`SmsCryptoService.wirePrefix`
`CallRules.normalizePrefix`	`CallBlockPrefixes.normalize`
`NumberListRules.normalizePattern`	`NumberLists.normalizePattern` ، با کلیدهای کانال و حالت
`NotificationPrivacy.PIN_KEY`	`AppConstants.pinKey`
`MemberCert.keyIdOf`	`certKeyIdFor` در `secure_identities.dart` ، هر دو به یک بردار ثابت
نام فلیورها و پایهٔ versionCode در `build.gradle.kts`	`AppEdition` و `scripts/release_build.{sh,ps1}`
`ActivationCode` ، `ActivationTarget`	تولیدکنندهٔ ویندوزی راهنما (قالب ثابت) و `tools/keybank/src/main/kotlin/Activation.kt`

چند چیز هم با آزمون ثابت شده‌اند و مقدار موردانتظارشان نباید عوض شود: قالب پیام با `GoldenVectorTest` ، رکورد مهرشده با `SealedBoxTest` و بانک کلید با `KeyBankTest` . تغییر در این‌ها یعنی نسخهٔ جدید قالب (`Wire.VERSION`)، نه عدد جدید در آزمون.

۷.۸ روش کار مشترک handlerها

- **هیچ handlerی مجوز نمی‌گیرد.** نبود مجوز به `SecurityException` می‌رسد و به `PERMISSION_DENIED` تبدیل می‌شود.
- **کار سنگین روی رشتهٔ جدا اجرا می‌شود.** خواندن فهرست تماس و سیم‌کارت هر کدام executor خود را دارند، چون اجرای آن‌ها روی رشتهٔ اصلی لمس را قفل می‌کرد. برای `CallLogSyncHandler` فقط یک رشته وجود دارد تا حذف و خواندن بعدش به‌ترتیب بمانند.
- **حجم پیام کانال محدود است.** تراکنش Binder حدود ۱ مگابایت جا دارد، پس خواندن‌های بزرگ صفحه‌به‌صفحه است.
- **`DeviceSyncQueue`** در Dart خواندن‌های سنگین از گوشی را پشت هم می‌چیند تا سه بار بزرگ هم‌زمان در حافظه نباشند. این اوج حافظه روی گوشی ضعیف برنامه را می‌کشت.
- **کلاس‌های خالص جدا هستند.** قانون‌هایی مثل `CallRules` ، `NumberListRules` و کل `smscrypto/` به Android وابسته نیستند و با آزمون JVM بررسی می‌شوند.

بخش ۸. پیامک: دریافت، ارسال و همگام‌سازی

۸.۱ فایل‌های اصلی

فایل	کار
`kt/SmsHandler.kt`	کانال `sms` ، ارسال، پرس‌وجو، حذف، گیرندهٔ زنده
`kt/SmsDeliverReceiver.kt`	نوشتن پیامک رسیده در `content://sms`
`kt/IncomingSmsReceiver.kt`	ذخیره و اعلان وقتی برنامه بسته است
`kt/SmsNotifier.kt`	همهٔ اعلان‌های پیامک
`kt/SmsNotificationActionReceiver.kt`	دکمه‌های اعلان
`features/messages/services/native_sms_service.dart`	پوشش کانال
`features/messages/services/sms_service.dart`	هماهنگی ارسال، دریافت و همگام‌سازی
`features/messages/repositories/message_repository.dart`	SQL پیام‌ها
`features/messages/bloc/message_bloc.dart`	وضعیت صندوق و گفتگو

۸.۲ نقش پیام‌رسان پیش‌فرض

برنامه نقش `ROLE_SMS` را می‌گیرد. بنر `DefaultSmsBanner` در صندوق این را پیشنهاد می‌کند و درخواست در `SmsHandler.requestDefaultSmsRole` است. برای پاسخ هم `isDefaultSmsApp()` معتبر است، نه `resultCode` .

اندروید برای این نقش چهار چیز می‌خواهد و هر چهار در مانیفست هستند: گیرندهٔ `SMS_DELIVER` ، گیرندهٔ `WAP_PUSH_DELIVER` (هم‌رسان MMS ندارد و این گیرنده فقط برای نقش است)، فیلتر `SEND` / `SENDTO` روی `MainActivity` ، و `HeadlessSmsSendService` .

تقسیم کار، بدون انجام دوباره

- `SmsDeliverReceiver` **فقط** پیامک را در `content://sms` می‌نویسد. وقتی نقش دست برنامه است، سیستم دیگر این کار را نمی‌کند.
- ذخیره در پایگاه‌داده، اعلان و رابط از مسیر `SMS_RECEIVED` می‌گذرند: گیرندهٔ زندهٔ `SmsHandler` وقتی برنامه باز است، و `IncomingSmsReceiver` وقتی بسته است.
- هر ارسال موفق در `content://sms/sent` هم نوشته می‌شود (`writeSentToProvider`)، وگرنه برنامه‌های دیگر آن را نمی‌بینند. شناسهٔ برگشتی در `messages.device_sms_id` ذخیره می‌شود.

۸.۳ مسیر دریافت

۱. پیامک به گیرندهٔ زنده یا `IncomingSmsReceiver` می‌رسد.

۲. **غربال:** متن پوششی برداشته می‌شود (`CoverText.uncover`)؛ شماره در دفترچهٔ مخفی بررسی می‌شود؛ پیام `#E:` جدا می‌شود؛ مسدودی و «فهرست سیاه و سفید» بررسی می‌شوند.

۳. پیام عادی به Dart می‌رسد: `NativeSmsService.onSmsReceived` ← `SmsService.listenToIncomingSms` ← ذخیره ← `MessageBloc.add(ReceiveMessage)` . اگر برنامه بسته باشد، Kotlin خودش در SQLite می‌نویسد.

۴. `SmsNotifier` اعلان را می‌فرستد.

`SmsService.listenToIncomingSms` یک بار در هر اجرا ثبت می‌شود و `MessageBloc` آن را با اولین `LoadThreads` و **پیش از** همگام‌سازی شروع می‌کند. قبلاً بعد از آن شروع می‌شد و پیامی که وسط همگام‌سازی می‌رسید شنونده نداشت.

نوع پیام	content://sms	messages	اعلان
عادی	بله	بله	کارت کامل
رمز `#E:`	خیر	خیر، در `secure_queue`	فقط «پیام رمز جدید»
شمارهٔ دفترچهٔ مخفی	خیر	خیر، مهرشده در `sealed_queue`	«پیام رمز جدید»، بدون نام و متن
مسدود یا ردشده با فهرست	خیر	خیر	هیچ

> گیرندهٔ زندهٔ `SMS_RECEIVED` باید `RECEIVER_EXPORTED` باشد، چون پیام سیستم از UID دیگری می‌آید و باید با مجوز فرستندهٔ `BROADCAST_SMS` ثبت شود. بدون این مجوز، هر برنامه‌ای می‌تواند پیامک جعلی وارد صندوق کند.

۸.۴ مسیر ارسال

`SmsService.sendSms` ← `NativeSmsService.sendSms` ← کانال ← `SmsHandler.sendSms`. سمت Kotlin پیش از `SmsManager` سیم‌کارت، آنتن و فهرست‌ها را بررسی می‌کند. Dart هم پیش از ذخیره کردن همان سؤال فهرست را از `NumberListsBridge.allows` می‌پرسد تا حبابی با «ارسال نشد» نماند.

تیک تحویل

`SmsHandler.SmsDeliveredReceiver` گزارش وضعیت را از **PDU** می‌خواند، نه از `resultCode` پیام. روی این گوشی‌ها `resultCode` برای هر گزارشی `RESULT_OK` است، حتی وقتی شبکه می‌گوید شکست خورد. خطای واقعی این بود که ✓✓ روی پیام نرسیده می‌آمد.

TP-Status	معنی	کار حباب
`0x00` تا `0x1F`	تحویل شد	✓✓
`0x20` تا `0x3F`	موقت، مرکز پیام هنوز تلاش می‌کند	هیچ، گزارش دیگری می‌آید
بقیه	شکست قطعی	«ارسال نشد»

گزارش CDMA (`3gpp2`) این فیلد را ندارد و تحویل‌شده حساب می‌شود. PDU خوانده‌نشده هم تحویل‌شده است، چون خود گزارش از شبکه آمده.

پیام چندبخشی برای هر بخش گزارش می‌دهد. `PartTally` می‌شمارد: ✓ و ✓✓ به **همهٔ** بخش‌ها نیاز دارند و شکست یک بخش کل پیام را قطعی شکست می‌دهد. `expect()` باید **پیش از** `sendTextMessage` صدا زده شود، چون رادیو ممکن است پیش از ادامهٔ coroutine جواب دهد.

۸.۵ همگام‌سازی آینه

صندوق برنامه آینهٔ `content://sms` است. `SmsService.syncDeviceMessages` آن را هماهنگ نگه می‌دارد. یک بار در هر اجرا روی `LoadThreads` اجرا می‌شود و بی‌صدا هم در بازگشت به برنامه، حداکثر هر ۲ دقیقه یک بار.

۱. فهرست **کامل شناسه‌های** هر صندوق را می‌خواند (`querySmsIds`، از تازه به قدیم).

۲. شناسه‌های شناخته‌شده را کم می‌کند (`MessageRepository.knownDeviceSmsIds`).

۳. فقط شناسه‌های جدید را در صفحه‌های ۲۰۰تایی می‌خواند (`querySmsByIds`).

۴. هر صفحه را با `reconcileDeviceRows` تطبیق می‌دهد: شناسهٔ شناخته‌شده ← تطبیق دقیق متن ← تطبیق تقریبی با اختلاف ۲ دقیقه ← درج.

۵. ردیف‌هایی را که شناسه‌شان از گوشی رفته حذف می‌کند.

تطبیق تقریبی لازم است، چون ردیف زنده زمان PDU را دارد و provider زمان دریافت را.

قانون‌هایی که نباید شکسته شوند

- **همگام‌سازی نقاشی و صف BLoC را نگه نمی‌دارد.** `MessageBloc._startBackgroundSync` آن را جدا اجرا می‌کند و پایانش به شکل `DeviceSyncFinished` برمی‌گردد. اگر داخل handler منتظرش بمانید، حدود ۵ ثانیه چرخانک می‌آید و بقیهٔ eventها، مثل باز کردن گفتگو، هم پشت آن می‌مانند.
- **`_hasImported` فقط بعد از موفقیت تنظیم می‌شود**، تا رد مجوز در بار اول دوباره تلاش شود.
- **`reconcileDeviceRows` هر صفحه را با سه دستور انجام می‌دهد، نه سه دستور برای هر ردیف.** روی ۲۰٬۰۰۰ پیامک این کار زمان ورود را از ۱۰۴ به ۱۳ ثانیه رساند.
- **`removeRowsMissingFromDevice` داخل SQLite** و با یک جدول موقت کار می‌کند. آوردن همهٔ ردیف‌ها به Dart وسط اسکرول UI را قفل می‌کرد.
- **سقف محتوا وجود ندارد.** سقف ۵۰۰ ردیفی قدیمی صندوق دریافت را زودتر قطع می‌کرد و کاربر با اسکرول به عقب فقط پیام‌های خودش را می‌دید.
- **`SmsService._unlinkable`** ردیف‌هایی را به یاد می‌سپارد که ردیف محلی نمی‌سازند، مثل پیام رمز یا تکراری دقیق. بدون آن، هر بازگشت به برنامه دوباره آن‌ها را می‌خواند.
- **پیشرفت ورود گزارش می‌شود** (`onProgress` و `_onSyncProgress`، هر ۲ ثانیه) تا صندوق کم‌کم پر شود.

۸.۶ حذف

حذف همیشه سراسری است: اول ردیف‌های گوشی، بعد محلی (`SmsService.deleteMessagesGlobally` و `deleteThreadGlobally`). ردیف گوشی با `device_sms_id` پیدا می‌شود و اگر نبود با متن و زمان تا اختلاف ۱۰ ثانیه. حذف گفتگو نشانی‌ها را با ۱۰ رقم آخر تطبیق می‌دهد.

> **نیمهٔ محلی حذف باید نرم باشد** (`softDeleteMessages`، `softDeleteThread`). حذف از گوشی بدون نقش پیامک کاری نمی‌کند. اگر ردیف محلی کاملاً پاک شود، همگام‌سازی بعدی آن را دوباره وارد می‌کند و کاربر می‌بیند پیام‌های حذف‌شده بعد از بازکردن دوبارهٔ برنامه برگشته‌اند. `_knownDeviceSmsIds` عمداً `is_deleted` را نادیده می‌گیرد.

۸.۷ اعلان‌های پیامک

همهٔ اعلان‌های پیامک فقط از `SmsNotifier` در Kotlin فرستاده می‌شوند و `NotificationService` در Dart هیچ مسیری برای پیامک ندارد. مسیر Dart را دوباره اضافه نکنید، چون اعلان دوتایی می‌شود.

- **یک کارت برای هر گفتگو** با `MessagingStyle`، فقط با پیام‌های خوانده‌نشده. شناسه‌اش `threadId.hashCode()` است و یک میان‌بر بلندمدت آن را به بخش گفتگوها می‌برد.
- **دکمه‌ها کاملاً بومی‌اند:** پاسخ با `RemoteInput`، «خواندم» و «مسدودسازی». با برنامهٔ بسته هم کار می‌کنند.

- **کارت همیشه بسته می‌شود**، حتی اگر نوشتن پایگاه‌داده شکست بخورد. قبلاً «خواندم» گاهی کار نمی‌کرد، چون قفل نوشتن دست اتصال Dart بود. نوشتن‌ها از `withDbRetry` می‌گذرند.
- **هر دکمه به برنامهٔ باز خبر می‌دهد:** `MainActivity.notifyThreadChanged` ← `threadChanged` ← `DeepLinkService.onThreadChanged` ← `ThreadChangedExternally` و `LoadBlocked`.
- **«مسدودسازی»** کارت را با یک کارت بی‌صدا عوض می‌کند که ۳۰ ثانیه «لغو» دارد (`BLOCKED_CARD_TIMEOUT_MS`).
- `SmsNotifier.reconcile` کارت‌هایی را که دیگر چیزی خوانده‌نشده ندارند می‌بندد. عدد روی آیکون برنامه شمارش همین کارت‌هاست. این تابع هم از `MainActivity.onResume` و هم از Dart صدا زده می‌شود. اگر نتواند چیزی را بخواند، کارت را نگه می‌دارد، چون پاک کردن اشتباه پیامی را گم می‌کند.
- `setNumber(unread + 1)` برای لانچرهایی است که عدد می‌کشند، مثل One UI.
- **تشخیص دیده شدن:** اعلان گفتگوی باز فقط وقتی حذف می‌شود که کاربر واقعاً آن را ببیند: `isResumed`، `visibleThreadId` یکسان، صفحه روشن و گوشی قفل نباشد. بعضی گوشی‌ها اکتیویتی را پشت صفحهٔ قفل هم resumed می‌کنند.
- **آیکون‌ها:** آیکون کوچک سفید و شفاف است (`ic_stat_message`، `ic_stat_call`)، چون سیستم فقط کانال آلفا را نگه می‌دارد. آیکون بزرگ عکس مخاطب یا `NotificationAvatars.letterAvatar` است و رنگش از شمارهٔ عادی‌شده می‌آید.
- **پیامک تماس بی‌پاسخ اپراتور:** اگر فرستنده غیرقابل‌شماره‌گیری باشد، متن شماره‌ای را داشته باشد که `CallInCallService.markMissedCall` علامت زده (تا ۳۰ دقیقه)، و عبارتش دربارهٔ تماس باشد، فقط اعلان حذف می‌شود و خود پیام در صندوق می‌ماند.

پنهان کردن محتوای اعلان

«پنهان کردن محتوای اعلان‌ها» وقتی فعال است که برنامه **الان** قفل باشد، نه صرفاً PIN داشته باشد. قفل بودن را `AppLockSession` در Kotlin می‌داند که فقط در حافظه است، پس فرایند تازه قفل حساب می‌شود.

- `SmsNotifier.privateCard` فقط «پیام جدید» یا «N پیام جدید» می‌گوید. نام، آواتار و دکمه ندارد، چون دکمه از بیرون قفل روی فرستنده کار می‌کرد.
- پنجرهٔ «قفل خودکار» در Kotlin و از `MainActivity.onStop` شمرده می‌شود، چون شاید Dart دیگر اجرا نشود.
- کارتی که هنگام باز بودن برنامه کامل نمایش داده شده، در لحظهٔ قفل با `privatizeActive` بی‌صدا دوباره فرستاده می‌شود.
- کارت تماس ورودی نام تماس‌گیرنده را نشان می‌دهد، چون کاربر باید برای پاسخ دادن تصمیم بگیرد.
- پرچم با `NotificationPrivacySync` در Dart به Kotlin می‌رسد. اگر هنوز چیزی نوشته نشده باشد، وجود `app_pin` (فقط نام، نه مقدار) ملاک است و خطا یعنی «پنهان کن».

۸.۸ لینک عمیق

همهٔ کلیک‌های بیرونی از یک مسیر می‌آیند: `MainActivity.consumeLaunchAction` ← `DeepLinkService.LaunchAction` ← `MainNavigation._handleLaunchAction`.

نوع	از کجا	نتیجه
`thread`	اعلان پیامک	باز شدن گفتگو
`dial`	`tel:` از برنامهٔ دیگر	برگهٔ شماره‌گیر با شمارهٔ پرشده

نوع	از کجا	نتیجه
`sms`	`sms:` ، `smsto:` ، `mms:` یا متن اشتراکی	گفتگو با متن آماده، یا `ContactSelectorScreen(initialText:)`

- `ACTION_SEND` آدرس داده ندارد، پس **فیلتر مانیفست جدا** با `mimeType="text/plain"` لازم است. آن را با فیلتر scheme یکی نکنید.
- `sms:…?body=` مبهم است و `getQueryParameter` خطا می‌دهد، پس کوئری دستی خوانده می‌شود.
- مقدار intent بعد از مصرف پاک می‌شود تا در resume بعدی تکرار نشود.

۸.۹ همگام‌سازی فهرست تماس

این مسیر هم شبیه پیامک است و جزئیاتش در بخش ۱۴ آمده. `CallLogSyncHandler.kt` یک `ContentObserver` روی `CallLog.Calls` می‌گذارد و تغییرها را با تأخیر کوتاه می‌فرستد. خواندن ردیف‌ها هم بومی است (`callLogEntries`)، چون افزونهٔ `call_log` خودش مجوز می‌گرفت و هنگام همپوشانی دو درخواست برنامه را روی نصب تازه کرش می‌داد.

بخش ۹. صندوق پیام، جستجو، گروه و مسدودی

۹.۱ صندوق

`MessagesListScreen` صفحهٔ «پیام‌ها» است و از چند منبع ساخته می‌شود:

- `MessageRepository.getAllThreads`: گفتگوها با صفحه‌بندی ۵۰تایی. صفحه‌بندی داخل یک CTE به نام `page` انجام می‌شود که فقط `thread_id`، `last_ts` و `is_pinned` را دارد. کار سنگین هر گفتگو، یعنی آخرین پیام و شمارش خوانده‌نشده، فقط روی ردیف‌های باقی‌مانده بعد از `LIMIT` اجرا می‌شود. تخت کردن این پرس‌وجو همان کندی قدیمی را برمی‌گرداند.
- `_mergeOverlays`: پیش‌نویس و پیام زمان‌بندی‌شده را روی ردیف‌ها می‌گذارد. ردیفی که فقط پیش‌نویس یا زمان‌بندی دارد ساختگی است و کشیدن و انتخاب گروهی ندارد.
- `_lastInbox`: آخرین `ThreadsLoaded` نقاشی‌شده. وقتی `MessageBloc` روی گفتگو یا بایگانی ایستاده، صندوق همین را می‌کشد. اگر چرخانک می‌آمد، sliver خراب می‌شد و کاربر بعد از هر بازگشت به بالای فهرست می‌رفت.

نگهبان‌های `MessageBloc`

- `_applyNames` نام را **بازنویسی** می‌کند و شمارهٔ بدون تطبیق بی‌عنوان می‌شود. وگرنه نام مخاطب حذف‌شده می‌ماند. به همین دلیل `MessageThread.copyWith` پارامتر `clearContactName` دارد. ردیف‌های گروه رد می‌شوند.
- `_loadedThreadCount` و `_loadedArchivedCount` نمی‌گذارند به‌روزرسانی فهرست را کوتاه کند. هر `LoadThreads` از صفر حداقل به تعداد ردیف‌های روی صفحه می‌خواند.
- `_onSyncProgress` فهرست روی صفحه را به‌روز می‌کند، حتی اگر بایگانی باشد.
- نتیجهٔ جستجو یک عکس فوری است، پس هر تغییر دوباره پرس‌وجو را اجرا می‌کند. یک `BlocListener` روی `ThreadsLoaded` این کار را انجام می‌دهد.
- **سنجاق حداکثر پنج تاست** و `MessageBloc._onPinThread` ششمی را رد می‌کند. بایگانی سنجاق را برمی‌دارد.

خوانده‌نشده و «علامت‌گذاری نخوانده»

گفتگو می‌تواند به دو دلیل پررنگ باشد و صندوق این دو را جدا نشان می‌دهد:

- پیام واقعاً رسیده: نشان **عدد** است.
- کاربر دستی علامت زده: نشان یک **نقطه** است.

علامت در جدول جدا (`unread_marks`) است و هیچ پیامی را تغییر نمی‌دهد. `MessageThread.manuallyUnread` آن را حمل می‌کند و `hasUnread` برابر `unreadCount > 0 || manuallyUnread` است. نشان نوار پایین و انتخاب تب فقط `unreadCount` را جمع می‌زنند، چون علامت دستی پیام تازه نیست. باز کردن گفتگو علامت را پاک می‌کند.

۹.۲ جستجوی پیام

`MessageRepository.searchMessages` و `searchThreads` تنها جستجوی پیام‌اند و متن همهٔ پیام‌ها را می‌گردند. پیش‌ترها فقط آخرین پیام گفتگوهای بارشده بررسی می‌شد و پیام‌های خود کاربر پیدا نمی‌شدند.

- **دو مرحله:** مرحلهٔ SQL فقط باید بیش از حد پهن باشد، نه کم. پاسخ نهایی با `SearchText.nameContains` در Dart گرفته می‌شود. جدول پیش‌تصویر از خود `SearchText.fold` ساخته می‌شود و کپی نیست، چون کپی با کد اصلی فاصله می‌گرفت.
- **FTS5 روی اندروید خاموش است.** SQLite سیستم اندروید ماژول FTS5 ندارد، پس `_createMessageSearchFts` شکست می‌خورد و `messageSearchFtsReady` همیشه false است. کد نگه داشته شده، چون اگر روزی SQLite جدا به برنامه اضافه شود فعال می‌شود. آن کار تصمیم بزرگی است، چون Kotlin با برنامهٔ بسته همان فایل را با موتور سیستم باز می‌کند و دو موتور باید روی WAL و قفل هم‌نظر باشند. FTS4 جایگزین نیست، چون trigram ندارد و جستجوی زیررشته نمی‌تواند.
- **نمایه فقط بحث سرعت است، نه درستی.** فقط وقتی استفاده می‌شود که نمایه وجود داشته باشد، عبارت حداقل ۳ حرف باشد و نمایه عقب نباشد.
- **مسیر کند محدود است.** حرکت از تازه به قدیم روی `idx_messages_search` است و سقف `_kMaxPrefilteredRows` دارد. عبارت یک یا دوحرفی فقط پیام‌های اخیر را می‌گردد.
- **تطبیق شماره در Dart است.** چون `thread_id LIKE` ممکن است چیزی را جا بیندازد.
- **نام مخاطب سهم صفحه است.** صفحه نام‌ها را تطبیق می‌دهد و شناسه‌ها را به شکل `alsoThreadIds` می‌فرستد.
- **پیام قالبی** بدون شرط از پیش‌فیلتر رد می‌شود و با `TemplateWire.displayText` سنجیده می‌شود.
- `searchThreads` نتیجه را از `getAllThreads(restrictToThreadIds:)` می‌کشد تا آخرین پیام و شمارش با صندوق یکی بمانند.
- **جستجوی یکپارچه** (`SearchBloc`) پیام‌ها را ۴۰تایی صفحه‌بندی می‌کند و تماس‌ها را از `getLatestCallPerNumber` می‌خواند. جزئیات تطبیق متن در بخش ۱۷ است.

۹.۳ فرستنده‌ای که نمی‌شود به آن پاسخ داد

`SmsAddress.canReceive` (`core/utils/sms_address.dart`) تنها آزمون این سؤال است که آیا می‌شود به این نشانی پیامک فرستاد. برای فرستندهٔ حرفی مثل «Snapp» و برای کمتر از سه رقم false است. آن وقت به جای نوشتن پیام، جملهٔ «امکان پاسخ به این گفتگو وجود ندارد» می‌آید. شماره‌های خدماتی عددی مثل `5000301630` عمداً قابل‌پاسخ می‌مانند، چون اپراتورها پاسخ به آن‌ها را می‌خواهند. گروه همیشه قبول است.

۹.۴ مسدودی و گزارش هرزنامه

- **گفتگوی مسدود از همهٔ فهرست‌ها می‌رود** (زیرپرس‌وجوی `NOT IN` در `getAllThreads`) و در `SpamAndBlockedScreen` می‌ماند. باز کردن آن چیزی را آزاد نمی‌کند.
- **یک حرکت، یک بار پرسش:** `showBlockNumberDialog` و `blockNumberWithConfirm` همه‌جا مشترک‌اند. تیک «گزارش به‌عنوان هرزنامه» برای شمارهٔ ناشناس روشن و برای مخاطب ذخیره‌شده خاموش است.
- `BlockedNumbersRepository.block` پرچم را ارتقا می‌دهد و مسدودی ساده هیچ‌وقت گزارش را پاک نمی‌کند.
- **فرستندهٔ حرفی هم قابل مسدودی است.** کلید مسدودی همان شناسهٔ گفتگوست، حتی اگر رقم نداشته باشد. قبلاً کلید خالی می‌شد، ردیفی نوشته نمی‌شد و پیام تأیید هم می‌آمد.
- **مسدودی کارت اعلان فرستنده را می‌بندد.**
- مسدودی در چهار مسیر دریافت بومی بررسی می‌شود و `CallInCallService` هم تماس زنگ‌خورده را رد می‌کند.

مسدودی سراسری

مسدودی روی کل گوشی اثر دارد. هر مسدودی در فهرست خود اندروید، `BlockedNumberContract`، هم نوشته می‌شود (`SystemBlockList` در Kotlin، کانال `blocked`). این فهرست را فقط برنامهٔ پیش‌فرض تلفن یا پیامک می‌تواند بنویسد. `blocked_numbers` همچنان سند برنامه است و همان چیزی است که مسیرهای دریافت می‌خوانند، چون باید با برنامهٔ بسته و بدون نقش هم کار کنند.

`BlockedNumbersRepository.syncWithSystem` همگام‌سازی **سه‌طرفه** است، با عکس فوری `blocked_system_snapshot`:

وضعیت	معنی	کار
از سیستم رفته، اینجا هست، در عکس فوری بوده	در برنامهٔ دیگری آزاد شده	اینجا آزاد می‌شود
در سیستم تازه آمده	در برنامهٔ دیگری مسدود شده	اینجا مسدود ساده می‌شود
اینجا هست، در سیستم نیست، در عکس فوری نیست	نوشته نشده بود	نوشته می‌شود
در عکس فوری هست، در سیستم هست، اینجا نیست	اینجا آزاد شد ولی نوشتن ممکن نبود	از سیستم برداشته می‌شود

برای حالت آخر، `_fromSystem` کلید را **فقط وقتی** از عکس فوری برمی‌دارد که حذف واقعاً انجام شده باشد. `remove` native برای «اجازهٔ نوشتن نیست» ۱- برمی‌گرداند. این هماهنگ‌سازی روی هر `LoadBlocked` اجرا می‌شود.

صفحهٔ مسدودی انتخاب گروهی دارد: «رفع مسدودی» یک تراکنش است (`unblockAll`) و «حذف گفتگو» شماره را مسدود نگه می‌دارد. نام و پیش‌نمایش ردیف‌ها یک‌جا خوانده می‌شوند.

۹.۵ «فهرست سیاه و سفید»

`NumberListsPage` (تنظیمات ← عمومی). چهار کانال مستقل دارد: پیامک دریافتی، پیامک ارسالی، تماس ورودی و تماس خروجی. هر کدام حالت خودش را دارد:

حالت	چه کسانی رد می‌شوند
`blacklist` «همه، به‌جز فهرست سیاه» (پیش‌فرض)	همه به‌جز تطبیق با فهرست سیاه
`whitelist` «فقط فهرست سفید»	فقط فهرست سفید، و با تیک «مخاطبین ذخیره‌شده هم مجازند» همهٔ مخاطبان
`nobody` «هیچ‌کس»	هیچ‌کس

- **ورودی‌ها عدد یا الگو هستند:** `*` هر تعداد رقم و `?` یا `؟` یک رقم، مقایسه روی کل شکل ملی. `*98912+` به `*0912` تبدیل می‌شود.
- **از `blocked_numbers` جداست.** آن فهرست همیشه اول بررسی می‌شود. این یکی در `SharedPreferences` (`set_list_*`) است و به فایل بومی `number_lists` هم نوشته می‌شود، چون تصمیم در مسیرهای بدون موتور Flutter گرفته می‌شود.
- **تصمیم فقط یک‌جاست:** `NumberListRules.allows`، کلاس خالص با آزمون JVM. جاهای اعمال: سه گیرندهٔ پیامک، `SmsHandler.sendSms`، کارگر زمان‌بندی، پاسخ از اعلان، `CallHandler.makeCall` و `CallInCallService.onCallAdded`. تماس در حال زنگ رد می‌شود، تماس در حال برقراری قطع می‌شود و تماس از قبل برقرار هرگز قطع نمی‌شود.

- **هیچ‌وقت رد نمی‌شوند:** شماره‌های کمتر از ۷ رقم (اضطراری، خدماتی، USSD، فرستندهٔ حرفی)، شمارهٔ اضطراری خروجی و شمارهٔ دفترچهٔ مخفی.

۹.۶ «گفتگوی جدید»

`ContactSelectorScreen` سه حالت دارد: عادی (باز کردن گفتگو)، `pickOnly` (برای «هدایت») و `pickMembers` (افزودن عضو).

- **«هدایت»:** لمس کوتاه برای یک نفر، لمس طولانی برای چند نفر. `ConversationScreen._forwardToMany` یک بار تأیید می‌گیرد و برای هر نفر یک پیامک جدا می‌فرستد، با سیم‌کارت گفتگوی خودش. گروه ساخته نمی‌شود.
- **«گفتگوهای اخیر» از `MessageBloc.lastInbox` می‌آید، نه `state`**، چون ممکن است از داخل گفتگو باز شده باشد.
- **تطبیق و الفبا** از `ContactRepository.matchContacts` و `core/widgets/contact_index_list.dart` می‌آید. چیزی که بالای بخش‌ها کشیده می‌شود باید ارتفاع ثابت داشته باشد و در `leadingOffset` شمرده شود، وگرنه پرش سریع به نام اشتباه می‌رود.
- **انتخاب هر نفر متن تایپ‌شده را پاک می‌کند.**
- **«ارسال به این شماره»** وقتی شماره متعلق به مخاطب موجود است نشان داده نمی‌شود (`_typedNumberOwner`)، وگرنه گفتگوی جدا با شناسهٔ دیگری باز می‌شد.

۹.۷ پیام گروهی

گروه یک گفتگوی واقعی است، اما ارسالش پخش پیامک است. چون MMS نداریم، پاسخ‌ها به گفتگوهای تک‌نفره برمی‌گردند. یک گفتگو، یک حباب برای هر ارسال، N پیامک واقعی.

- **شناسهٔ گفتگو `g:<groupId>` است**، یک `thread_id` عادی که با شماره تداخل ندارد.
- **`message_group_targets` جدول کلیدی است.** یک پیام گروهی، یک ردیف `messages` با `device_sms_id` خالی و N ردیف در provider دارد. `knownDeviceSmsIds` این جدول را هم UNION می‌کند. وگرنه همگام‌سازی بعدی N-1 کپی در گفتگوهای تک‌نفره می‌ساخت.
- **تیک حباب بدترین وضعیت گیرندگان است** (`GroupSendSummary`). «اطلاعات» فهرست هر نفر را نشان می‌دهد و «ارسال مجدد» فقط ناموفق‌ها را می‌فرستد.
- **گزارش تحویل با `tracking_id`** جدای هر گیرنده مسیریابی می‌شود. `SmsService` پیش از مسیر شناسهٔ پیام، `GroupRepository.applyTargetStatus` را می‌پرسد.
- **حذف، ردیف‌ها را یکی‌یکی نام می‌برد.** پیام‌ها نرم حذف می‌شوند و targetها می‌مانند.
- **گروه یک‌نفره گروه نیست** و به گفتگوی تک‌نفره برمی‌گردد.
- **`findOrCreate` مجموعهٔ اعضا را کلید می‌گیرد.** انتخاب همان سه نفر همان گروه را باز می‌کند.
- **عنوان تا وقتی تنظیم نشده ساخته می‌شود:** «علی و مریم»، «علی، مریم و رضا»، سپس «علی، مریم و ۲ نفر دیگر».
- **زمان‌بندی در گروه خاموش است**، چون ردیف زمان‌بندی یک شماره دارد و کارگر بومی چیزی از گروه نمی‌داند.
- **ورودی‌ها:** «شروع گفتگوی گروهی» در انتخابگر و «پیامک گروهی» در صفحهٔ برچسب.
- **نتیجهٔ جستجوی عددی گروه‌ها را نمی‌آورد**، چون UUID پر از رقم است. گروه با نام یا عضو پیدا می‌شود.

۹.۸ فیلتر برچسب در صندوق

ردیف چیپ زیر سرصفحهٔ صندوق گفتگوها را با برچسب مخاطب فیلتر می‌کند و وقتی فهرست کوتاه است صفحهٔ بعد را هم می‌خواند. منبع و ویجت با تب مخاطبین مشترک است (`ContactFilters`، بخش ۱۶).

۹.۹ گفتگوی روی گفتگو

`MessageBloc` سراسری است و در هر لحظه یک گفتگو دارد. پس `ConversationScreen._isMine` هر خواندن را با `state.threadId` فیلتر می‌کند و `_lastLoaded` آخرین صفحهٔ همان گفتگو را نگه می‌دارد. صفحه `RouteAware` است و `didPopNext` دوباره `LoadMessages` را می‌فرستد و `setVisibleThread` را پس می‌گیرد.

بخش ۱۰. صفحهٔ گفتگو و نوشتن پیام

۱۰.۱ فایل‌های اصلی

فایل	کار
`messages/screens/conversation_screen.dart`	صفحهٔ گفتگو با ۲٬۱۱۴ خط. وضعیت، ارسال، پیش‌نویس، زمان‌بندی
`messages/screens/widgets/message_bubble.dart`	حباب و لمس آن
`message_bubble_body`	بدنهٔ حباب، مشترک بین فهرست و نسخهٔ بلندشده
`message_action_overlay.dart`	بلند کردن حباب با لمس طولانی
`linkified_text.dart`	لینک، USSD، شماره و جهت متن
`message_composer.dart`	جعبهٔ نوشتن
`emoji_panel.dart`	صفحهٔ ایموجی
`core/widgets/caret_line_fix.dart`	اصلاح جای نشانگر متن
`core/utils/script_direction.dart`	جهت هر پاراگراف
`core/utils/ussd_code.dart`	تشخیص و اصلاح USSD

۱۰.۲ یک گفتگو فقط گفتگوی خودش را نشان می‌دهد

`MessageBloc` سراسری است. وقتی روی یک گفتگو گفتگوی دیگری باز شود، مثلاً با «هدایت»، صفحهٔ زیرین همچنان زنده است و بازسازی می‌شود. `ConversationScreen._isMine` هر state را با `threadId` می‌سنجد و `_lastLoaded` آخرین صفحهٔ همان گفتگو را نگه می‌دارد.

۱۰.۳ حباب

- **سمت‌ها فیزیکی‌اند.** حباب پیام با `textDirection: ltr` چیده می‌شود: ارسالی راست و دریافتی چپ، مثل همهٔ پیام‌رسان‌ها. جهت خود متن از این پیروی نمی‌کند.
- **هر پاراگراف جهت خودش را دارد.** `ScriptDirection.paragraphs` متن را به تکه‌هایی از خط‌های هم‌جهت می‌برد و برای هر تکه یک `Text` می‌کشد. جهت پاراگراف از **اکثریت حروف** می‌آید، نه اولین حرف. پس «Hamresan یک پیام‌رسان امن است» فارسی می‌ماند. تساوی راست‌به‌چپ است. رقم و ایموجی شمرده نمی‌شوند.
- **کپی خط جدید را برمی‌گرداند.** `SelectionArea` متن فرزندانش را بدون فاصله به هم می‌چسباند، پس `_ParagraphsSelection` آن‌ها را با `\n` پیوند می‌زند.
- **زیرنویس فقط برای پیام لمس‌شده است** (`_expandedMessageId`): زمان، سیم‌کارت، تیک و توضیحش. پیام ناموفق همیشه ردیف قرمز دارد، چون کاربر باید کاری بکند.
- **تیک:** ⏱ در حال ارسال، ✓ ارسال شد، ✓✓ تحویل شد. شناسهٔ پیام پیش از ارسال بومی به‌عنوان `trackingId` ساخته می‌شود و با eventهای `{"type":"status"}` برمی‌گردد.

۱۰.۴ لمس طولانی و انتخاب متن

لمس طولانی روی حباب حباب را بلند می‌کند. `MessageBubble` جای حباب و محل انگشت را اندازه می‌گیرد و به `showMessageActionOverlay` می‌دهد. این تابع یک `PopupRoute` با پس‌زمینهٔ تار است و کارت کارها («ستاره، کپی، هدایت، اطلاعات، انتخاب، حذف») کنار لبهٔ حباب می‌آید.

- **حباب بلندشده بزرگ‌نمایی نمی‌شود.** `SelectableRegion` جای ذره‌بین را فقط از translation تبدیل می‌خواند. با `Transform.scale` ذره‌بین و دستگیره‌ها از جای درست دور می‌شوند.
- **بدنه همان ویجت است.** `SelectableBubbleText` همان `LinkifiedText` را داخل `SelectionArea` می‌گذارد. `SelectableText` و `TextField` حاشیهٔ نشانگر دارند و متن را یک خط طولانی‌تر می‌شکنند.
- **هنگام باز شدن چیزی انتخاب نیست.** لمس طولانی داخل حباب بلندشده کلمه را انتخاب می‌کند.
- **«جستجو در وب»** از خود ماست. `SelectableBubbleText` متن انتخاب‌شده را از `onSelectionChanged` می‌گیرد و `google.com/search?q=…` را باز می‌کند. متن انتخاب در یک فیلد ساده است، نه `setState` ، تا دستگیره‌ها به هم نریزند.
- **برچسب‌های نوار انتخاب دستی‌اند،** چون `MaterialLocalizations` فارسی نداریم.
- **لرزش:** هر لمس طولانی `HapticFeedback.mediumImpact()` دارد.

۱۰.۵ لینک، شماره و USSD داخل پیام

`LinkifiedText` به این ترتیب تطبیق می‌دهد: لینک کامل (`http(s)://` ، `www.`)، لینک بدون scheme، USSD و شماره تلفن.

- **لینک بدون scheme:** یا دامنه با TLD فهرست‌شده (`digikala.com`) یا هر TLD حرفی با مسیر (`b2n.ir/xK9`). «1.500.000» و «report.pdf» لینک نمی‌شوند.
- **`isWebLink` و `webUriOf`** تنها جاهایی‌اند که لینک از شماره جدا و `https://` اضافه می‌شود. پیش‌نمایش لینک (`firstUrl`) فقط برای آدرس صریح است.
- **لمس شماره** `showPhoneActionSheet` را باز می‌کند (تماس، پیامک، مخاطب، کپی).

> لمس شماره هرگز نباید `tel:` را اجرا کند. این برنامه تلفن پیش‌فرض است و intent به خودش برمی‌گردد: رابط حدود ۹ ثانیه یخ می‌زد و ممکن بود اکتیویتی از بین برود.

USSD

- **با کاراکترهای `*` و `#` متن وارونه می‌شد:** `#140*11#` به شکل `#11*140*` درمی‌آمد. هر لینک داخل یک isolate چپ‌به‌راست (U+2066 و U+2069) کشیده می‌شود. این فقط هنگام نمایش است و چیزی در پایگاه‌داده تغییر نمی‌کند.
- **`showUssdActionSheet`** شماره‌گیری با هر سیم‌کارت را جدا نشان می‌دهد، چون پاسخ مانده به سیم‌کارت بستگی دارد.
- **قانون شکل از AOSP است** (`isMmiCode`): با `*` یا `#` شروع و با `#` تمام می‌شود.
- **کد آینه‌شده اصلاح می‌شود.** ایرانسل گاهی `#5*555*` می‌فرستد. `UssdCode.correctedOf` اول از جلو می‌خواند و فقط اگر کد نبود از عقب می‌خواند. پس `#100#` برعکس نمی‌شود.
- **`toDialable`** هر دو رقم فارسی و عربی را به لاتین می‌برد. `CallInCallService.onCallAdded` شماره‌گیری MMI را رد می‌کند تا صفحهٔ تماس جلوی پیام شبکه نیاید.

۱۰.۶ کد یک‌بارمصرف

`OneTimeCode.find` زیر حباب دریافتی چیپ «کپی» می‌گذارد. دو شرط دارد: پیام باید کلمه‌ای مثل رمز، کد، پویا یا otp داشته باشد؛ و رقم باید مستقل باشد: ۴ تا ۸ رقم، تکهٔ عدد بزرگ‌تر نباشد، واحد مثل تومان نداشته باشد و سه صفر یا بیشتر در انتها نداشته باشد. کد به رقم لاتین کپی می‌شود و کدهای حرفی عمداً پشتیبانی نمی‌شوند.

۱۰.۷ جعبهٔ نوشتن

- **بلندی:** فیلد حدود ۱۰ خط رشد می‌کند و بعد داخل خودش اسکرول می‌شود. `_maxLinesFor` از فضای واقعی می‌سازد و ارتفاع خط **مقیاس‌شده** است. `keyboardInset` از بیرون می‌آید، چون داخل `Scaffold` مقدار `viewInsets.bottom` صفر است.
- **شمارندهٔ بخش‌ها** به شکل «۴۶/۹» و داخل `Directionality(ltr)` است.
- **رنگ انتخاب:** `selectionHeightStyle: max` و `selectionWidthStyle: tight` .
- **صفحه‌کلید خودش برنمی‌گردد.** بستن صفحه‌کلید فوکوس را هم می‌گیرد (`_lastInset`).
- **پیش‌نویس گفتگو:** `ComposerDraftStore` متن را ذخیره می‌کند. متن آماده (`initialText`، از هدایت، اشتراک یا `sms:?body=`) جای پیش‌نویس ذخیره‌شده را می‌گیرد، اگر فیلد خالی باشد.
- **دکمهٔ رفتن به پایین** داخل `Stack` فهرست است، نه `floatingActionButton` که روی جعبهٔ نوشتن می‌آمد.

اصلاح جای نشانگر (`CaretLineFix`)

Flutter در پاراگراف شکسته‌شدهٔ فارسی نشانگر را جایی جز `EditText` اندروید می‌گذارد. این ویجت سه مشکل را اصلاح می‌کند:

- لمس بعد از آخرین کلمهٔ خط یک حرف جلوتر می‌رفت و Backspace فاصله را پاک می‌کرد (`_beforeHangingSpace`).
- لمس لبهٔ راست خط به انتهای خط بالا می‌رفت (`_correct`).
- کشیدن دستگیره با حرکت جانبی کم به خط دیگر می‌پرید (`_clampToDraggedLine`).

این ویجت نباید برای تایپ، انتخاب بازه، یا حرکت نشانگر بدون انگشت اجرا شود. `--dart-define=CARET_TRACE=true` لاگ هر حرکت را می‌نویسد.

دکمهٔ «+»

گزینه	کار
قالب آماده	`showTemplatePicker`
زمان‌بندی ارسال	برگهٔ زمان‌بندی
موقعیت	مختصات را به متن اضافه می‌کند: رقم لاتین، LRM در ابتدا، ۶ رقم اعشار. `LocationHandler.kt` با `LocationManager` کار می‌کند، و `getLastKnownLocation` فقط تا ۲ دقیقه معتبر است. نگهبان `replied` پاسخ دوم را می‌گیرد
تصویر کوچک	پایین

«دوربین»، «گالری» و «صدا» عمداً حذف شده‌اند، چون MMS نداریم.

«تصویر کوچک»

`TinyImage` یک عکس کوچک را به نویسه تبدیل می‌کند و به شکل پیامک متنی می‌فرستد:

```
[#I1:56x42:r]<base64url>
```

- ۱ بیت برای هر پیکسل، خام یا PackBits، هرکدام کوتاه‌تر باشد. همهٔ نویسه‌ها GSM-7 هستند و هر بخش ۱۵۳ نویسه جا دارد.
- سه اندازه: ۴۰، ۵۶ و ۷۲ پیکسل، با حداکثر ۲، ۴ و ۶ پیامک. کنتراست کشیده و تصویر dither می‌شود.
- فقط از گالری. چون `CAMERA` در مانیفست هست و بدون آن مجوز، `ACTION_IMAGE_CAPTURE` رد می‌شود.
- مستقیم از برگهٔ پیش‌نمایش فرستاده می‌شود، چون تغییر یک نویسه تصویر را خراب می‌کند.
- `TemplateWire.displayText` در Dart و Kotlin کلمهٔ «تصویر» را برمی‌گرداند. فقط `MessageBubbleBody` آن را با `TinyImageView` می‌کشد.
- سیاه روی سفید در هر دو پوسته.
- نسخهٔ جدید مثل `[#I2:` تطبیق نمی‌خورد و متن خام می‌ماند. `tiny_image_test.dart` این متن را ثابت نگه می‌دارد.
- هدایت، خود payload را می‌فرستد و تأیید می‌گیرد.

۱۰.۸ صفحهٔ ایموجی

`EmojiPanel` نوار دسته، اسکرول پیوسته، «اخیر»، رنگ پوست با لمس طولانی و Backspace خودش را دارد.

- **بستن پنل باید صفحه‌کلید را باز کند:** `requestFocus()` و `TextInput.show` با هم، چون فوکوس ممکن است از قبل باشد.
- **قوانین ارتفاع:** ارتفاع صفحه‌کلید فقط هنگام **بالا آمدن** و با فوکوس ثبت می‌شود. از `clamp` استفاده نکنید، چون حد پایین می‌تواند بیشتر از حد بالا شود و خطا بدهد. ارتفاع رسم‌شده برابر هدف منهای `inset` فعلی است.
- **تب فعال** یک `ValueNotifier` است تا اسکرول پنل را بازسازی نکند.
- **رنگ پوست:** جایگزین `FE0F` می‌شود، وگرنه یک مربع رنگی اضافه می‌آید.

۱۰.۹ «اندازه متن پیام»

یک مقدار ماندگار (`SettingsState.messageTextScale`) با سه راه تغییر: نیشگون، منوی گفتگو و تنظیمات.

- `MessageTextScaler` با مقیاس گوشی ترکیب می‌شود و جایگزینش نمی‌شود، و `==` و `hashCode` دارد تا هر فریم همه چیز بازسازی نشود.
- فقط روی فهرست و جعبهٔ نوشتن اعمال می‌شود. نوارها و بنرها بیرون‌اند، چون در ۲۰۰٪ می‌شکنند.

> نیشگون با `Listener` خام است، نه `ScaleGestureRecognizer`. تشخیص‌دهندهٔ کشیدن فهرست پایین‌تر است و وقتی انگشت اول حرکت کند، بقیه را رد می‌کند. پس انگشت دوم دیگر دیده نمی‌شد و نیشگون فقط با تماس کاملاً هم‌زمان کار می‌کرد. `Listener` در رقابت حرکت‌ها شرکت نمی‌کند.

- `PinchScope` هنگام دو انگشت فهرست را قفل می‌کند (`NeverScrollableScrollPhysics`).
- `_gain = 1.6` حساسیت است. روی یکی از `MessageTextScale.steps` می‌نشیند و یک بار، در رها کردن، ذخیره می‌شود.

۱۰.۱۰ ارسال ردشده پیام را نگه می‌دارد

- **ردیف پیش از رادیو نوشته می‌شود،** با وضعیت `pending` . قبلاً در حالت پرواز هیچ چیزی نمی‌ماند.
- `_transmit` **تنها مسیر** ارسال اول، «ارسال مجدد» و تحویل زمان‌بندی است.
- **ردیف ناموفق فقط محلی است** (`device_sms_id` خالی) و همگام‌سازی آن را حذف نمی‌کند.
- **تلاش دوباره همان ردیف را می‌فرستد** (`RetryMessage` و `resendMessage`). ردیفی که `failed` نباشد رد می‌شود، پس لمس دوتایی دو پیام نمی‌سازد.
- **تحویل زمان‌بندی** `optimistic: false` **می‌فرستد،** وگرنه برای هر تلاش یک حباب مرده می‌ماند.

اعلان‌های یک‌باره در کانال state

`MessageSent` ، `MessageSendFailed` و `MessageError` حالت صفحه نیستند، ولی از همان کانال state می‌آیند. برای جلوگیری از گیر کردن صفحه روی چرخانک:

- `MessageBloc._lastDurable` آخرین `ThreadsLoaded` یا `MessagesLoaded` است. نگهبان بارگذاری آن را می‌پرسد، نه `state` .
- `_notify` اعلان را می‌فرستد و بلافاصله به `_lastDurable` برمی‌گردد.
- `ConversationScreen` روی `MessageSendFailed` هم خودش را دوباره بار می‌کند.

۱۰.۱۱ پیشنهاد هرزنامه

برای شمارهٔ ذخیره‌نشده‌ای که پیام داده: «این شماره در مخاطبین شما نیست. هرزنامه است؟». پاسخ در `SpamPromptStore` ذخیره می‌شود (حداکثر ۵۰۰ مورد). تا `_contactResolved` نشده نمایش داده نمی‌شود، چون گفتگویی که از اعلان باز شده فقط شماره دارد.

بخش ۱۱. پیش‌نویس و قالب آماده

۱۱.۱ پیش‌نویس‌ها و دسته‌ها

دو صفحه و یک `DraftBloc` :

- **`DraftsListScreen`** برد دوستونی است، چون طول پیش‌نویس‌ها متفاوت است. `SliverTwoColumnBoard` (`core/widgets/two_column_board.dart`) کارت‌ها را یکی‌درمیان در دو `SliverList` داخل `SliverCrossAxisGroup` می‌گذارد.
- **`SliverCrossAxisGroup` برای RTL آینه نمی‌شود.** ستون اول دستی انتخاب می‌شود. اگر این را بردارید، جدیدترین پیش‌نویس به چپ می‌رود.
- **فیلتر دسته** یک ردیف چیپ است: «همه»، «بدون دسته‌بندی» و یکی برای هر دسته.
- **`MessageCategoriesScreen`:** نام و تعداد. «همه» و «بدون دسته‌بندی» ردیف مجازی‌اند و انتخاب نمی‌شوند، چون جدولی پشت آن‌ها نیست.
- `active` (فیلتر نمایش‌داده‌شده) و `selected` (انتخاب گروهی) دو چیز جدا هستند و می‌توانند با هم درست باشند.
- **حذف دسته** پیش‌نویس‌هایش را به «بدون دسته‌بندی» می‌برد، در یک تراکنش. `DraftBloc` فیلتری را که به دستهٔ حذف‌شده اشاره می‌کند پاک می‌کند.
- **انتخاب گروهی** در حالت `pickMode` خاموش است و شناسه‌های ازبین‌رفته بعد از فریم پاک می‌شوند.
- **سنجاق:** `_onSave` پرچم قبلی را نگه می‌دارد.

۱۱.۲ قالب آماده

قالب متنی با جایگاه‌های `[...]` است. `TemplateEngine` (`models/message_template_model.dart`) از روی جایگاه‌ها فرم می‌سازد و جواب‌ها را جایگزین می‌کند. **برای هیچ قالبی کد جدا وجود ندارد**، پس قالبی که کاربر می‌نویسد همان صفحهٔ پر کردن را می‌گیرد. قالب‌های داخلی با `DatabaseHelper._seedMessageTemplates` در جدول قرار می‌گیرند و ردیف عادی‌اند.

نام جایگاه	نوع فیلد
تاریخ، مورخ	انتخاب تاریخ
ساعت، زمان، وقت	انتخاب ساعت
توضیح، متن، آدرس، نشانی، پیام	چندخطی
بقیه	یک‌خطی

- **«تاریخ» و «زمان» یک انتخابگر می‌شوند** (`fieldsOf`).
- **`TemplateFillScreen`:** کلید «درج نام مخاطب»، فیلدها، پیش‌نمایش زنده، انصراف و تأیید. جواب‌ها در یک map ساده و یک `ValueNotifier` هستند تا تایپ فیلدها را بازسازی نکند.
- **پیش‌نمایش جایگاه پرنشده را نگه می‌دارد و درج آن را برمی‌دارد.** حذف جایگاه حرف ربط قبلش را هم برمی‌دارد (`_dropDanglingConnector`).
- **قالب بدون سؤال** با یک لمس درج می‌شود (`needsInput`).
- **`TemplatesListScreen`** در `pickMode` متن آماده را برمی‌گرداند و در حالت مدیریت ویرایش و انتخاب گروهی دارد.

۱۱.۳ قالب پیامکی روی سیم

فارسی در هر بخش فقط ۷۰ نویسه جا دارد. پس قالب **داخلی** متنش را نمی‌فرستد و فقط یک سرآیند و جواب‌ها را می‌فرستد:

```
[#T1:mtg:1](۴۹ نویسه)    ۳ اتاق |۱۰:۳۰|۱۴۰۵/۰۵/۲۰|علی|جلسه هفتگی
علی عزیز⏎جلسه جلسه هفتگی در مورخه ۱۴۰۵/۰۵/۲۰ ساعت ۱۰:۳۰ در محل اتاق ۳ برقرار می‌باشد.  (۸۵ نویسه)
```

تکه	معنی
`#T1`	نسخهٔ ۱ قالب
`mtg`	`BuiltInTemplate.code`
`1`	بیت‌های پرچم؛ بیت ۰ یعنی تکهٔ اول خطاب «<نام> عزیز» است
بقیه	یک تکه برای هر جایگاه به ترتیب `tokens`. `\`، `|` و خط جدید escape می‌شوند و خالی‌های انتها حذف می‌شوند

`TemplateWire.encode` در این حالت‌ها **null** می‌دهد و متن عادی فرستاده می‌شود: قالب کاربر، قالب داخلی ویرایش‌شده، قالب بدون جایگاه، و وقتی خروجی کوتاه‌تر نیست. گیرندهٔ بدون این برنامه فقط سرآیند و کلمه‌های تایپ‌شده را می‌بیند.

قانون‌ها

- **به ثابت‌های کامپایل‌شده وابسته است، نه ردیف پایگاه‌داده.** اگر شناسهٔ ردیف می‌رفت، ویرایش یک طرف پیام طرف دیگر را عوض می‌کرد. `BuiltInTemplates.of` فقط تا وقتی متن دست‌نخورده است قالب داخلی را برمی‌گرداند. **هیچ `code` را دوباره استفاده نکنید**، چون پیام‌های قدیمی با آن باز می‌شوند.
- **پایگاه‌داده همان متن روی سیم را نگه می‌دارد.** این متن با `content://sms` مقایسه می‌شود. نمایش از `TemplateWire.displayText` می‌گذرد و کپی و هدایت هم متن نمایشی را می‌برند.
- **جعبهٔ نوشتن payload را نشان نمی‌دهد.** `TemplateFillResult` متن انسانی و payload را با هم دارد. `ConversationScreen` فقط وقتی payload را می‌فرستد که متن فیلد حرف‌به‌حرف یکی باشد. `_outgoingBody` هم برای ارسال و هم برای زمان‌بندی است.
- **Kotlin:** `TemplateWire.kt` فقط نیمهٔ دریافت را دارد، چون اعلان‌ها بومی‌اند. regex، نسخه، escape و متن قالب‌ها باید حرف‌به‌حرف با Dart یکی باشند. اختلاف خطا نمی‌دهد و پیام دیگری نشان می‌دهد.

بخش ۱۲. پیام زمان‌بندی‌شده

۱۲.۱ رابط کاربری

یک برگه است، بدون صفحهٔ جدا. لمس طولانی روی ارسال یا «زمان‌بندی ارسال» در «+» برگهٔ `showScheduleSendSheet` را باز می‌کند. این برگه فقط یک `ScheduleChoice` برمی‌گرداند. جعبهٔ نوشتن بنر نشان می‌دهد و دکمهٔ ارسال `schedule_send` می‌شود. ردیف وقتی نوشته می‌شود که ارسال زده شود. ویرایش همان برگه را با `ScheduleChoice.fromMessage` باز می‌کند.

- **دو خروجی:** زمان سریع فوراً برگه را می‌بندد؛ «انتخاب تاریخ و ساعت» فقط لحظه را تنظیم می‌کند و «تأیید» می‌بندد. بدون دومی، تغییر پراکندگی بدون دست زدن به زمان ممکن نبود.
- **«پراکندگی زمان ارسال» زیر «تکرار» نیست،** چون برای یک ارسال تکی هم کاربرد دارد.
- **ساعت‌شمار `dialOnly` است.** حالت ورود از صفحه‌کلید در فارسی خراب است، چون Flutter رقم فارسی را با `int.tryParse` می‌خواند.
- **ردیف گفتگو بالا می‌آید** (`scheduledText` ، `scheduledTime`) و ⏱ می‌گیرد.

۱۲.۲ دو تحویل‌دهنده

تحویل‌دهنده	کجا	چه زمانی
Dart	`ScheduledMessageBloc` و `ScheduledDeliveryService.deliverDue()`	هر ۳۰ ثانیه
Kotlin	`ScheduledSmsWorker.processDue()`	با آلارم، وقتی برنامه بسته است

اگر موتور Flutter زنده باشد، `ScheduledSmsAlarmReceiver` کار را از کانال `deliverDue` به Dart برمی‌گرداند. این مهم است: کارگر بومی مستقیم در SQLite می‌نویسد و BLoCها کهنه می‌مانند. `MainActivity` کانال را در `configureFlutterEngine` منتشر می‌کند و در `onDestroy` پاک می‌کند.

> **هیچ پیام زمان‌بندی‌شده‌ای بدون گرفتن قفل فرستاده نمی‌شود.** `ScheduledMessageRepository.claimDue(now, token)` با یک `UPDATE` ردیف‌ها را با `status='sending'` و `claim_token` مهر می‌زند و هر تحویل‌دهنده فقط ردیف‌های توکن خودش را می‌فرستد. ردیفی که بیشتر از `staleClaimTimeout` در `sending` بماند آزاد می‌شود.

۱۲.۳ `lazy: false`

`ScheduledMessageBloc` باید از اول ساخته شود. قبلاً فقط از صفحهٔ گفتگو خوانده می‌شد و در اجرای عادی اصلاً ساخته نمی‌شد. پس آلارم دوباره تنظیم نمی‌شد و پیام ۹ صبح، ساعت ۱۱ که کاربر برنامه را باز می‌کرد فرستاده می‌شد.
`MainNavigation` هم در هر resumed آلارم را دوباره تنظیم می‌کند، چون Doze و باتری‌سازهای شرکت‌ها ممکن است آن را بیندازند.

۱۲.۴ پراکندگی

هر سه طرف باید **یک مقدار** حساب کنند. `ScheduledMessage.jitterOffset` مقدار قطعی از `jitterSeed` می‌سازد: FNV-1a سی‌ودو بیتی روی واحدهای UTF-16 شناسه، زمان و شمارهٔ تکرار. این مقدار با بردارهای مرجع در

`scheduled_message_test.dart` ثابت شده و در `ScheduledSmsWorker.jitterSeed` حرف‌به‌حرف تکرار می‌شود.
`Object.hash` و `String.hashCode` بین دو زبان فرق دارند و استفاده از آن‌ها ممنوع است.

- `isDueAt` با `effectiveSendAt` مقایسه می‌کند.
- پنجره در SQL قابل بیان نیست، پس `ScheduledDeliveryService` می‌خواند، فیلتر می‌کند و شناسه‌ها را به `claimDue(…, restrictTo:)` می‌دهد.
- `earliestPending` زمان مؤثر را می‌دهد و آلارم دقیقاً همان را تنظیم می‌کند.
- **«ارسال فوری»** همهٔ این‌ها را رد می‌کند: `SendScheduledNow` با `deliverDue(force: {id})`.

۱۲.۵ آلارم

فاصله تا ارسال	نوع آلارم	چرا
کمتر از ۲۴ ساعت	`setAlarmClock`	تنها آلارمی که App Standby عقب نمی‌اندازد
بیشتر	`setExactAndAllowWhileIdle`	تا نماد ساعت چند روز نماند

`AlarmClockInfo` برای نمایش، لانچر برنامه را باز می‌کند، نه broadcast تحویل. با `dumpsys alarm` می‌توانید آن را به‌صورت `Next wake from idle` ببینید.

۱۲.۶ بقیهٔ قانون‌ها

- ارسال ناموفق `next_attempt_at` را عقب می‌برد. بعد از `maxAttempts` وضعیت `failed` می‌شود.
- `advanceAfterSend()` تکرارهای ازدست‌رفته را رد می‌کند و دوباره نمی‌فرستد.
- `ScheduledSmsWorker` آینهٔ منطق تکرار، تلاش دوباره و قفل در `scheduled_message_model.dart` است.
- کارگر بومی پیام را در `messages` هم می‌نویسد.
- هر پیام خروجی روی `SmsService.onMessageSent` منتشر می‌شود و `MessageBloc` آن را به گفتگوی باز اضافه می‌کند.
- `DeliverDueScheduled` جدول را در هر تیک دوباره می‌خواند، چون ممکن است کارگر بومی ردیفی را تمام کرده باشد.
- `BootReceiver` بعد از روشن شدن گوشی آلارم را تنظیم می‌کند.
- فهرست همهٔ زمان‌بندی‌ها در `ScheduledMessagesScreen` است.

بخش ۱۳. تماس و شماره‌گیر

این حساس‌ترین قسمت برنامه به مدل گوشی است و با آزمون خودکار کامل پوشش داده نمی‌شود. پیش از هر تغییر این بخش را کامل بخوانید و بعد از تغییر روی گوشی واقعی آزمایش کنید.

۱۳.۱ فایل‌های اصلی

فایل	خط	کار
`kt/call/CallInCallService.kt`	۱٬۹۹۱	قلب تماس: دریافت همهٔ تماس‌ها از Telecom، کارت‌ها، وضعیت
`kt/call/CallHandler.kt`		کانال `call`
`kt/call/CallRules.kt`		قانون‌های خالص مسدودی
`kt/call/CallPrefs.kt`		کپی بومی تنظیمات تماس
`kt/call/ProximityGate.kt`		خاموش شدن صفحه کنار گوش
`kt/call/CallConnectionService.kt`		مسیر VoIP غیرفعال
`kt/MainActivity.kt`	۷۳۴	پنجره، صفحهٔ قفل، recents
`core/navigation/call_ui_coordinator.dart`		چه صفحه‌ای و چه زمانی باز شود
`core/navigation/return_to_call_bar.dart`		نوار بازگشت به تماس
`dialer/bloc/dialer_bloc.dart`	۸۵۸	وضعیت شماره‌گیر و تماس
`dialer/screens/incoming_call_screen.dart`		تماس ورودی
`dialer/screens/in_call_screen.dart`	۱٬۰۷۳	تماس جاری
`dialer/screens/dialer_screen.dart`، `widgets/dialer_widgets.dart`		شماره‌گیر

۱۳.۲ نقش تلفن پیش‌فرض

برنامه `ROLE_DIALER` را می‌گیرد (`CallHandler.requestDefaultDialerRole`). وقتی این نقش را دارد، Telecom سرویس `CallInCallService` را bind می‌کند و **تنها رابط تماس گوشی همین برنامه است**.

```
Telecom → CallInCallService.onCallAdded → Call.Callback
       → CallEventStreamHandler → DialerBloc.callStatus → CallUiCoordinator → صفحهٔ تماس
```

`stickyState` در `onListen` دوباره پخش می‌شود تا موتور تازه‌راه‌افتاده تماس زنگ‌خورده را از دست ندهد.

> هرگز `ROLE_DIALER` را بدون `CallInCallService` فعال نخواهید. آن وقت تماس ورودی هیچ رابطی ندارد.

۱۳.۳ چه چیزی تماس زنده است

تماس فقط در `CallInCallService.LIVE_STATES` زنده است: `SELECT_PHONE_ACCOUNT`، `CONNECTING`، `DIALING`، `RINGING`، `ACTIVE`، `HOLDING`، `PULLING_CALL` و `SIMULATED_RINGING`. Telecom تماس را در `STATE_NEW` اضافه می‌کند و سامسونگ هنگام کنفرانس و حتی بعد از قطع کامل هم چند تا اضافه می‌کند. شمردن این‌ها همان خطایی است که صفحهٔ تماس را با تایمر روشن بعد از پایان نگه می‌داشت.

چهار مسیر پایان تماس

صفحهٔ گیرکرده بدون کشتن برنامه درست نمی‌شود، پس چهار مسیر مستقل وجود دارد:

۱. `onCallRemoved`

۲. `onStateChanged`، وقتی چیز زنده‌ای نمانده

۳. `republishCurrent()`

۴. در Dart: `CALLS_CHANGED` با `count == 0` و `SyncCallState` در هر بازگشت به برنامه که مستقیم از Telecom می‌پرسد

خطای کانال نباید رابط تماس زنده را خراب کند، پس `_onSyncCallState` در خطا برمی‌گردد.

۱۳.۴ قانون‌های `CallInCallService`

- **پایان تماس دوم پایان تماس نیست.** تماس انتظار `currentCall` می‌شود. وقتی تمام شد، `liveSuccessorTo` رابط را به تماس باقی‌مانده می‌دهد (ACTIVE مقدم است). فقط وقتی گوشی خالی شد `DISCONNECTED` فرستاده می‌شود.
- **میزبان کنفرانس تماس تازه نیست.** `onCallAdded` برای `PROPERTY_CONFERENCE` برمی‌گردد.
- **وضعیت ادغام در جزئیات است.** Telecom رابطهٔ `parent` را بعد می‌گذارد، پس `onParentChanged`، `onChildrenChanged`، `onDetailsChanged` و `onConferenceableCallsChanged` همگی ثبت شده‌اند. چون پشت سر هم می‌آیند، `publishCallsChanged` و `publishState` تکراری را می‌گیرند، به‌جز `DISCONNECTED` که همیشه فرستاده می‌شود و حافظه را پاک می‌کند.
- **`republishCurrent()` برای تماس مرده چیزی نمی‌فرستد.**
- **هویت تماس‌گیرنده جایگزین می‌شود، نه ادغام.** `DialerState.copyWith` مقدار null را «دست نزن» می‌فهمد، پس شمارهٔ ذخیره‌نشده نام نفر قبلی را می‌گرفت. `clearActiveName` و `clearActiveSubscriptionId` این را درست می‌کنند و `_namesTheParty` فقط برای eventهای حامل شماره اجرا می‌شود.
- **رد تماسی که بعد ACTIVE می‌شود خطا نیست.** روی ایرانسل، رد تماس طرف را به پیام اپراتور می‌برد.
- **DTMF:** فقط صفحهٔ کلید داخل تماس از `startDtmf` استفاده می‌کند. شماره‌گیر از `playKeypadTone` استفاده می‌کند، وگرنه هنگام «افزودن تماس» شماره در گوش طرف اول پخش می‌شود.
- **حداکثر دو تماس.** «افزودن تماس» در دو تماس غیرفعال است.

۱۳.۵ `CallUiCoordinator`

- **بالای احراز هویت است** تا تماس روی PIN هم بیاید. فقط صفحهٔ تماس از پشت قفل دیده می‌شود. ناوبری تماس را به `MainNavigation` نبرید.

- **فهرست route نگه می‌دارد، نه یکی** (`_callRoutes`). یک فیلد تنها با تماس انتظار بازنویسی می‌شد و route قبلی یتیم می‌ماند. `_pushCall` جدید را می‌گذارد و بقیه را حذف می‌کند. `call_ui_coordinator_test.dart` این را ثابت می‌کند.
- **رابطه با قفل برنامه:** تماس زنگ‌خورده همیشه روی قفل می‌آید. قفل هم گذاشته می‌شود و `raiseAboveLock()` تماس را بالا می‌آورد. پس رد تماس برنامه را باز نمی‌گذارد. تماس پاسخ‌داده‌شده قفل نمی‌شود.
- **«رمز برای پاسخ به تماس»** (خاموش به‌طور پیش‌فرض): قفل روی صفحهٔ تماس ورودی می‌آید و دکمهٔ پاسخ کارت همان صفحه را باز می‌کند. تماسی که از هدست یا خودرو پاسخ داده شود پشت PIN نمی‌رود. مقدار با `SettingsBloc.readPinToAnswerCalls()` خوانده می‌شود، چون هنوز `LoadSettings` نرسیده.

۱۳.۶ کارت یا تمام‌صفحه

`CallInCallService.incomingScreenCall` این را در `onCallAdded` و پیش از اولین انتشار تصمیم می‌گیرد:

وضعیت	نتیجه
گوشی در حال استفاده است (`deviceInUse()`)	فقط کارت. `CallUiCoordinator` چیزی باز نمی‌کند و `bringActivityToFront()` صدا زده نمی‌شود
صفحه خاموش یا قفل	تمام‌صفحه
صفحهٔ تماس جلوست (تماس انتظار)	صفحهٔ ورودی همان‌جا، کارت بی‌صدا

- **لمس کارت** `EXTRA_INCOMING_CALL` را می‌فرستد و صفحه باز می‌شود.
- **کارت به `callScreenShowing()` وابسته است**، نه به شروع اکتیویتی. ترک صفحهٔ تماس فقط وقتی کارت پرصدا را دوباره می‌آورد که `deviceInUse()` باشد.
- **Back روی صفحهٔ ورودی** از `CallUiCoordinator.minimize()` می‌گذرد و `appInFrontAtRing` و `returnWhenOver` برمی‌گرداند.
- **`CallStyle`** با `VISIBILITY_PUBLIC` برای دکمه‌های واقعی پاسخ و رد روی صفحهٔ قفل لازم است.
- **`yieldToKeyguard`:** بعد از جمع شدن صفحهٔ تماس روی قفل، پنجره به keyguard واگذار می‌شود.

۱۳.۷ صفحهٔ قفل و پنجره

- **`showOverLockScreen(true)`** با `setShowWhenLocked` و `setTurnScreenOn` تماس را روی قفل می‌آورد. این برای هر تماس جدا روشن می‌شود و در مانیفست نیست، چون فقط تماس حق نمایش روی قفل را دارد.
- **`showOverLockScreen(false)`** فقط وقتی `moveTaskToBack` می‌کند که واقعاً یک تماس پنجره را آنجا آورده باشد (`overLockScreen`). وگرنه برنامه خودش را به پس‌زمینه می‌فرستاد.
- **`FLAG_KEEP_SCREEN_ON` فقط هنگام زنگ** است (`hasRingingCall()`). `syncLockScreenVisibility()` در هر تغییر وضعیت صدا زده می‌شود.
- **`ProximityGate`** هنگام گوشی داخلی `PROXIMITY_SCREEN_OFF_WAKE_LOCK` دارد و روی بلندگو یا هدست آزاد می‌شود.
- **`setRecentsSnapshotEnabled`:** عکس recents در طول تماس خاموش است و فقط وقتی تماسی نمانده در `onResume` روشن می‌شود. وگرنه صفحهٔ تماس پایان‌یافته در فهرست برنامه‌ها یخ می‌زند.
- **`android:taskAffinity=""` حذف شده،** چون برنامه را به‌جای برگشتن دوباره شروع می‌کرد.

۱۳.۸ ترک تماس بدون پایان دادن

سه قطعه که همه لازم‌اند:

- **Back و «کوچک کردن»** route تماس را برمی‌دارند و `CallUiCoordinator.minimized` را تنظیم می‌کنند. `restore()` برمی‌گرداند.
- **`ReturnToCallBar`** از `MaterialApp.builder` و بالای navigator است. شکل درخت باید ثابت بماند، چون navigator زیر آن است و تغییر شکل آن را دوباره می‌سازد. به جای کشیدن روی محتوا، `MediaQuery.padding.top` را زیاد می‌کند.
- **کارت «تماس در جریان»** (`refreshOngoingNotification`) وقتی صفحهٔ تماس جلو نیست. `CallStyle` بدون سرویس پیش‌زمینه یا `fullScreenIntent` خطا می‌دهد و کرش آن تماس را به شماره‌گیر سازنده می‌دهد.
- **لمس کارت** با `EXTRA_RETURN_TO_CALL` یک `SHOW_CALL_UI` روی کانال تماس می‌فرستد، نه از مسیر `consumeLaunchAction` که پشت قفل است.
- **زمان تماس از Telecom است** (`connectTimeMillis`، `callConnectedAt`). صفحه خودش نمی‌شمارد.
- `NativeCallEvent.unknown` eventهای ناشناس را بی‌اثر می‌کند.

۱۳.۹ صدا

خروجی صدا یک route است، نه یک مقدار درست و غلط. `CallAudioState.supportedRouteMask` و `route` با هر event می‌آیند و انتخابگر فقط همان‌ها را نشان می‌دهد. `SelectAudioRoute` منتظر نتیجه نمی‌ماند و مقدار معتبر از Telecom برمی‌گردد. قطع صدا و بلندگو از `InCallService.setMuted` و `setAudioRoute` است، چون `AudioManager` روی تماس Telecom اثر ندارد.

۱۳.۱۰ صفحهٔ کلید داخل تماس

- **بخشی از همان صفحه است** (`_showKeypad`)، نه `showModalBottomSheet`. چون `CallUiCoordinator` route را جابه‌جا می‌کند و برگه از route مرده جدا می‌ماند.
- **`DialerState.dtmfDigits`** رقم‌های فرستاده را نگه می‌دارد و با تماس تازه پاک می‌شود.
- **کلید از لمس تا رها کردن ارسال می‌کند** (`DtmfKeyDown` و `DtmfKeyUp`) با `Listener` خام. `dtmfTarget()` تماس ACTIVE را انتخاب می‌کند و `DTMF_MAX_MS` آن را متوقف می‌کند.

۱۳.۱۱ لمس شماره‌گیر

- **`DialKey` با `Listener` خام روی touch-down اجرا می‌شود.** برگهٔ کشیدنی، تپ را در رقابت حرکت‌ها می‌برد و در تایپ سریع رقم‌ها گم می‌شدند.
- **لمس طولانی یک تایمر است** که با رها کردن لغو می‌شود. چون رقم از قبل تایپ شده، `_longPressFor` اول آن را پاک می‌کند.
- **`DialerScreen` بالای درخت `BlocBuilder` ندارد.** فقط نمایش شماره، پیشنهادها و دکمه بازسازی می‌شوند.
- **دکمه‌های پاسخ و رد** هم همین `Listener` هستند و `_fired` آن‌ها را یک‌بارمصرف می‌کند.
- **نمایش شماره:** لمس، نشانگر را جابه‌جا می‌کند (`DialerState.dialCaret`، null یعنی انتها). لمس طولانی کپی، برش و چسباندن دارد. کلیپ‌بورد با رقم لاتین نوشته می‌شود. متن چسبانده‌شده از `sanitizeDialable` می‌گذرد.

- **اقدام‌های زیر شماره** (`DialerNumberActions`): «ایجاد مخاطب جدید»، «افزودن به مخاطب موجود»، «ارسال پیامک». در حالت «افزودن تماس» حذف می‌شوند.
- **پیشنهادها** بعد از تایپ `*` یا `#` پنهان می‌شوند.
- **دکمهٔ سبز به `isInCall` وابسته نیست،** چون همین کلید برای تماس دوم هم هست.

۱۳.۱۲ USSD و کد MMI

- `CallHandler.makeCall` از `PhoneNumberUtils.stripSeparators` استفاده می‌کند. strip دستی `*` و `#` را پاک می‌کرد.
- آدرس با `Uri.fromParts("tel", clean, null)` ساخته می‌شود. `Uri.parse` در `#` قطع می‌کند.
- **کد MMI تماس نیست.** `onCallAdded` آن را رها می‌کند (`isMmiCode`: با `*` یا `#` شروع و با `#` تمام) و `onCallRemoved` برای تماس ثبت‌نشده برمی‌گردد.

۱۳.۱۳ شماره‌گیری سریع و پست صوتی

- ۰ برای «+»، ۱ برای پست صوتی و ۲ تا ۹ برای شماره‌گیری سریع.
- `SpeedDialService` کش **هم‌زمان** است و از سازندهٔ `DialerBloc` بار می‌شود. نام داخل ردیف هم ذخیره می‌شود.
- فقط از فیلد خالی کار می‌کند. کلید بدون شماره می‌پرسد و صفحهٔ مدیریت `SpeedDialScreen` است.
- `callVoicemail` اول سیم‌کارت را می‌پرسد و بعد شماره را می‌خواند. اگر اپراتور شماره نداده باشد، این را می‌گوید.

۱۳.۱۴ «تماس مجدد خودکار»

خاموش به‌طور پیش‌فرض، ۱، ۲، ۳، ۵ یا ۱۰ تلاش، با ۵ ثانیه شمارش معکوس و «لغو». تنظیمات در `AutoRedialPage` و `AutoRedialPolicy` است.

- **فقط تماس برقرارنشده** (`_redialAfter`): تماس خروجی که `active` نشده، `connectTimeMillis == 0`، و علت `busy`، `remote`، `error` یا `other`. **تماس برقرارشده هرگز تکرار نمی‌شود.** از داخل برنامه، پیام «بعداً تماس بگیرید» اپراتور و صدای آدم فرقی ندارند.
- **یک قطع ممکن است سه `DISCONNECTED` بدهد** و اولی ممکن است علت نداشته باشد (`UNKNOWN`). `_undecidedAttempt` این را نگه می‌دارد. فقط تصمیم روی تماس در حال شماره‌گیری می‌تواند سری را پاک کند (`_RedialVerdict.decided`).
- **`DialerState.autoRedial` کنار `callStatus` است.** سیم‌کارت همان سیم‌کارت تماس ناموفق است و بلندگو هم حفظ می‌شود (`_speakerChosen` از لمس کاربر).
- **`dueAt` پیش از تماس پاک می‌شود** تا زنگ خود سری از تماس دستی جدا شود (`_redialSupersededBy`). تماس ورودی، تماس برقرارشده، پایان، `MakeCall` و «لغو» سری را تمام می‌کنند. `_redialWatchdogTimeout` هم هست.
- **صفحهٔ پایان تماس همان شمارش معکوس است.** Back لغو می‌کند.
- `[redial]` در logcat تصمیم را می‌نویسد، بدون شماره.

۱۳.۱۵ قانون‌های مسدودی تماس

به ترتیب اجرا در `onCallAdded`:

۱. `blocked_numbers`

۲. «مسدود کردن تماس‌های ناشناس»: `handlePresentation` برابر PAYPHONE، RESTRICTED یا UNKNOWN، یا آدرس خالی. مقدار در `CallPrefs` است.

۳. `CallRules.decide` (`blockNonContacts`، `blockInternational`، `blockAdvertising` و `adPrefixes`)

۴. `NumberListRules.allows`

از قانون‌های ۳ معاف‌اند: مخاطب ذخیره‌شده، شمارهٔ دفترچهٔ مخفی، شماره‌ای که در ۷ روز گذشته گرفته‌اید و شمارهٔ کمتر از ۷ رقم. اگر دفترچه خوانده نشود، قانون مخاطبین اعمال نمی‌شود. فهرست تبلیغاتی با `0942` شروع می‌شود و `CallBlockPrefixes.normalize` آینهٔ `CallRules.normalizePrefix` است. صفحهٔ آن `CallBlockRulesPage` است.

۱۳.۱۶ تماس بی‌پاسخ

- **`MissedCallNotificationReceiver`** به Telecom می‌گوید کارت خودش را نفرستد، طبق `shouldManageNotificationThroughDefaultDialer`. این گیرنده چیزی نمی‌فرستد، چون `onCallRemoved` کارت را می‌فرستد.
- `EXTRA_CLEAR_MISSED_CALLS_INTENT` نگه داشته می‌شود و `cancelSystemMissedCallNotification` برای شماره‌گیر سازنده هم هست.
- کارت «تماس بی‌پاسخ» با قفل برنامه نام ندارد. دفترچهٔ مخفی را بخش ۲۵ توضیح می‌دهد.

۱۳.۱۷ اعلان نبودن یعنی تماس نامرئی

اگر اعلان خاموش باشد، کارت و full-screen intent هر دو می‌افتند و هیچ صفحه‌ای نمی‌آید. پس `ensureChannels` از `MainActivity.onCreate` اجرا می‌شود و `DefaultRole.callNotifications` این را می‌خواهد.

۱۳.۱۸ دروازهٔ نقش‌ها (`DefaultAppGate`)

اول پیامک، بعد تلفن، سپس `callNotifications` و `fullScreenIntent`.

- برگهٔ سیستم فقط با لمس کاربر باز می‌شود. اندروید بعد از دو رد برگهٔ سیستم نقش را برای همیشه رد می‌کند.
- در هر resumed دوباره بررسی می‌شود و «فعلاً نه» روی نقش تا resume بعدی است.
- **«فعلاً نه» روی دو مجوز اختیاری** کم‌صداتر می‌شود (`_GrantSnooze`): ۳ روز، ۱۴ روز، بعد دیگر خودکار نمی‌پرسد. هر دو از «تنظیمات ← برنامه‌های پیش‌فرض» در دسترس‌اند.
- هرگز روی تماس نمی‌آید.
- بعد از گرفتن نقش‌ها `SyncDeviceMessages`، `SyncCallLogs` و `RefreshContacts` فرستاده می‌شوند.

بخش ۱۴. «اخیر»: فهرست تماس و یادآور

۱۴.۱ فایل‌های اصلی

فایل	کار
`call_history/bloc/call_log_bloc.dart`	وضعیت فهرست
`call_history/services/call_log_service.dart`	همگام‌سازی و نگاشت
`call_history/services/native_call_log_service.dart`	پوشش کانال
`call_history/repositories/call_log_repository.dart`	SQL
`call_history/screens/call_history_screen.dart`	صفحه
`kt/calllog/CallLogSyncHandler.kt`	خواندن و observer
`kt/reminders/`	یادآورها

۱۴.۲ نقاشی اول

۱. `CallLogBloc._onLoadCallLogs` صفحهٔ محلی را فوراً می‌فرستد، با نام از `ContactNameCache`.

۲. نام‌های معتبر را در یک emit دوم می‌گذارد. اگر چیزی عوض نشده باشد، Equatable رسم دوباره را حذف می‌کند.

۳. گذر روی گوشی جدا و یک بار برای هر bloc اجرا می‌شود (`_backgroundSync`). `ensureSynced` می‌گوید «سراغ گوشی رفتم» که روی گوشی خالی همیشه درست است، پس بدون این قفل حلقه می‌شد.

۴. چرخانک فقط وقتی می‌آید که واقعاً چیزی نیست. «تماس اخیری وجود ندارد» تا اولین خواندن گوشی نشان داده نمی‌شود.

هرگز کل جدول خوانده نمی‌شود. `ensureSynced()` فقط `hasAnyCallLogs()` با `LIMIT 1` را می‌پرسد و ردیف‌ها از `getAllCallLogs(limit: 50)` می‌آیند.

۱۴.۳ همگام‌سازی

محرک	کار
بازگشت به برنامه	فقط ردیف‌های تازه‌تر از جدیدترین ردیف ذخیره‌شده
`ContentObserver` روی `CallLog.Calls`	همان، با بررسی حذف حداکثر هر ۵ دقیقه
کشیدن برای تازه‌سازی	همه چیز

- **اولین ورود** بازهٔ زمانی را تکه‌های ۳۰ روزه می‌خواند و بین آن‌ها فضا می‌دهد. **تازه‌ترین ۷ روز** اول و بیرون `DeviceSyncQueue` می‌آید، و `CallLogBloc` بعد از هر تکه دوباره بار می‌کند.
- `syncFromDevice` یک گذر در هر لحظه اجرا می‌کند و `SyncCallLogs` وسط ورود اول به گذری بعد از آن موکول می‌شود.
- **حذف** فقط شناسه‌ها را می‌خواند (`callLogIdsSince`). ردیف‌های شناسه‌عددی که از گوشی رفته‌اند حذف می‌شوند.
- **نگاشت** روی `Isolate.run` است و غنی‌سازی با مخاطبین روی رشتهٔ اصلی. `_contactIndex` با هویت کش مخاطبین نگه داشته می‌شود.
- `CallType` **در Dart نگاشت می‌شود** تا ثابت ناشناس هم برسد.

- `await SimService.instance.ensureLoaded()` پیش از نگاشت لازم است، چون نتیجه ذخیره می‌شود و بدون آن همهٔ تماس‌ها برای همیشه «بدون سیم» می‌شوند.
- `NativeCallLogService.initialize()` در `LoadCallLogs` دوباره صدا زده می‌شود، چون ممکن است ثبت observer پیش از مجوز باشد.
- `CallLogSyncHandler` هر خواندن را روی یک رشتهٔ جدا اجرا می‌کند (`offMain`).
- `ContactName` شمارهٔ کامل به جای نام نمی‌گذارد و `_normalize` در `CallHistoryScreen` و `call_detail_sheet` با شکل کانونی کار می‌کند.

۱۴.۴ رفتار ردیف

- **لمس:** `CallLogTile` همان‌جا باز می‌شود و فقط دکمه‌ها را نشان می‌دهد.
- **لمس طولانی یا «سابقه»:** `showCallDetailSheet` جدول زمانی تماس و پیامک. از SQLite و صفحه‌به‌صفحه می‌خواند: ۳۰ تا از هر منبع، ادغام‌شده. فقط تا قدیمی‌ترین ردیف بارشدهٔ هر دو منبع نشان می‌دهد تا صفحهٔ بعدی بالای ردیف‌ها وارد نشود.
- **تماس‌های پیاپی یک فرد** یک ردیف «(۲)» می‌شوند.
- `save_number_actions.dart` «ایجاد مخاطب جدید» و «افزودن به مخاطب موجود» را مشترک می‌کند. انتخابگر با `pickNumber: false` و `editableOnly: true` باز می‌شود و ویرایشگر شماره را با تکرار اضافه نمی‌کند.
- **«پاک کردن سابقه تماس»** جدول را پاک می‌کند: `ClearCallLogs` ← `clearAllCallLogsGlobally`. نیمهٔ محلی فقط وقتی اجرا می‌شود که حذف گوشی رد نشده باشد.
- **«اخیر» با قانون عادی سیم‌کارت تماس می‌گیرد** و سیم‌کارت ردیف فقط پیشنهاد است (`placeCall(suggested:)`).

۱۴.۵ «یادآور تماس یا پیامک»

از هر ردیف «اخیر» می‌شود یادآور گذاشت (`setCallReminder`، `call_reminder_sheet.dart`). «یادآورها» بالای فهرست است. سر موعد یک کارت می‌آید که شماره‌گیر با شمارهٔ آماده یا گفتگو را باز می‌کند، با دکمه‌های «شماره‌گیری»، «پیامک» و «۱۰ دقیقه بعد». شماره نمی‌گیرد.

- **Kotlin صاحب فهرست و آلارم است** (`reminders/CallReminders.kt`)، چون آلارم باید بدون موتور اجرا شود. فایل preferences است، نه جدول. `call_reminders.dart` آن را با کانال در یک `ValueNotifier` نگه می‌دارد.
- **یک آلارم برای نزدیک‌ترین** (`rearm`). کمتر از یک روز `setAlarmClock`؛ بیشتر، یک آلارم دقیق یک روز قبل (`ReminderTiming`). `BootReceiver` و هر بازگشت به برنامه دوباره تنظیم می‌کنند.
- **زمان کارت لحظهٔ زنگ است، نه `at`**، چون سیستم کارت با زمان خیلی قدیمی را حذف می‌کند.
- **هنگام قفل** کارت فقط «یادآور» می‌گوید و دکمه ندارد (`NotificationPrivacy.KIND_REMINDER`).
- **شمارهٔ مخفی‌شده** یادآورهایش را با خود می‌برد (`HiddenHandler.setNumbers` و `dropHidden`) و `fireDue` دوباره بررسی می‌کند.
- لینک‌ها `reminderCardId` دارند و `consumeLaunchAction` کارت را برمی‌دارد.

بخش ۱۵. دو سیم‌کارت

۱۵.۱ فایل‌های اصلی

فایل	کار
`kt/sim/SimRegistry.kt`	تنها مرجع سیم‌کارت، حتی بدون موتور Flutter. یک `object` است
`kt/sim/SimHandler.kt`	کانال `sim`
`core/sim/sim_service.dart` ، `sim_bloc.dart` ، `sim_card.dart`	سمت Dart
`core/sim/sim_call.dart`	`placeCall` و `chooseVoiceSim`
`core/sim/widgets/`	چیپ، انتخابگر، `CallIconButton` ، `SimAware`

۱۵.۲ اصول

- **یک سیم‌کارت یعنی هیچ تغییری.** همهٔ امکانات به `SimService.isMultiSim` وابسته‌اند.
- **چیزی که جابه‌جا می‌شود `subscriptionId` است.** این یک شمارندهٔ دستگاه است و slot فقط برای نمایش است.
- **null یعنی نامعلوم، نه سیم ۱.** نشان اشتباه بدتر از نبود آن است.
- **همهٔ خواندن‌های `SimRegistry` در try هستند،** چون `SubscriptionManager` پیش از `READ_PHONE_STATE` خطا می‌دهد و نباید پیامکی گم شود.

۱۵.۳ قانون انتخاب سیم‌کارت

به این ترتیب، از اولویت بالا:

ترتیب	منبع	برای
۰	سیم‌کارت تماس جاری (`CallRoute.LIVE_CALL`). گوشی دوسیم‌کارته فقط با یک کارت تماس دارد	تماس دوم
۱	انتخاب برای همین یک تماس	تماس
۲	کارت مخاطب («سیم‌کارت تماس»، `PREFERRED_PHONE_ACCOUNT`)	تماس
۲	کارت گفتگو («ارسال با»، `thread_sim`)	پیامک
۳	پیش‌فرض برنامه: `SimSettingsPage` ، `setAppDefault` ، با «هر بار بپرس» برای تماس	هر دو
۴	پیش‌فرض گوشی	هر دو
۵	پرسیدن از کاربر	هر دو

این قانون بومی است (`SimRegistry.routeFor` و `defaultSmsFor`)، چون تماس از اعلان و پاسخ سریع بدون موتور انجام می‌شوند. Dart فقط `callRoute` را می‌پرسد تا بداند انتخابگر لازم است یا نه.

- **پیش‌فرض تماس گوشی** از Telecom است و با **نام نبردن کارت** استفاده می‌شود. خواندن از `SubscriptionManager` روی بعضی گوشی‌ها با تنظیمات یکی نبود.

- **پیش‌فرض پیامک** کارت را نام می‌برد (`SmsManager.getDefaultSmsSubscriptionId`).
- **پیش‌فرض‌ها هر بار خوانده می‌شوند:** `SimService.refreshDefaults` پیش از هر تماس و هر بازگشت به برنامه.
- **`STATE_SELECT_PHONE_ACCOUNT`:** `selectAccountFor` کارت مخاطب، پیش‌فرض یا اولی را می‌دهد، چون صفحهٔ تماس انتخابگر ندارد.
- **`placeCall` تنها مسیر تماس است.** `NativeCallService.makeCall` را مستقیم صدا نزنید. انتخابگر بسته‌شده یعنی لغو تماس.
- **راه دستی همیشه هست:** لمس طولانی هر دکمهٔ تماس (`placeCallPickingSim`) و ردیف‌های `simCallRows`.
- **`CallIconButton`** را به جای `IconButton` داخل `GestureDetector` استفاده کنید، چون tooltip در رقابت لمس طولانی برنده می‌شد.

۱۵.۴ نگاشت حساب Telecom به سیم‌کارت

Telecom یک `PhoneAccountHandle` می‌دهد. `SimRegistry.subscriptionOfHandle` همان `TelephonyManager.getSubscriptionId(handle)` است (API 30+) و در API 26 تا 29 با reflection. فقط اگر جواب نداد حدس می‌زند: برابری `handle.id`، سپس ICC ID، سپس برچسب **فقط اگر یکتا باشد**، سپس slot. دو کارت یک اپراتور برچسب یکسان دارند و حدس قدیمی تماس را از کارت اشتباه می‌فرستاد. اگر Telephony برای همهٔ حساب‌ها کارتی گفته و کارت خواسته‌شده نیست، جواب null است. `iccMatches` مقایسهٔ تحمل‌پذیر دارد و `IccMatchTest` آن را آزمایش می‌کند.

۱۵.۵ پیامک

- **`thread_sim` فقط وقتی کاربر انتخاب کرده پر می‌شود.** گفتگوی بدون انتخاب پیش‌فرض را دنبال می‌کند. `SmsNotificationActionReceiver.chosenThreadSim` آینهٔ آن است.
- **بررسی آنتن برای هر سیم‌کارت جداست:** `SmsHandler.isInService(subscriptionId)`.
- **زمان‌بندی `subscription_id` دارد** و کارگر بومی null را به پیش‌فرض تبدیل می‌کند.
- **`MessageModel` را فیلدبه‌فیلد بازسازی نکنید و از `copyWith` استفاده کنید.** وگرنه فیلد تازه گم می‌شود.

۱۵.۶ رابط کاربری

- **چیپ جعبهٔ نوشتن:** نماد سیم‌کارت با شماره. همان `_sim` که ارسال استفاده می‌کند. لمس آن همان `chooseThreadSim` است.
- **چیپ شماره‌گیر:** با `callRoute` و تأخیر ۲۵۰ میلی‌ثانیه حل می‌شود. لمسش برای همین تماس است. هنگام تماس جاری قابل لمس نیست.
- **نام روی دکمه:** زیر «تماس» و روی دکمهٔ ارسال، حتی در جعبهٔ رمز (`SecureComposer._simName`).
- **نام اختصاصی:** `SimService.setSimName`، حداکثر ۱۶ نویسه، به‌صورت بومی در `sim_prefs` با کلید `name_<subId>`. `SimRegistry.labelOf` در اعلان‌ها استفاده می‌کند.
- **`ConversationDetailsScreen`:** صفحهٔ جزئیات گفتگو با کارت «ارسال با». کارهایی که گفتگو را تمام می‌کنند به صورت `ConversationDetailsOutcome` برمی‌گردند.

۱۵.۷ به‌روز ماندن

- **`SimRegistry.subscriptions` کش ۳۰ ثانیه‌ای دارد** و `OnSubscriptionsChangedListener` آن را پاک می‌کند.

- **`SimService.revision` و `SimAware`:** ساختن از فهرست ایستا درست است ولی تغییر را نمی‌فهمد. `SimAware` آن را به بازسازی تبدیل می‌کند.
- **صفحه‌های تماس** `ensureLoaded()` را در `initState` صدا می‌زنند و با `SimAware` پیچیده شده‌اند، چون زنگ معمولاً فرایند مرده را بیدار می‌کند.
- **`_refreshDerived`** پیش‌فرض‌ها و حساب‌ها را هم دوباره می‌خواند.

۱۵.۸ مخاطبان سیم‌کارت

مخاطبان سیم در `ContactsContract` نیستند. `SimContactsHandler.kt` برای هر اشتراک `content://icc/adn/subId/<subId>` را **روی رشتهٔ جدا** می‌خواند، چون بار اول ممکن است چند ثانیه طول بکشد. نشانی `content://icc/adn` همیشه سیم پیش‌فرض را می‌دهد. `ContactRepository._mergeSimContacts` تکراری‌ها را حذف می‌کند. این مخاطبان `ContactSource.sim` هستند: بدون عکس و بدون ویرایش مستقیم («کپی در تلفن»). `ContactBloc` با هر تغییر فهرست سیم‌کارت دوباره می‌خواند.

بخش ۱۶. مخاطبین و موردعلاقه‌ها

۱۶.۱ اصل کار

مخاطب در پایگاه‌دادهٔ برنامه نیست. **منبع، دفترچهٔ تلفن گوشی است.** همهٔ نوشتن‌ها مستقیم روی گوشی انجام می‌شوند و eventی `ContactBloc` برای ساختن، ویرایش یا حذف ندارد. جدول `contacts` از قدیم مانده و چیزی در آن نوشته نمی‌شود.

فایل	کار
`contacts/repositories/contact_repository.dart`	خواندن، کش، تطبیق، مرتب‌سازی
`contacts/bloc/contact_bloc.dart`	وضعیت
`contacts/screens/contacts_list_screen.dart`	تب مخاطبین، ۱٬۲۱۹ خط
`contacts/screens/device_contact_detail_screen.dart`	صفحهٔ مخاطب
`contacts/screens/add_edit_contact_screen.dart`	فرم
`contacts/services/contact_filters.dart`	چیپ‌های فیلتر
`contacts/models/vcf_file.dart` ، `contact_file_service.dart`	فایل `.vcf`
`kt/contacts/ContactExtrasHandler.kt`	خواندن سریع، پیام‌رسان‌ها، پیش‌فرض
`kt/contacts/ContactGroupsHandler.kt`	برچسب
`kt/contacts/ContactLinkHandler.kt`	ادغام
`kt/contacts/ContactFileHandler.kt`	فایل
`kt/contacts/SimContactsHandler.kt`	سیم‌کارت
`kt/media/PhotoHandler.kt`	عکس

۱۶.۲ خواندن دفترچه

- **خواندن مجوز نمی‌گیرد** (`_hasPermission`). خواندن بدون مجوز فهرست خالی می‌دهد و **کش نمی‌شود**، وگرنه تا اجرای بعد خالی می‌ماند.
- **خواندن بومی با سه نوع داده است:** نام، شماره، اولین ایمیل (`ContactExtrasHandler.readAddressBook`). روی ۳٬۰۰۰ مخاطب از ۱۲ ثانیه به ۰٫۴۵ ثانیه رسید. خواندن افزونه پشتیبان است.
- **کش ایستا شمارندهٔ نسل دارد.** `invalidateCache()` آن را بالا می‌برد و خواندنی که پیش از آن شروع شده نتیجه‌اش را منتشر نمی‌کند.
- `DeviceSyncQueue` خواندن‌های سنگین را به صف می‌کند.

۱۶.۳ تطبیق شماره با مخاطب

اول تطبیق دقیق با `PhoneNormalizer.toThreadId`، سپس **هفت رقم آخر** با `toTailKey`. زنگ‌خوردن با `PhoneLookup` اندروید نام می‌گیرد که هفت رقم آخر را مقایسه می‌کند، و این نام باید با بقیهٔ برنامه یکی باشد.

سه نمایه این کار را می‌کنند: `ContactRepository._tailIndex`، `CallLogService._tailIndex` و `ContactNameCache._tailMemo`. **دنباله‌ای که دو مخاطب مشترک دارند null است**، چون نام اشتباه بدتر از شماره است.

شمارهٔ کمتر از هفت رقم تطبیق تقریبی ندارد.

۱۶.۴ یک سیگنال برای تغییر نام

`ContactRepository.revision` یک `ValueNotifier` است که `invalidateCache()` آن را بالا می‌برد. `MainNavigation` گوش می‌دهد و `RefreshContacts`، `RefreshContactNames` و `RefreshCallLogContactNames` را می‌فرستد. سرصفحهٔ گفتگو، جزئیات و `_FavoriteCard` هم خودشان دوباره می‌خوانند.

- `forceRefresh` آن را بالا نمی‌برد، وگرنه حلقه می‌شود.
- `LazyContactAvatar.generation` هم با همین بالا می‌رود تا عکس عوض شود.
- `call_logs` ستون نام ندارد و در هر صفحه نام روی آن گذاشته می‌شود، پس نام می‌تواند از بین برود.

۱۶.۵ قالب نام و ترتیب

«قالب نام» و «مرتب‌سازی بر اساس» **هنگام ساختن** `ContactModel` اعمال می‌شوند. پس فهرست، جستجو و نمایهٔ سریع با هم یکی‌اند. `ContactNameStyle` آینهٔ ایستای این دو تنظیم است. `apply` اول مقدارها را می‌گذارد و بعد `onChanged` را می‌فرستد.

- `sortName` ملاک ترتیب و بخش‌بندی است.
- **مرتب‌سازی با ماست** (`_sorted`)، چون ترتیب خانوادگی، مخاطبان سیم و پرش سریع به آن نیاز دارند.
- `PersianCollator` ترتیب واقعی الفبای فارسی است. `compareTo` پ، چ، ژ، ک و گ را پراکنده می‌کند. کلید برای هر مخاطب یک بار ساخته می‌شود.

۱۶.۶ برچسب‌ها

- `ContactLabel` یعنی نام به‌علاوهٔ همهٔ شناسه‌های گروهی که آن نام را دارند. هر حساب «Family» خودش را دارد و نام‌های داخلی پنهان‌اند.
- **عضویت بومی نوشته می‌شود** (`ContactGroupsHandler`). `GroupMembership` فقط وقتی معتبر است که حساب گروه و حساب مخاطب یکی باشند. `insertGroup` افزونه حساب نمی‌نویسد و برچسب خالی می‌ماند.
- **رتبهٔ حساب برای برچسب تازه** (`busiestAccount`، `accountRank`): بدون آداپتور همگام‌سازی = حافظهٔ محلی (۲)؛ `supportsUploading` = ابر واقعی (۱)؛ فقط دانلود = کپی (۰). پیام‌رسان‌ها با کپی کل دفترچه بیشترین ردیف را دارند ولی جای درستی نیستند. سیم‌کارت حذف است.
- **هر برچسب در هر حسابی که عضو دارد هست.** `memberIds` بر اساس عنوان تطبیق می‌دهد.

> `AddEditContactScreen._save` **باید پیش از نوشتن** `contact.groups` **را خالی کند.** افزونه در هر به‌روزرسانی ردیف عضویت را اضافه می‌کند و حذف نمی‌کند، پس هر ذخیره تعداد را دو برابر می‌کرد. حدود بار دهم، دسته از ۵۰۰ عملیات رد می‌شد، `applyBatch` خطا می‌داد و چون افزونه روی coroutine بدون try اجرا می‌شود، کل فرایند کرش می‌کرد. این همان گزارش «عکس را عوض می‌کنم و برنامه می‌بندد» بود. **هر دسته برای provider زیر ۵۰۰ عملیات تکه می‌شود** (`applyInChunks`).

`applyLabels` ردیف‌های تکراری را هم پاک می‌کند.

۱۶.۷ فیلترها

ردیف چیپ زیر جستجوی مخاطبین: «همه»، هر برچسب، هر پیام‌رسان و «رمز». زیر صندوق فقط برچسب‌ها می‌آیند. منبع `ContactFilters` و ویجت `ContactFilterBar` است. ⚙ صفحهٔ «نمایش گروه‌ها» را باز می‌کند.

- **پیام‌رسان از چیزی که می‌نویسد شناخته می‌شود** (`getMessengerAccounts`): حسابی که ردیف با MIME سفارشی دارد.
- **«رمز»** فقط وقتی بخش امن باز است نمایش داده می‌شود و کش نمی‌شود، تا قفل بودن چیزی دربارهٔ طرف‌های گفتگو نگوید.

۱۶.۸ برنامه‌های متصل

`ContactExtrasHandler.kt` ردیف‌های شخص ثالث را می‌خواند. دو چیز لازم است: `<queries>` مانیفست برای `AccountAuthenticator` و `VIEW` روی `vnd.android.cursor.item/*`، و اجرای **component مشخص**، چون پیام‌رسان‌ها چند alias دارند و intent ضمنی برگهٔ انتخاب نشان می‌دهد.

۱۶.۹ ادغام

`ContactLinkHandler.kt` با `AggregationExceptions` نوع `TYPE_KEEP_TOGETHER` می‌نویسد. چیزی کپی یا حذف نمی‌شود و هر حساب ردیف خودش را نگه می‌دارد. `unlink` با `TYPE_KEEP_SEPARATE` برمی‌گرداند.

- **ورودی‌ها:** نوار انتخاب، `DuplicateContactsScreen` (union-find روی شماره یا نام یکسان) و «جدا کردن» وقتی `rawContactCount > 1`.
- **`showMergeContactsDialog`** می‌پرسد نام و عکس چه کسی بماند، و `IS_SUPER_PRIMARY` روی ردیف نام و عکس نوشته می‌شود.
- شناسهٔ پیوندشده ممکن است هیچ‌کدام از ورودی‌ها نباشد، پس تازه‌سازی لازم است.

۱۶.۱۰ انتقال مخاطب

همه از `ContactFileService` و `ContactFileHandler.kt` هستند و مجوز تازه نمی‌خواهند. دفترچهٔ مخفی و بانک کلید هرگز به این مسیر نمی‌رسند.

- **«ذخیره مخاطبین در فایل»:** `ACTION_CREATE_DOCUMENT` و `CONTENT_VCARD_URI`. رکورد سیم با `toVCard` نوشته می‌شود.
- **«افزودن مخاطبین از فایل»:** `ContactImportScreen`. تکراری‌ها بدون تیک و با «تکراری» می‌آیند.
- **`VcfFile`** quoted-printable را خودش باز می‌کند، چون افزونه با `%` خطا می‌دهد. فایل ورودی نامطمئن است، پس `maxFields` و `maxPhotoBytes` دارد.
- **«کپی به...»** ردیف تازه می‌نویسد و اصلی را نگه می‌دارد.
- **مقصدها** (`saveTargets`): گوشی، هر سیم‌کارت و هر حسابی که آداپتور آپلود دارد.
- **سیم‌کارت ایرادگیر است:** فاصله و خط تیره رد می‌شوند (`simNumber`) و نام طولانی هم (`simNames`) کوتاه‌تر امتحان می‌کند).
- **«محل ذخیره پیش‌فرض»** در `contact_save_target`.

۱۶.۱۱ عکس مخاطب

برگهٔ سه‌گزینه‌ای و `PhotoCropScreen`. **رابط در Dart و پیکسل در Kotlin است.** صفحهٔ برش چرخش و مستطیل نرمال‌شده می‌فرستد و `PhotoHandler` کار می‌کند.

- **«گرفتن عکس» مجوز دوربین می‌خواهد.** `CAMERA` برای عکس ورود ناموفق در مانیفست هست و سیستم بدون مجوز `ACTION_IMAGE_CAPTURE` را رد می‌کند. `pickContactPhoto` آن را می‌گیرد.
- **شماره‌های درخواست ۹۲xx اهمیت دارند.** شمارهٔ قدیمی ۹۰۰۲ با نقش پیامک تداخل داشت. نقش‌ها ۹۱xx extras، ۹۰xx، عکس ۹۲xx.
- **`resultCode` در مسیر دوربین معتبر نیست.** فایل مقصد ملاک است.
- **EXIF** هنگام decode اعمال می‌شود.
- **«حذف عکس» بومی است** و بعد از `update()` اجرا می‌شود، چون افزونه `photo = null` را نادیده می‌گیرد.

۱۶.۱۲ فرم و صفحهٔ مخاطب

- **فیلد شماره LTR است.**
- **ردیف شماره اندازه‌گیری شده:** `CompactLabelDropdown` با حداکثر ۶۲dp و `RemoveFieldButton` با ۳۲dp. به اندازهٔ پیش‌فرض برنگردانید.
- **«برچسب‌ها» در فرم اصلی است**، نه پشت «فیلدهای بیشتر».
- **شمارهٔ پیش‌فرض همان `IS_SUPER_PRIMARY` اندروید است** (`setDefaultPhone`، `clearDefaultPhone`، در یک دسته).
- **تطبیق شماره با ۹ رقم آخر است** و `PHONE_NUMBERS_EQUAL` استفاده نمی‌شود.
- **پیش‌فرض جواب است:** `pickContactNumber` اگر یک پیش‌فرض باشد برگه باز نمی‌کند. `pickContactNumberFor` برای کسی است که فقط مخاطب را دارد.
- **`save_number_actions.dart`** در چهار جا مشترک است.

۱۶.۱۳ سنجاق و انتخاب گروهی

- حداکثر ۵ مخاطب سنجاق‌شده، با `PinnedContacts` در `pinned_contacts`. ردیف سنجاق تکرار است و `_visible` آن را نمی‌شمارد. `_pinnedExtent` همان `leadingOffset` پرش سریع است.
- **لمس طولانی انتخاب گروهی را شروع می‌کند:** تعداد، ستاره، اشتراک، حذف، انتخاب همه. حذف از `FlutterContacts.deleteContacts` است.

۱۶.۱۴ «موردعلاقه‌ها»

`features/favorites/`. جدول `favorites` با کلید `normalized`.

- **ستاره برای فرد است، نه شماره.** `_FavoriteCard` مخاطب زنده را می‌خواند و نام ذخیره‌شده فقط پشتیبان است.
- **برگهٔ لمس طولانی** با `_chooseNumber` همان سؤال را می‌پرسد.
- شماره‌گیری سریع جای دیگری است (بخش ۱۳).

۱۶.۱۵ مخاطب سیم‌کارت

بخش ۱۵ را ببینید.

بخش ۱۷. جستجوی مخاطب، T9 و عادی‌سازی شماره

۱۷.۱ عادی‌سازی شماره

`PhoneNormalizer` (`core/utils/phone_normalizer.dart`) کلید اتصال همهٔ برنامه است. چون مخاطب از گوشی می‌آید و شناسهٔ ثابت در پایگاه‌داده ندارد، همه چیز به شمارهٔ `09xxxxxxxxx` وصل می‌شود (تصمیم D2).

تابع	کار
`toThreadId`	شناسهٔ کانونی گفتگو. برای نشانی غیرقابل‌تجزیه همان متن را برمی‌گرداند
`toNational`	شکل ملی
`toTailKey`	هفت رقم آخر

هرگز رقم‌ها را دستی جدا نکنید. چند خطای جدی دقیقاً از همین آمده: مسدودی بی‌اثر، موردعلاقهٔ دوتایی و تقسیم یک مخاطب در «اخیر». آینهٔ Kotlin آن `BlockedNumbers.normalizeToThreadId` است. شماره‌های خارجی فقط به رقم پاک تبدیل می‌شوند (K6).

۱۷.۲ `SearchText`

هر فیلتر مخاطب، یعنی تب مخاطبین، پیشنهادهای شماره‌گیر و `searchContacts`، از `SearchText` (`core/utils/search_text.dart`) و `ContactRepository.matchContacts` / `matchPhoneDigits` می‌گذرد. `String.contains` ساده جستجوی واقعی روی این دفترچه نیست.

شماره در همهٔ شکل‌ها

`+98…`، `0098…`، `98…`، `09…` و `9…` برابرند. هم شمارهٔ ذخیره‌شده و هم عبارت تایپ‌شده باز می‌شوند (`phoneForms`، `queryForms`).

> یک ناهمسانی باید بماند: رقم‌ها **همان‌طور که تایپ شده** هر جایی از شماره تطبیق می‌خورند، اما شکلی که با حذف `0` یا `98` ساخته می‌شود فقط **از ابتدا**. وگرنه جستجوی `021…` هر موبایلی را که `21` دارد برمی‌گرداند.

نام تا شده

`ي` به `ی`، `ك` به `ک`، `آ/أ/إ` به `ا`، `ة` به `ه`، `ؤ` به `و`؛ حرکات، ZWNJ و نشانه‌های جهت حذف؛ رقم فارسی و عربی به لاتین؛ و فاصله اختیاری («محمدرضا» «محمد رضا» را پیدا می‌کند).

برای رنگ‌آمیزی از `SearchText.matchRange` استفاده می‌شود که اندیس را در متن **اصلی** می‌دهد. تا کردن حروف را حذف می‌کند، پس اندیس متن تاشده حرف اشتباه را رنگ می‌کند.

سرعت

- عبارت **یک بار برای هر فهرست** به `PhoneQuery` تبدیل می‌شود، نه برای هر مخاطب.
- `phoneForms` برای هر رشته memoize می‌شود.
- جستجوی مخاطبین ۲۰۰ میلی‌ثانیه تأخیر دارد و نتیجه روی (عبارت، هویت منبع) نگه داشته می‌شود.
- `_resultNumbers` شماره را برای هر عبارت حساب می‌کند، نه در `build` هر ردیف.

جزئیات مهم

- `DialerBloc._onFilterContacts` **در هر فیلتر** `getAllContacts()` **را می‌خواند** که معمولاً کش است. فیلتر کردن یک عکس ثابت باعث می‌شد شمارهٔ تازه تا اجرای بعد پیشنهاد نشود.
- **ورودی شماره‌گیر ۳۰۰ میلی‌ثانیه تأخیر دارد.**
- `SearchBloc` هم از همین تطبیق‌دهنده استفاده می‌کند.

۱۷.۳ جستجوی T9

همان رقم‌ها دو بار خوانده می‌شوند: به‌عنوان شماره (`PhoneQuery`) و به‌عنوان حروف روی کلیدها (`SearchText.t9Of`، `t9MatchRange`)، هر دو در `PhoneMatch.matchPhoneDigits` با `nameStart >= 0` یعنی تطبیق T9، و ردیف نام را رنگ می‌کند نه شماره را.

- **جدول حروف فارسی است** (`_t9Groups`، ۲ ابپتث تا ۹ هی) به‌علاوهٔ حروف لاتین. `DialerScreen._keyRows` همان جدول را روی کلیدها می‌نویسد.
- **تطبیق فقط از ابتدای کلمه** (`T9Name.wordStarts`) و حداقل `SearchText.minT9Length` رقم.
- **تطبیق عددی اول می‌آید** و مخاطب تکراری نمی‌شود.
- `T9Name.positions` رقم را به نام اصلی برمی‌گرداند.

۱۷.۴ ردیف مخاطب

`ContactNumbersLine` (`core/widgets/`) شماره‌ها را زیر نام نشان می‌دهد، چون ردیف فقط‌نام دو «علی» را جدا نمی‌کند. اگر تطبیق عددی باشد با `HighlightedPhone` آن را نشان می‌دهد؛ وگرنه شماره‌ها را با «·» و «+N». `_kRowHeight` تب مخاطبین برای این دو خط تنظیم شده و در پرش سریع اثر دارد، پس تغییر ارتفاع ردیف یعنی تغییر این ثابت.

۱۷.۵ فهرست الفبایی

`core/widgets/contact_index_list.dart` بین تب مخاطبین و `ContactSelectorScreen` مشترک است: بخش‌ها، سرصفحه، نوار حروف و محاسبهٔ پرش. پرش فقط وقتی درست است که ترتیب فهرست، رتبهٔ نوار و ارتفاع ردیف یکی باشند.

۱۷.۶ جستجوی یکپارچه

`features/search/` با `SearchBloc`. سه بخش مخاطب، تماس و پیام. بخش پیام‌ها آخر است، چون صفحه‌به‌صفحه و ۴۰تایی است (`LoadMoreSearchMessages`). تماس‌ها یک ردیف برای هر شماره از کل جدول دارند (`getLatestCallPerNumber`).

بخش ۱۸. تنظیمات

۱۸.۱ قانون اصلی

هر کلیدی که در تنظیمات هست را چیزی واقعاً می‌خواند. کلیدهای بی‌اثر مانده حذف شده‌اند، نه اینکه روی صفحه بمانند: TTY و سمعک، شناسهٔ تماس‌گیرنده، پاسخ‌های سریع، صدای صفحه‌کلید و لرزش هنگام تماس.

۱۸.۲ ساختار کد

فایل	کار
`settings/bloc/settings_bloc.dart`	بار و ذخیره، کپی به سمت بومی
`settings/bloc/settings_state.dart`	`SettingsState` و `BoolSetting`
`settings/screens/settings_screen.dart`	صفحهٔ اصلی، ۱٬۱۲۱ خط
`settings/screens/`	صفحه‌های جدا: `SimSettingsPage`، `PrivacyPolicyPage`، `SpamAndBlockedScreen`، `SpeedDialScreen`، `NumberListsPage`، `CallBlockRulesPage`، `AutoRedialPage`

مقادیر در `SharedPreferences` هستند. `SetBoolSetting` یک `BoolSetting` را عوض می‌کند. مقادیر جدا مثل `SetCalendarType` و `SetMessageTextScale` event خودشان را دارند.

۱۸.۳ مقادیری که کپی بومی دارند

بعضی تصمیم‌ها بدون موتور Flutter گرفته می‌شوند. برای این‌ها مقدار دو بار نوشته می‌شود: `SharedPreferences` و فایل بومی. `SettingsBloc` این کار را هم در تغییر و هم در `LoadSettings` انجام می‌دهد.

تنظیم	مقصد بومی	چرا
`blockUnknownCallers`	`CallPrefs`	تصمیم در `onCallAdded`
`blockNonContacts`، `blockInternational`، `blockAdvertising`، `adPrefixes`	`CallPrefs` (`setCallBlockRules`)	همان
`pinToAnswerCalls`	`CallPrefs`	کارت پاسخ بومی است
`hideNotificationContent`	`NotificationPrivacy`	از `NotificationPrivacySync` به شکل PIN و کلید
فهرست سیاه و سفید	`NumberLists`	گیرنده‌ها
سیم‌کارت پیش‌فرض	`sim_prefs`	`routeFor`

`AutoRedialPolicy` و `ContactNameStyle` و `DateFormatter.calendar` هم آینهٔ ایستا هستند، ولی داخل Dart. یک BLoC نمی‌تواند BLoC دیگری را بخواند، پس این کلاس‌ها راه تبادل‌اند.

بعضی مقادیر مستقیم خوانده می‌شوند، چون `LoadSettings` ناهمگام است و ممکن است زودتر لازم شوند: `readShowDialpadOnStart()` و `readPinToAnswerCalls()`.

۱۸.۴ صفحه‌ها

صفحه	محتوا
«عمومی»	پوسته، تقویم، `SimSettingsPage`، `CallBlockRulesPage`، `NumberListsPage`
«پیامک‌ها»	«اندازه متن پیام»، گزارش تحویل، کشیدن ردیف
«کمک‌های تماس»	«تماس مجدد خودکار»
«صدا و لرزش»	لینک به تنظیمات گوشی، صدای کلید و لرزش کلید
«امنیت»	PIN، کد بازیابی، «قفل خودکار»، اثر انگشت، «پنهان کردن محتوای اعلان‌ها»، «رمز برای پاسخ به تماس»، و در نسخه‌های امن «بخش امن»، «بانک کلید»، «کنترل امنیت سامانه» و «عکس از ورود ناموفق»
«گزینه‌های نمایش»	«قالب نام»، «مرتب‌سازی بر اساس»، «محل ذخیره پیش‌فرض»
«برنامه‌های پیش‌فرض»	نقش پیامک و تلفن، «اعلان‌های تماس»، «اعلان تمام‌صفحه تماس»
«شماره‌گیری سریع»	`SpeedDialScreen`
«هرزنامه و مسدودشده»	`SpamAndBlockedScreen`
«بخش امن»	در نسخه‌های دارای بخش امن: PIN بخش، اثر انگشت، بانک کلید، کنترل امنیت
«تشخیص خطا»	گزارش خطا، جز در نسخهٔ سازمانی
«درباره برنامه»	نسخه با `package_info_plus`، نام نسخه، `PrivacyPolicyPage`

۱۸.۵ نکته‌های مهم

- **زنگ و لرزش زنگ مال Telecom است.** برنامه زنگ پخش نمی‌کند، پس «صدا و لرزش» به `ACTION_SOUND_SETTINGS` لینک می‌دهد.
- **«باز کردن صفحه‌کلید هنگام اجرای برنامه»** در `MainNavigation` و بعد از فریم اول اجرا می‌شود. لینک عمیق برنده است و اگر route جاری نباشد کاری نمی‌کند.
- **«پوسته»** با `AppThemeMode.system` شروع می‌شود، پس فریم اول از پوستهٔ گوشی است.
- **«حریم خصوصی و دسترسی‌ها»** برای هر گروه مجوز یک کارت دارد و وضعیت را فقط می‌خواند، بدون درخواست. **هر مجوز تازه در مانیفست، در همان کامیت کارت می‌گیرد.** این همان سیاست فروشگاه است و نسخهٔ URL عمومی از روی این صفحه ساخته می‌شود.
- **`SettingsChoice.enabled`** گزینه را کم‌رنگ می‌کند و پنهان نمی‌کند.
- **`SettingsBloc._retiredKeys`** کلیدهای حذف‌شده را پاک می‌کند.

بخش ۱۹. احراز هویت و قفل برنامه

این بخش PIN **برنامه** را توضیح می‌دهد. PIN بخش امن جداست و در بخش ۲۱ آمده است.

۱۹.۱ فایل‌های اصلی

فایل	کار
`authentication/repositories/auth_repository.dart`	PIN، کد بازیابی، جلسه
`authentication/bloc/auth_bloc.dart`	وضعیت
`authentication/screens/`	`AuthChoiceScreen`، `AuthWrapperScreen`، `PinSetupScreen`، `PinAuthScreen`، `VerifyPinScreen`، `RecoveryCodeScreen`
`authentication/screens/widgets/pin_pad.dart`	`PinDots`، `PinKeypad`، `PinErrorText`، `BiometricUnlockKey`
`authentication/repositories/pin_attempt_limiter.dart`	بودجهٔ تلاش
`authentication/services/biometric_unlock.dart`، `app_lock_platform.dart`	اثر انگشت و پل کانال `app_lock`
`authentication/services/notification_privacy_sync.dart`	هماهنگی «پنهان کردن محتوای اعلان‌ها» با Kotlin
`core/widgets/app_lock_wrapper.dart`	`AppLockWrapper`، `AppLock` و `RelockScreen`
`kt/security/BiometricGate.kt`	`BiometricPrompt`
`kt/security/AppLockSession.kt`، `NotificationPrivacy.kt`	جلسهٔ قفل برای اعلان‌ها

۱۹.۲ ذخیرهٔ PIN

- نوع احراز هویت در `flutter_secure_storage` است و خود PIN **نمک‌دار و کشیده‌شده** به شکل `v2:<salt>:<hash>` با ۶۰٬۰۰۰ دور SHA-256 ذخیره می‌شود. مقدار بدون `v2:` رد می‌شود.
- **PIN برنامه در همهٔ نسخه‌ها ۴ رقم است** (`PinPolicy.appLength`) و PIN بخش امن ۶ رقم. طول در `auth_pin_length` ثبت می‌شود و هر صفحه از آن اندازه می‌گیرد. هرگز ۴ را در کد ننویسید.
- **«باز بودن» در حافظهٔ فرایند است، نه ذخیره** (`isAuthenticated`). فرایند تازه همیشه قفل است. این قبلاً ذخیره می‌شد و قفل هرگز دوباره نمی‌آمد.

۱۹.۳ بودجهٔ تلاش

هر بررسی PIN از `PinAttemptLimiter` داخل `AuthRepository.validatePin` می‌گذرد: ۵ تلاش آزاد، بعد قفل با دو برابر شدن از ۳۰ ثانیه تا ۳۰ دقیقه. هنگام قفل، `validatePin` بدون نگاه به PIN `false` می‌دهد و صفحه با `pinRetryAfter()` دو حالت را جدا می‌کند. تغییر PIN با `VerifyPinScreen` اول PIN فعلی را می‌خواهد.

۱۹.۴ کد بازیابی

PIN فراموش‌شده قفل همیشگی نیست.

- تنظیم PIN یک کد بازیابی یک‌بارمصرف می‌سازد (`regenerateRecoveryCode`). فقط هش آن ذخیره می‌شود و الفبایش O/0 و I/1 ندارد.
- `PinSetupScreen` آن را یک بار با `RecoveryCodeScreen` نشان می‌دهد.
- `AuthBloc` حالت `AuthRecoveryCodeIssued` را **پیش از** `AuthAuthenticated` می‌فرستد. اگر برعکس شود، کد ساخته می‌شود و در همان فریم گم می‌شود.
- «رمز را فراموش کرده‌ام» کد را بررسی می‌کند و به ساختن PIN می‌رود و مستقیم برنامه را باز نمی‌کند، چون کد کاغذی نباید رمز دوم شود.
- «کد بازیابی جدید» در تنظیمات است.

۱۹.۵ قفل دوباره

`AppLockWrapper` جلسهٔ باز را بعد از ماندن در پس‌زمینه به اندازهٔ «قفل خودکار» قفل می‌کند (فوراً، ۱، ۵ یا ۳۰ دقیقه؛ پیش‌فرض ۱ دقیقه؛ `relockAfterSeconds`).

- **قفل یک route است** (`RelockScreen` با `PopScope(canPop: false)`) روی `appNavigatorKey`، نه overlay. باید همهٔ صفحه‌ها را بپوشاند و Back را بگیرد، و `CallUiCoordinator` بتواند تماس را روی آن بیاورد.
- **فقط `paused` و `hidden` خروج حساب می‌شوند،** نه `inactive`.
- PIN را از repository بررسی می‌کند تا `AuthBloc` و کل برنامه زیر قفل بازسازی نشوند.
- **روی تماس پاسخ‌داده‌شده قفل نمی‌کند.** تماس زنگ‌خورده زیر قفل گذاشته می‌شود (بخش ۱۳).

۱۹.۶ لینک عمیق و قفل

`DeepLinkService._deliver` با `AppLock.whenDown` منتظر می‌ماند. قفل هم یک route روی همان navigator است و قبلاً اعلان، گفتگو را روی قفل باز می‌کرد و پیام از پشت PIN خوانده می‌شد. `call_over_app_lock_test.dart` این را ثابت می‌کند.

۱۹.۷ خروج‌های برنامه

`AppHandoff` (`core/services/app_handoff.dart`): هر فراخوانی که صفحهٔ برنامهٔ دیگری را برای نتیجه باز می‌کند، مثل انتخابگر فایل کلید، باید داخل `AppHandoff.run` باشد. وگرنه بخش امن پشت آن بسته می‌شود و قفل برنامه PIN می‌خواهد. این معافیت حداکثر ۲ دقیقه است.

۱۹.۸ اثر انگشت

«باز کردن با اثر انگشت یا چهره» (تنظیمات ← امنیت، همهٔ نسخه‌ها، فقط با PIN، خاموش به‌طور پیش‌فرض).

- کلید اثر انگشت سمت چپ ۰ (`BiometricUnlockKey`). هر بار که قفل ظاهر شود، فقط اگر برنامه resumed و قفل بالاترین route باشد، یک بار خودش پنجره را باز می‌کند، و هرگز روی صفحهٔ تماس نه.
- **`BiometricPrompt` مستقیم روی کانال `app_lock`** است، چون `local_auth` به `FragmentActivity` نیاز دارد.
- API 28+، `BIOMETRIC_WEAK`، و هرگز `DEVICE_CREDENTIAL`. رمز گوشی رمز برنامه نیست.

- **فقط قفل برنامه را باز می‌کند.** بخش امن تنظیم خودش را دارد.
- **اثر انگشت تازه آن را خاموش می‌کند:** یک کلید نگهبان Keystore با `setInvalidatedByBiometricEnrollment(true)` . روی گوشی فقطضعیف این بررسی نیست و ثبت می‌شود.
- `clearAuth` آن را با PIN پاک می‌کند. پنجره داخل `AppHandoff.run` اجرا می‌شود.

۱۹.۹ صفحه‌های PIN

- **نقطه‌ها چپ‌به‌راست پر می‌شوند** (`PinDots`). PIN یک عدد است.
- **`PinKeypad` هم LTR است.**
- **`AuthWrapperScreen` در stateهای موقت بازسازی نمی‌شود** (`AuthLoading` ، `AuthValidationFailure` ، `AuthRecoveryCodeIssued`). قبلاً یک PIN اشتباه صفحهٔ خالی بدون صفحه‌کلید می‌آورد. `ValidatePin` بعد از شکست به `AuthSet` برمی‌گردد.
- **`DisableAuth` اول `AuthLoading` می‌فرستد** تا تنظیمات منتظر `AuthAuthenticated` بعدی بماند.

۱۹.۱۰ رفتن از PIN به اعلان

`AuthRepository.reportSession` هر تغییر جلسه را به `AppLockSession` می‌فرستد، به‌صورت زنجیره تا قفل از باز شدن عقب نماند. جزئیات در بخش ۸ است.

بخش ۲۰. نسخه‌ها و فعال‌سازی

۲۰.۱ ماتریس امکانات

تقسیم امکانات بین نسخه‌ها در ماتریس `docs/پیوست فنی.docx` است. اولویت‌ها **رنگ سلول‌ها**ست: سرخابی ۱، سبز ۲، آبی روشن ۳، زرد ۴ و سیاه یعنی این نسخه ندارد. سلول بنفش ردیف ۳۰ سازمانی هم اولویت ۱ حساب می‌شود.

۲۰.۲ یک شناسه، یک کلید امضا

هر دو فلیور با `ir.hamrasan.app` نصب می‌شوند و کلید امضایشان یکی است. این الزام است: فروشگاه «نصب است» را از روی نام بسته تصمیم می‌گیرد. مدیر بسته‌ها هم هر دو را یک برنامه می‌بیند، پس نصب APK سازمانی روی نسخهٔ تجاری **داده را نگه می‌دارد**.

۲۰.۳ بازهٔ versionCode

چیزی که جلوی به‌روزرسانی نسخهٔ سازمانی به تجاری را می‌گیرد، versionCode است:

فلیور	versionCode
`commercial`	`git rev-list --count HEAD` = `N`
`organization`	`2_000_000_000 + N`

- فروشگاه فقط versionCode بزرگ‌تر را پیشنهاد می‌کند و مدیر بسته‌ها کوچک‌تر را با `INSTALL_FAILED_VERSION_DOWNGRADE` رد می‌کند.
- سقف اندروید `2_100_000_000` است. `build.gradle.kts` اگر `N` از `1..100_000_000` بیرون برود ساخت را متوقف می‌کند.
- **بازه در Gradle و برای همهٔ نوع‌های ساخت اعمال می‌شود،** حتی debug. آن را به اسکریپت‌ها نبرید.
- `versionNameSuffix = "-org"` نسخه را در App info نشان می‌دهد.

۲۰.۴ `AppEdition` در Dart

`core/edition/app_edition.dart`.

- `AppEdition.current` و `kOrganizationBuild` **ثابت زمان کامپایل** از `appFlavor` هستند. پس کامپایلر AOT کد نسخهٔ دیگر را حذف می‌کند. برای بازبین امنیتی سازمان، «پشت if پنهان» با «وجود ندارد» فرق دارد.
- `AppEdition.verifyBuildFlavor()` اول `main` اجرا می‌شود و فلیور ناشناس را رد می‌کند.
- اجرای بدون فلیور (`flutter test`) تجاری است.

تفاوت دو APK: گزارش خطا در سازمانی حذف شده است (`CrashReporting.isAvailable`)، «نوع نسخه» در «درباره برنامه» فرق دارد، سازمانی تا فعال‌سازی باز نمی‌شود، و «متن پوششی» فقط در سازمانی است. مانیفست یکسان است. اگر روزی مجوز فقط برای یک نسخه لازم شد، مانیفست آن را در `android/app/src/<flavor>/AndroidManifest.xml` بگذارید.

۲۰.۵ کد فعال‌سازی

نسخهٔ تجاری با یک کد آفلاین بین‌سازمانی می‌شود. **قالب، همان قالب تولیدکنندهٔ ویندوزی موجود است و نباید تغییر کند.**

مرحله	مقدار
کد دستگاه	BLAKE3 بایت‌های `ANDROID_ID` ، ۳ بایت اول، hex کوچک: ۶ نویسه
درهم‌سازی	`d0 1 d1 1 d2 1 d3 1 d4 1 d5 1` برای شناسهٔ برنامهٔ `111111` : ۱۲ نویسه
کد فعال‌سازی	BLAKE3 متن درهم، ۵ بایت اول، hex کوچک: ۱۰ نویسه

فایل	کار
`core/utils/blake3.dart`	BLAKE3 خالص Dart. `blake3_test.dart` با ۱۹ بردار ثابت شده
`core/edition/activation_code.dart`	مشتق‌گیری، `normalize` تحمل‌پذیر و مقایسهٔ زمان ثابت
`kt/edition/DeviceIdentityHandler.kt`	`ANDROID_ID` خام را می‌دهد
`edition/repositories/activation_repository.dart`	خود کد را ذخیره می‌کند و در هر خواندن با دستگاه فعلی بررسی می‌کند
`edition/bloc/edition_bloc.dart`	نسخهٔ مؤثر
`edition/screens/`	`InterOrgActivationScreen` ، و در `organization_activation_gate.dart` هم `OrganizationActivationGate` و `OrganizationActivationScreen`
`edition/widgets/activation_code_form.dart`	فرم مشترک `ActivationCodeForm` و `DeviceCodeCard`
`edition/services/device_identity_service.dart`	پل Dart به `device_identity`

- کد ذخیره می‌شود و هر بار دوباره بررسی می‌شود. پس مقدار جعلی، کپی به گوشی دیگر و بازنشانی کارخانه همگی «فعال نشده» می‌خوانند.
- `AppEdition.current` **هرگز** `interOrganization` **نیست،** چون این یک حالت زمان اجراست. شاخهٔ کد از `state.hasSecureFeatures` است.
- `ActivationFailure` کد اشتباه را از کد درست ولی ذخیره‌نشدنی جدا می‌کند.

فعال‌سازی نسخهٔ سازمانی

- **همان مشتق‌گیری با شناسهٔ دیگر:** `ActivationTarget.organization` برابر `222222` است که فقط پنل ما صادر می‌کند. کد هر نسخه را دیگری رد می‌کند.
- **کلید ذخیرهٔ جدا** (`org_activation_code`).
- `OrganizationActivationGate` پایین `CallUiCoordinator` و بالای `AuthWrapperScreen` است. پس نصب فعال‌نشده هرگز پیام‌رسان یا تلفن پیش‌فرض نمی‌شود.
- `EditionState.activated` واقعیت بررسی‌شده است و `hasSecureFeatures` در سازمانی همان `activated` است.
- `DeactivateInterOrganization` در سازمانی نادیده گرفته می‌شود.

محدودیت پذیرفته‌شده

مشتق‌گیری و شناسه‌ها در APK هستند، پس کسی که برنامه را مهندسی معکوس کند می‌تواند کد بسازد (K4). سرور و لغو وجود ندارد. سازگاری با تولیدکنندهٔ موجود مهم‌تر بوده است. بازنشانی کارخانه هم `ANDROID_ID` را عوض می‌کند و کد تازه لازم است.

۲۰.۶ کدام امکان در کدام نسخه

امکان	تجاری	بین‌سازمانی	سازمانی
پیامک، تماس، مخاطب و بقیهٔ هسته	✓	✓	✓
PIN برنامه، اثر انگشت، فهرست سیاه و سفید	✓	✓	✓
تصویر کوچک، یادآور، سیم‌کارت نام‌دار، فایل `.vcf`	✓	✓	✓
بخش امن، پیام و گروه رمز، بانک کلید، دفترچهٔ مخفی		✓	✓
پیام زمان‌دار، کنترل امنیت، عکس ورود ناموفق		✓	✓
متن پوششی			✓
گزارش خطا	اختیاری	اختیاری	کامپایل نمی‌شود
نیاز به کد پیش از شروع			✓

بخش ۲۱. بخش امن

بخش امن جایی است که همهٔ دادهٔ محرمانه در آن قرار دارد: پیام‌های رمز، کلیدها، دفترچهٔ مخفی و عکس‌های ورود ناموفق. سه تصمیم طراحی شکل آن را تعیین کرده‌اند:

- **PIN خودش را دارد**، جدا از PIN برنامه، یا اثر انگشت.
- **PIN فراموش‌شده یعنی از بین رفتن محتوا.** هیچ راه بازیابی وجود ندارد.
- **لحظه‌ای که برنامه از جلو برود قفل می‌شود.**

۲۱.۱ فایل‌های اصلی

فایل	کار
`secure/repositories/secure_store.dart`	تنها بازکنندهٔ `secure.db` و تنها طرح آن
`secure/services/secure_vault_service.dart`	پل به Keystore
`secure/services/sms_crypto_service.dart`	پل به رمزنگاری
`secure/bloc/secure_session_bloc.dart`	وضعیت قفل، ۷۰۴ خط
`secure/screens/secure_unlock_screen.dart`، `secure_pin_entry_screen.dart`	ساخت، باز کردن و گرفتن PIN برای یک مرحله
`secure/widgets/secure_session_guard.dart`	قفل با خروج از برنامه
`settings/screens/secure_section_settings_page.dart`	تغییر PIN، اثر انگشت، حذف
`secure/widgets/secure_lock_button.dart`	نماد قفل و `openSecureSection`
`secure/widgets/secure_locked_view.dart`	صفحهٔ «بخش امن بسته است»
`kt/secure/VaultCrypto.kt`، `SecureVaultHandler.kt`	مهر کردن کلید
`kt/secure/KeystoreKeyWrapper.kt`	کلید Keystore
`kt/secure/SecureBiometric.kt`	کلید اثر انگشت

۲۱.۲ کلیدها

```
PIN (۶ رقم) — PBKDF2-HMAC-SHA256، ۲۱۰٬۰۰۰ کلید داخلی ──▶─ دور، نمک
Android Keystore (غیرقابل صدور، StrongBox اگر باشد) ──▶ کلید بیرونی
                                ▼
AES-256-GCM کلید داده (۳۲ بایت تصادفی) — مهر دوباره با ──▶ secure_vault.txt
                                ▼
                  SQLCipher با کلید خام x'…' ──▶ secure.db
```

- **دو لایه:** لایهٔ داخلی با PIN و لایهٔ بیرونی با کلید Keystore. tag در GCM خودش بررسی PIN است. فایل بیرون از گوشی بی‌فایده است، پس PIN شش‌رقمی روی کپی فایل قابل حدس نیست.
- **فایل:** `noBackupFilesDir/secure_vault.txt` با قالب `hamresan-vault:1:<rounds>:<salt>:<blob>`.
- **کلید فقط در حافظه است** و هر بار باز شدن یک بار از کانال می‌گذرد. SQLCipher از Dart باز می‌شود.
- **`sqflite_sqlcipher` فقط در `lib/features/secure/` import می‌شود** و factory سراسری sqflite را عوض نمی‌کند.

- `VaultCryptoTest` قالب و هر حالت شکست را ثابت می‌کند.

۲۱.۳ وضعیت‌ها

`SecureSessionBloc` این وضعیت‌ها را دارد:

وضعیت	معنی
`unavailable`	نسخهٔ تجاری فعال‌نشده
`notCreated`	هنوز ساخته نشده
`locked`	ساخته شده و بسته
`unlocked`	باز
`broken`	فایل هست ولی باز نمی‌شود. فقط بازنشانی ممکن است

این BLoC باید **بعد از** `EditionBloc` باشد. با هر تغییر نسخه و هر بازگشت به برنامه وضعیت را دوباره می‌خواند.

۲۱.۴ PIN بخش امن

- **شش رقم و هرگز برابر PIN برنامه.** `AuthRepository.isAppPin` آن را هنگام ساخت و تغییر رد می‌کند. طرف دیگر بررسی نمی‌شود، چون امتحان PIN تازهٔ برنامه روی vault یک راه حدس بی‌محدودیت می‌شد.
- **بودجهٔ خودش را دارد:** `PinAttemptLimiter.secureSection` پیش از PBKDF2 بررسی می‌شود. تلاش‌های اشتباه برای عکس ورود ناموفق هم شمرده می‌شوند (`IntruderReporter`).
- **هیچ کار PIN برنامه به آن نمی‌رسد.** `AuthBloc` اصلاً `SecureStore` ندارد.
- **«تغییر رمز بخش امن»:** PIN فعلی (`SecureVerifyPinRequested`)، بعد تازه دو بار، بعد `rekey`. کلید داده و پایگاه‌داده دست نمی‌خورند و نسخهٔ اثر انگشت معتبر می‌ماند.
- **«رمز بخش امن را فراموش کرده‌ام»:** هشدار، `SecureResetRequested`، و حالت ساخت. PIN برنامه دست نمی‌خورد. «حذف همیشگی بخش امن» هم همین است.

۲۱.۵ FLAG_SECURE

پیش از باز شدن بخش تنظیم می‌شود و با قفل دستی برداشته می‌شود. بعد از قفل **پس‌زمینه** تا resume بعدی می‌ماند، چون عکس recents هنگام خروج گرفته می‌شود و برداشتن زودتر بخش باز را ثبت می‌کند.

`SecureSessionGuard` (در builder برنامه) `paused` و `hidden` را به قفل تبدیل می‌کند و `resumed` پنجره را آزاد می‌کند. `inactive` نادیده گرفته می‌شود (نوار اعلان، پنجره‌ها، تماس).

۲۱.۶ اثر انگشت بخش امن

یک نسخهٔ دوم و مستقل از کلید داده، مهرشده با کلیدی که **فقط بیومتریک قوی (کلاس ۳) و برای هر بار استفاده** می‌تواند باز کند (`setUserAuthenticationParameters(0, AUTH_BIOMETRIC_STRONG)`)، و اندروید آن را با ثبت اثر انگشت تازه **از بین می‌برد** (`setInvalidatedByBiometricEnrollment`).

- فایل `secure_vault_bio.txt` با قالب `hamresan-vault-bio:1:<iv>:<ciphertext+tag>`، و `BioSealTest` آن را ثابت می‌کند.

- **روشن کردن:** PIN، `bioEnable`، کلید تازه، و `BiometricGate.authenticateWithCipher`. هر شکستی خاموش نگه می‌دارد.
- **باز کردن:** `bioUnlock`، پنجره، cipher، کلید داده. موفقیت بودجهٔ تلاش را هم پاک می‌کند.
- **اثر انگشت تازه:** `BIO_INVALIDATED`. پس کسی که رمز گوشی را می‌داند نمی‌تواند انگشتش را اضافه کند و وارد شود.
- **چهرهٔ ضعیف قبول نمی‌شود.** چهرهٔ سامسونگ کلاس ۲ است و نمی‌تواند کلید Keystore را آزاد کند. `strongStatus` این را می‌گوید.
- `create`، `destroy` و خطای تجزیه نسخه را پاک می‌کنند. `rekey` آن را نگه می‌دارد.

۲۱.۷ راه ورود

- **`SecureLockButton`** نماد قفل در سرصفحهٔ همهٔ تب‌هاست و در نسخهٔ تجاری نیست. هرگز نمی‌گوید بخش چیزی دارد یا نه.
- **همهٔ بازکردن‌ها از `openSecureSection`** می‌گذرند. `secure_locked_view_test.dart` این را ثابت می‌کند.
- **«پیام رمز جدید»** (`MainNavigation._openSecureAfterAppLock`): بعد از قفل برنامه، صندوق رمز و `openSecureSection`. هرگز روی قفل نمی‌آید.
- **اجرای عادی بخش را بسته نگه می‌دارد.**

۲۱.۸ خالی شدن با قفل

`KeyBankBloc`، `SecureMessagesBloc` و `HiddenBloc` با قفل شدن بخش خودشان را خالی می‌کنند. هیچ چیز محرمانه‌ای نباید بعد از قفل در حافظهٔ BLoC بماند. فیلتر «رمز» هم هر بار از BLoC خوانده می‌شود و کش نمی‌شود.

۲۱.۹ اندازهٔ APK

SQLCipher کتابخانهٔ بومی دارد و APK حدود ۵ مگابایت بزرگ‌تر می‌شود.

۲۱.۱۰ وابستگی ساخت

`sqflite_sqlcipher` به `androidx.sqlite:sqlite:2.5.2` نیاز دارد که در `android/third_party/maven/` قرار دارد (بخش ۲).

بخش ۲۲. رمزنگاری پیامک

این بخش لایهٔ رمزنگاری را از پایه توضیح می‌دهد. بخش بعدی دربارهٔ کلید و گواهی است و بخش ۲۴ موتوری است که از این لایه استفاده می‌کند.

۲۲.۱ هدف‌ها

- **نامتقارن و مقاوم در برابر کوانتوم،** با الگوریتم‌های استاندارد.
- **ارزان:** اولین تماس باید چند بخش پیامک هزینه داشته باشد، نه ده‌ها بخش. پس حفاظت پساکوانتومی در پس‌زمینه اضافه می‌شود.
- **بدون سرور و بدون فهرست اعضا:** گواهی داخل دست‌دهی جابه‌جا می‌شود.

۲۲.۲ جای کد

```
kt/smscrypto/
  Identity.kt          کلید بلندمدت عضو
  Kdf.kt               SHA-256، HMAC و HKDF
  SmsCryptoException.kt همهٔ کدهای خطا
  CertHandshake.kt     دست‌دهی ۲+۲ بخشی
  Upgrade.kt           ارتقای ML-KEM
  Session.kt           ratchet و رمز پیام
  Payload.kt           قالب محتوا و جدول یک‌بایتی فارسی (PersianCodePage)
  Wire.kt              قالب روی سیم
  CoverText.kt, CoverWords.kt  متن پوششی (بخش ۲۶)
  SealedBox.kt         مهر کردن دفترچهٔ مخفی (بخش ۲۵)
  SmsCryptoHandler.kt  کانال
  SecureSmsInbox.kt    صف پیام رسیده
  KeyFilePicker.kt     انتخاب فایل
  keybank/             کلید و گواهی
```

همهٔ این‌ها به جز `SmsCryptoHandler`، `SecureSmsInbox` و `KeyFilePicker` خالص JVM هستند و ابزار ویندوزی همان سورس را کامپایل می‌کند. **هرگز کپی نکنید.** پل Dart در `secure/services/sms_crypto_service.dart` است.

۲۲.۳ کانال بدون وضعیت

کانال `sms_crypto` هیچ حالتی نگه نمی‌دارد. هر راز، دست‌دهی در حال انجام و نشست **به شکل بایت وارد می‌شود و حالت بعدی برمی‌گردد.** فراخواننده آن را **پیش از فرستادن** در `secure.db` می‌نویسد. پس Kotlin هیچ کلیدی را نگه نمی‌دارد.

متد	کار
`inspect`	مسیریابی بدون کلید: نوع، sid، شمارنده، و محتوای گواهی بررسی‌نشده
`ownInitWins`	حل INIT همزمان
`certInitiate`، `certRespond`، `certComplete`	دست‌دهی
`certInfo`	خواندن گواهی
`upgradeOffer`، `upgradeAccept`، `upgradeComplete`	ارتقا
`encryptText`، `encryptControl`، `encryptGroupInfo`	رمز کردن

متد	کار
`decrypt`	باز کردن
`coverEncode` ، `canonicalPhone`	کمکی
`openKeyFile`	باز کردن فایل کلید
`generateIdentity` ، `publicIdentity` ، `openSealed`	کلید مهر کردن دفترچهٔ مخفی

خطاها نام‌های `SmsCryptoException.Code` هستند.

۲۲.۴ دست‌دهی (`CertHandshake.kt`)

```
A → B  INIT      sid ‖ cert(A) ‖ eA              ۱۳۸ بخش ۲ بایت،
B → A  RESPONSE  sid ‖ cert(B) ‖ eB ‖ confirm    ۱۵۴ بخش ۲ بایت،
```

نماد	معنی
`eA` ، `eB`	کلید X25519 موقت
`X_A` ، `X_B`	کلید X25519 بلندمدت داخل گواهی
`confirm`	اثبات پاسخ‌دهنده

کلیدها:

```
HKDF( X25519(eA,eB) ‖ X25519(eA,X_B) ‖ X25519(X_A,eB), salt = هش متن دست‌دهی )
  → دو زنجیره، کلید تأیید و ریشه
```

این شکل Noise-XX است، با گواهی‌های آشکار (شماره‌های داخلشان نشانی پیامک هم هستند).

- **احراز دوطرفه** از جمله‌های ایستا و **محرمانگی پیشرو** از جملهٔ موقت می‌آید.
- **درخواست گیرنده را نام نمی‌برد.** هر کس گواهی آن شماره را دارد جواب می‌دهد.
- **گواهی پیش از هر ذخیره یا جواب** با مراجع مورد اعتماد بررسی می‌شود.
- **پاسخ‌دهنده می‌تواند فوراً بفرستد.** آغازگر منتظر RESPONSE می‌ماند.
- **INIT همزمان:** `ownInitWins` روی هر دو گوشی یکسان تصمیم می‌گیرد: شناسهٔ کلید کوچک‌تر جلو می‌رود.
- **تا پیش از ارتقا پساکوانتومی نیست.** این تصمیم D13 است، چون دست‌دهی کامل پساکوانتومی حدود ده بخش پیامک می‌خواهد.

۲۲.۵ ارتقا (`Upgrade.kt`)

```
آغازگر → PQ_OFFER (0x13)  کلید ML-KEM بخش ۱۱ بلندمدت،
پذیرنده → PQ_ACCEPT (0x14)  ciphertext، ۱۰ بخش
آغازگر → PQ_DONE (0x15)  ۱ بخش
```

- هر سه روی نشست موجود رمز می‌شوند و بسته‌های کنترلی‌اند، پس اعلان ندارند.
- هر دو طرف نشست تازه (sid تازه) را از `HKDF(ریشه ‖ ss)` می‌سازند. **این راز امن می‌ماند اگر هرکدام از دست‌دهی کلاسیک یا ML-KEM امن بماند.**
- نشست قدیمی برای پیام‌های در راه می‌ماند.

- **پذیرنده نشست تازه را آماده نگه می‌دارد** (`pq-standby`) و تا شنیدن طرف روی آن چیزی نمی‌فرستد. وگرنه پاسخ گم‌شده باعث می‌شد روی نشستی بنویسد که طرف ندارد.
- بدون پاسخ بعد از `upgradeRetry` (۳۰ دقیقه) دوباره پیشنهاد می‌شود.

۲۲.۶ رمز هر پیام (`Session.kt`)

یک ratchet هشی برای هر جهت:

```
mk  = HMAC(ck, 1)
ck' = HMAC(ck, 2)
mk → HKDF → کلید AES-256 و nonce برای GCM
```

- **nonce روی سیم نیست،** چون هر کلید فقط یک پیام را مهر می‌کند.
- **بی‌نظمی:** تا ۱۰۰۰ جلوتر. حداکثر ۱۰۰۰ کلید جامانده نگه داشته می‌شود.
- **شمارندهٔ تکراری** `DUPLICATE` است، چون پیامک گاهی دو بار می‌رسد.
- **دستکاری** `AUTH_FAILED` است و وضعیت تغییر نمی‌کند، چون `Session` تغییرناپذیر است.
- وضعیت ذخیره‌شده ریشه را هم برای ارتقا دارد.

۲۲.۷ محتوا (`Payload.kt`)

یک بایت نوع و سپس متن:

نوع	معنی
`1`	UTF-8
`2`	صفحهٔ کد فارسی هم‌رسان: ASCII و ۹۷ نویسهٔ فارسی، هر کدام یک بایت. `0xFF` هر نویسهٔ دیگر را UTF-8 می‌کند
`0x10`	SEEN(sid)، شمارنده تا)
`0x11`	DELETE(sid)، شمارنده)
`0x12`	GROUP_INFO
`0x13` ، `0x14` ، `0x15`	مراحل ارتقا

پرچم	معنی
`0x80`	حذف پس از دیدن
`0x40`	پیام گروه
`0x20`	زمان‌دار

جدول `PersianCodePage` **منجمد** است و `PayloadTest` هش آن را ثابت کرده. تغییرش یعنی نوع تازه.

۲۲.۸ قالب روی سیم (`Wire.kt`)

`#E:` و سپس Base64 بدون padding. همهٔ این نویسه‌ها GSM-7 پایه هستند، پس پیامک ۱۶۰ نویسه و هر بخش ۱۵۳ نویسه جا دارد. کروشه GSM توسعه‌یافته است و دو برابر هزینه دارد.

بایت ۰ برابر `(Wire.VERSION << 4) | type` است:

type	معنی
۱	پیام
۲	INIT
۳	RESPONSE
۴	CONTROL (رسید یا مرحلهٔ ارتقا)

`sid` دو بایت و شمارنده LEB128 بدون علامت است.

بسته	اندازه	بخش پیامک
INIT / RESPONSE	۱۳۸ / ۱۵۴ بایت	۲ / ۲
PQ_OFFER / PQ_ACCEPT / PQ_DONE		۱۰ / ۱۱ / ۱
پیام	۲۱ بایت + متن	**۱ بخش تا ۹۶ نویسهٔ فارسی**

پیامک فارسی معمولی ۷۰ نویسه جا دارد، پس پیام رمز فارسی تقریباً همان هزینه را دارد.

۲۲.۹ آزمون‌ها

آزمون	چه چیزی را ثابت می‌کند
`KnownAnswerTest`	بردارهای NIST ACVP برای X25519، ML-KEM-768 طبق RFC 7748 و HKDF طبق RFC 5869
`CertHandshakeTest`	عضوهای جدا صادرشده، درخواست از بیرون سازمان، گواهی دیگری، کلید خودمان، دستکاری، ارتقا، کلید جایگزین‌شده، INIT همزمان
`SessionTest`، `WireTest`، `PayloadTest`	ratchet، قالب، محتوا
`GoldenVectorTest`	کل پروتکل بایت به بایت: گواهی، INIT، RESPONSE، پیام، PQ_ACCEPT، PQ_OFFER، پیام

> شکست `GoldenVectorTest` یعنی قالب عوض شده. راه‌حل `Wire.VERSION` تازه است، **نه عدد موردانتظار تازه**. چون گوشی‌هایی با نسخهٔ قدیم نمی‌توانند پیام را باز کنند.

بخش ۲۳. بانک کلید و گواهی عضو

۲۳.۱ مدل کلی

یک **مرجع** (سازمان) برای هر عضو کلید می‌سازد و یک فایل کلید به او می‌دهد. گوشی کلید را فقط از این فایل می‌گیرد. برنامه فقط **کلید عمومی مرجع** را در خود دارد.

تصمیم	معنی
مرجع کلید را می‌سازد	امانت‌داری کلید پذیرفته شده. ابزار کلید خصوصی نگه نمی‌دارد و دوباره مشتق می‌کند
هیچ فهرست عضوی روی گوشی نیست	گوشی نام و شمارهٔ اعضا را ندارد. گواهی داخل دست‌دهی می‌آید
ورود عضو تازه به بقیه ربطی ندارد	فقط فایل همان عضو عوض می‌شود

۲۳.۲ کلید عضو (`Identity.kt`)

ML-KEM-768 و X25519، ذخیره‌شده به شکل بذر: ۹۷ بایت (قالب، `d‖z` برای ML-KEM و X25519).

```
fromSeed( HKDF(memberSeed, phone ‖ generation) )
```

`phone` اولین شمارهٔ عضو است. پس صدور دوباره همان کلید را می‌دهد و بالا بردن `generation` کلید را عوض می‌کند. کلید ML-KEM هنگام تجزیه بررسی FIPS 203 می‌شود (`BAD_KEY`).

۲۳.۳ گواهی عضو (`keybank/MemberCert.kt`)

۱۰۳ بایت:

```
شماره ‖ قالب (۰ به ۰ از بعد رقم ۱۰ BCD (فشرده ‖ generation ‖ X25519 ‖ Ed25519(64)
```

- **فقط با Ed25519 مرجع امضا می‌شود.** امضای ML-DSA حدود ۳۳۰۹ بایت است، یعنی سی بخش برای اولین تماس.
- **شناسهٔ مرجع امضا می‌شود ولی فرستاده نمی‌شود.** بررسی‌کننده همهٔ مراجع را امتحان می‌کند.
- **فقط شماره‌های `09…`** گواهی می‌گیرند. `generation` بین ۰ و ۲۵۵ است.

شناسهٔ کلید تنها نام عضو در همه‌جاست:

```
SHA-256("hamresan.sms.cert-key-id.v1" ‖ X25519)[0..8]
```

`MemberCert.keyIdOf` و `certKeyIdFor` در Dart هر دو با بردار `6d0190893fd7bb4f` (۳۲ بار `0x07`) ثابت شده‌اند.

۲۳.۴ کلید مرجع

ML-DSA-65 و Ed25519، کلید عمومی ۱۹۸۵ بایت.

- `trust_anchors.dart` این کلید را دارد. مرجع واقعی `7fda025a5a997b93` است و مرجع قبلی دیگر معتبر نیست.
- **نیمهٔ ML-DSA الان چیزی امضا نمی‌کند،** ولی در کلید و شناسه هست تا امضای پساکوانتومی آینده مرجع تازه نخواهد.
- `AuthorityFile` (`.hka`) قالب ۲ با سه بذر ۳۲ بایتی: Ed25519، ML-DSA و بذر اعضا.

۲۳.۵ فایل کلید (.hkb)

KeyFile.kt . فقط کلید و گواهی همین عضو، ۲۶۱ بایت.

```
[1] ‖ bytes16 identity(97) ‖ bytes16 cert(103)  = ۲۰۵ بایت ← ۲۶۱ بایت مهرشده
```

- KeyFile.create می‌خواهد cert.x25519 == identity باشد.
- open گواهی را با هر مرجع امتحان می‌کند (UNTRUSTED) و رازی که گواهی نام نبرده را رد می‌کند (BAD_BUNDLE).

فایل مهرشده (SealedFile): magic ‖ format ‖ Argon2 t,m,p ‖ salt16 ‖ nonce12 به‌عنوان AAD و بعد AES-GCM. هزینهٔ PasswordKdf.KEY_FILE برابر ۶۴ مگابایت و ۳ گذر است و پیش از مشتق‌گیری سقف می‌خورد (حداکثر ۲۵۶ مگابایت و ۱۰ گذر). نشانهٔ HMRKB فایل کلید و HMRKA فایل مرجع است. گذرواژه ۲۰ نویسه است.

۲۳.۶ Canon

Canon.kt منجمد است و KeyBankTest آن را ثابت می‌کند:

- شماره در هر شکل (حتی رقم فارسی) به ۰۹… .
- گذرواژهٔ فایل به A تا Z و 0 تا 9 بزرگ.

مقادیر طلایی KeyBankTest : شناسهٔ مرجع e09334807b8501e0 ، شناسهٔ کلید 72cadfdd2e2336fe . تغییر در این‌ها **کلید همهٔ اعضا را عوض می‌کند.**

۲۳.۷ در برنامه

قطعه	کار
kb_own_keys	برای هر مرجع یک کلید. فقط KeyBankRepository به آن دست می‌زند
secure/repositories/key_bank_repository.dart : KeyBankRepository.importKeyFile	نتیجه: added ، replaced (نسل تازه‌تر)، already Imported ، older (رد)
KeyBankBloc	با باز شدن بار می‌کند و با قفل خالی می‌شود. فقط دادهٔ عمومی
KeyBankScreen	تنظیمات ← امنیت ← «بانک کلید»
KeyFilePicker.kt	ACTION_OPEN_DOCUMENT ، بدون مجوز، حداکثر ۸ مگابایت، درخواست ۹۳۰۱

انتخابگر فایل داخل AppHandoff.run است، وگرنه بخش امن پشت آن بسته می‌شود.

پیام‌های نتیجه: imported، replaced، alreadyImported، older، wrongPassword، notAKeyFile، untrusted، badBundle، failed.

۲۳.۸ مرجع آزمایشی و نگهبان انتشار

tools/keybank/dev/ که در git نیست، یک مرجع آزمایشی دارد. برنامه آن را فقط با --dart-define=HAMRESAN_DEV_ANCHOR=true قبول می‌کند و صفحهٔ بانک کلید بنر قرمز «نسخه آزمایشی» نشان می‌دهد.

scripts/release_build.* هر APKی را که libapp.so آن کلید آزمایشی را دارد رد می‌کند.

> فایل مرجع، گذرواژه‌ها و فایل‌های کلید هرگز در git یا سندها نمی‌آیند. برای کار با پنل مرجع واقعی، فقط مسئول سازمان آن را در اختیار دارد.

۲۳.۹ آزمون‌ها

- `KeyBankTest` : canon، امضا، فایل کلید با گذرواژهٔ اشتباه یا مرجع دیگر، گواهی جعلی، راز نام‌نبرده، گواهی، فهرست اعضا و فایل کلید طلایی.
- `KeyBankKnownAnswerTest` : بردارهای NIST ACVP برای RFC 8032، ML-DSA-65 برای Ed25519 و RFC 9106 برای Argon2id.
- bloc، Dart: repository، سرویس، صفحه، `AppHandoff` ، ترتیب provider.

بخش ۲۴. موتور پیام رمز و گروه رمز

بخش ۲۲ دربارهٔ رمز کردن بایت‌ها بود. این بخش دربارهٔ کدی است که از آن لایه استفاده می‌کند: نگهداری نشست‌ها، صف‌ها، تکرار، رسید و گروه.

۲۴.۱ فایل‌های اصلی

فایل	کار
`secure_sms/services/secure_messenger.dart`	موتور اصلی
`secure/repositories/`	`secure_message_store.dart` و `secure_group_store.dart`: جدول‌های `sm_` و `sg_` در `secure.db`
`secure_sms/repositories/secure_queue_repository.dart`	خواندن و پاک کردن `secure_queue`
`secure_sms/services/`	`secure_identities.dart`، `secure_sms_sender.dart` و `hidden_sms_source.dart`
`secure_sms/bloc/secure_messages_bloc.dart`	حالت صفحه‌ها
`secure_sms/screens/`	`SecureInboxScreen`، `SecureConversationScreen`، `SecureGroupScreen`، `SecureGroupEditScreen`، `SecureNewConversationScreen`، `SecureInboxEntry` (`SecureComposer` و `SecureBanner` داخل `secure_conversation_screen.dart` هستند)
`kt/smscrypto/SecureSmsInbox.kt`	پارک کردن پیام رسیده

۲۴.۲ مسیر پیام رسیده

```
پیامک #E: می‌رسد (برنامه شاید مرده باشد)
   ▼
SmsDeliverReceiver: وارد content://sms نمی‌شود
   ▼
SecureSmsInbox.enqueue: در secure_queue (پایگاه‌دادهٔ اصلی) پارک می‌شود
   ▼
arrival(body): اعلان را تصمیم می‌گیرد
   ▼ ... بعد از باز شدن بخش امن ...
SecureMessenger.drain: صف را به sm_inbox در secure.db می‌برد و باز می‌کند
```

Kotlin کلید ندارد، پس فقط متن رمز را پارک می‌کند. متن رمز چیزی را لو نمی‌دهد و نوشتنش در پایگاه‌دادهٔ اصلی مجاز است.

نوع بسته	اعلان
پیام یا INIT	«پیام رمز جدید»
RESPONSE تنها	«پیام رمز شما آماده ارسال است...»، مگر بخش باز باشد (`SecureVaultHandler.sectionOpen`)

نوع بسته	اعلان
کنترلی	هیچ

شناسهٔ اعلان `0x5EC0`، کانال `secure_sms`، و launch action برابر `{type: secure}` است. آینه بسته‌ها را با `looksEncrypted` رد می‌کند. فرستادن با `sendSms(private: true)` است که در provider نمی‌نویسد و شناسهٔ دستگاه آن `-1` است.

۲۴.۳ قانون ترتیب ذخیره

`SecureMessenger` همه‌چیز را سریال می‌کند (`_serial`) و **حالت را پیش از پیامک ذخیره می‌کند.** اگر پیامک فرستاده شود و حالت ذخیره نشود، کلید دوبار مصرف می‌شود. برعکس آن فقط یک پیام شکست‌خورده می‌دهد که می‌شود دوباره فرستاد.

۲۴.۴ برقراری کانال

کاربر فقط پیامش را می‌نویسد. همه‌چیز خودکار است.

۱. `SecureIdentities.forNumber`: کلید خودمان برای این شماره.

۲. INIT می‌رود، بنر «در حال برقراری کانال امن…» نشان داده می‌شود و پیام در صف می‌ماند.

۳. RESPONSE می‌رسد و پیام‌های صف فرستاده می‌شوند (`_flush`).

۴. ارتقا پشت پیام اول شروع می‌شود (`_maybeUpgrade`).

مقدار	معنی
`pendingTimeout`	۱۵ دقیقه: INIT بی‌پاسخ دوباره فرستاده می‌شود
`crossedGrace`	۳ دقیقه: مهلت INIT همزمان
`_noAnswerAfter`	۲ دقیقه: بنر «پاسخی نیامد» با `SecureRetryChannel`
`upgradeRetry`	۳۰ دقیقه
`upgradeRetryWhenHeard`	۲ دقیقه بعد از شنیدن طرف

کانالی که نمی‌آید هرگز ساکت نیست. شکست ارسال درخواست با شناسهٔ `cs:<phone>:<sid>` (`_controlId`) ردیابی می‌شود و `markSendFailed` آن را علامت می‌زند. `channelInfo` وضعیت را به صفحه می‌دهد.

۲۴.۵ صف ورودی

`sm_inbox`: هر بسته تا ۷ روز یا ۵۰ تلاش می‌ماند، چون بسته‌ای که هنوز نمی‌شود باز کرد (نشستش نرسیده) شاید بعداً باز شود.

`sm_answered_requests`: درخواست‌های جواب‌داده‌شده، جدیدترین ۱۰٬۰۰۰ تا. **INIT تکراری کانال را خراب نمی‌کند،** چون هم پیامک ممکن است دوبار برسد و هم حمله‌کننده می‌تواند بازپخش کند.

۲۴.۶ نشست‌ها

نوع در `sm_sessions`	معنی
`cert-i`، `cert-r`	نشست دست‌دهی، آغازگر یا پاسخ‌دهنده

نوع در `sm_sessions`	معنی
`pq`	نشست ارتقایافته
`pq-standby`	آماده، تا شنیدن طرف

`keepNewestStandby` و `maxStandby` (۲) نگه می‌دارند که نشست‌های آماده زیاد نشوند.

هویت طرف مقابل با `_owner`، `phoneForPeerKey` و `bindCertPeer` مشخص می‌شود. گوشی دو سیمی ممکن است با سیم دیگری جواب بدهد، پس بسته‌ها با شناسهٔ کلید یا نشست مسیریابی می‌شوند، نه با شمارهٔ فرستنده. این خطا روی گوشی واقعی پیدا شد: یک RESPONSE دور انداخته‌شده کانال را ۲۴ ساعت متوقف می‌کرد.

`peer_generation` نسل قدیمی‌تر طرف را رد می‌کند (گوشی گم‌شده). `_keysCurrent` و `_current` پیش از فرستادن بررسی می‌کنند که کلید خودمان هنوز همان باشد.

۲۴.۷ سیم‌کارت

پیام رمز با سیم انتخابی در منوی «پیام‌های رمز» می‌رود («ارسال با سیم ...»، `secure_meta` کلید `sms_sim`). پیام رمز از سیم پیش‌فرض می‌رفت که گاهی اعتبار نداشت.

۲۴.۸ سربرگ صفحه

سربرگ فقط «رمزشده» یا «پیامک رمزنشده» می‌گوید. **ارتقا نامرئی است** و کاربر کاری با آن ندارد.

۲۴.۹ دیده شد و حذف

امکان	چطور
«دیده شد»	`markSeen` با نوع `0x10`. تنظیم `send_seen_receipts`
«حذف پس از دیدن»	پرچم `0x80` روی محتوا
«حذف برای هر دو»	نوع `0x11`، **فقط پیام خودمان**
ترک گفتگو	`leaveConversation`

هر دو بسته‌های کنترلی‌اند و یک پیامک هزینه دارند.

۲۴.۱۰ ظاهر

`SecureInboxEntry` ردیف بالای تب پیام‌هاست و فقط وقتی بخش باز است نشان داده می‌شود. `SecureMessagesBloc` با قفل بخش خالی می‌شود، ولی شماره یا گروه صفحهٔ باز را نگه می‌دارد تا بعد از باز شدن دوباره صفحه خالی نماند.

۲۴.۱۱ گروه رمز

تصمیم‌ها

- **هر عضو** می‌تواند از روی دارندگان کلید گروه بسازد. فایل کلید یا ابزار ویندوزی عوض نمی‌شود.
- **سازنده حالت را انتخاب می‌کند:** «گفتگوی گروهی» (پاسخ به همه می‌رسد) یا «اطلاع‌رسانی» (پاسخ فقط به سازنده).
- **فقط سازنده** اعضا را عوض می‌کند.
- «دیده شد» **فقط به فرستنده** می‌رود.

پخش از روی نشست‌های دوبه‌دو

پیامک گروهی وجود ندارد، پس یک پیام گروه **برای هر عضو جدا، روی نشست خودش** مهر می‌شود. کلید فرستنده فقط پردازش را کم می‌کرد و یک پیامک هم کم نمی‌کرد. پس هزینه روشن است: هر پیام، هر گیرنده، یک پیامک. جفتی که هرگز حرف نزده‌اند یک بار ۲+۲ بخش می‌پردازند.

قالب

- `FLAG_GROUP` (`0x40`) روی متن: ۸ بایت شناسهٔ گروه و بعد متن.
- `KIND_GROUP_INFO` (`0x12`)، بستهٔ کنترلی بدون اعلان:

```
id ‖ version ‖ mode ‖ name ‖ count ‖ count × (شناسهٔ کلید ۸ بایتی ‖ BCD ‖ تعداد رقم)
```

اعضا همه به جز سازنده هستند، چون هر گیرنده فرستنده را اضافه می‌کند. شناسه‌های کلید به گوشی کمک می‌کنند **خودش** را پیدا کند. عضوی که هنوز جواب نداده با شناسهٔ صفر می‌آید و گیرنده خودش را با شمارهٔ گواهی می‌شناسد. سقف ۱۰۰ عضو و نام ۲۰۰ بایت است و تجزیه سخت‌گیرانه است.

قوانین موتور

- **تعریف فقط از سازنده و فقط با نسخهٔ جدیدتر** پذیرفته می‌شود (`SecureGroupStore.applyInfo`).
- **گروهی که اول با پیام دیده شود** جای‌نگهدار است (`pending_info`) و سازنده‌اش اولین فرستندهٔ تعریف است.
- **تعریفی که این گوشی را ندارد** گروه را `left` می‌کند (فقط خواندنی).
- **`_flush` برای هر عضو:** اول تعریف نرسیده (`version` > `info_version`)، بعد پیام‌های دوبه‌دو، بعد پیام‌های گروه. عضو حذف‌شده تا خبر نگرفته `removed` می‌ماند.
- **گیرنده‌ها (`SecureGroup.recipients`):** گفتگو به همه؛ اطلاع‌رسانی به همه اگر سازنده ما باشیم، وگرنه به سازنده.
- **گفتگوهای دوبه‌دوی لازم فهرست نمی‌شوند** (`sm_conversations.listed = 0`) تا پیامی در آن‌ها گفته شود.
- **گزارش تحویل:** شناسهٔ `g:<messageId>:<phone>`.
- **پذیرفته‌شده:** «دیده شد» به نقطه‌ای در نشست یک جفت اشاره می‌کند، پس پیام‌های دوبه‌دوی قبلی آن جفت را هم می‌پوشاند.

جدول‌ها و صفحه‌ها

`sg_groups`، `sg_members`، `sg_messages` و `sg_deliveries` (برای هر گیرنده) فقط با `SecureGroupStore` دست‌کاری می‌شوند. `SecureGroupScreen` نام فرستنده، «دیده شده ۲ از ۳»، وضعیت هر عضو، خط هزینهٔ «به N نفر · هر پیام N پیامک» و بنرها را دارد.

۲۴.۱۲ محدودیت‌های شناخته‌شده

- گوشی با گواهی از دو مرجع با اولی جواب می‌دهد که شاید طرف قبول نداشته باشد.
- حذف عضو چیزی را لغو نمی‌کند. لغو در طراحی نیست و گواهی او با هرکسی که خبر نگرفته کار می‌کند.

۲۴.۱۳ آزمون

`secure_messenger_test.dart`: سه موتور روی رادیوی شبیه‌سازی‌شده با `secure.db` واقعی: دست‌دهی، صف، بی‌نظمی، تکرار، بازپخش، INIT همزمان و گم‌شده، دو سیم، دیده شد، حذف، زمان‌دار، گروه، کلید جایگزین‌شده، گوشی گم‌شده.

بخش ۲۵. دفترچهٔ مخفی

دفترچهٔ تلفنی جدا که **در دفترچهٔ سیستم نیست** و تماس‌ها و پیامک‌هایش **در سیستم عامل ثبت نمی‌شوند** و برنامه‌های دیگر آن‌ها را نمی‌بینند.

تصمیم‌های رفتاری:

- مخاطب مخفی وقتی بخش **قفل است** با شمارهٔ خالی زنگ می‌زند. نام فقط وقتی بخش باز است نشان داده می‌شود.
- تماس بی‌پاسخ او فقط «تماس بی‌پاسخ» می‌گوید.
- پیامک ساده‌اش هم مخفی می‌شود.
- دفترچه و تاریخچه‌اش ردیف‌هایی بالای «مخاطبین» و «اخیر» هستند و فقط با بخش باز دیده می‌شوند.

۲۵.۱ مشکل و سه قطعهٔ پاسخ

هر اتفاقی که **در حالت قفل** می‌افتد (پایان تماس، رسیدن پیامک) در Kotlin اجرا می‌شود و Kotlin کلید `secure.db` را ندارد. سه قطعه این را بدون نوشتن راز در پایگاه‌دادهٔ اصلی حل می‌کنند.

برچسب‌ها (`hidden/HiddenNumbers.kt`)

Kotlin مجموعهٔ مخفی را فقط به شکل `HMAC-SHA256(کلید برچسب، شمارهٔ استاندارد)` بریده‌شده به ۱۶ بایت در SharedPreferences می‌شناسد.

- کلید برچسب ۳۲ بایتی با کلید Keystore غیرقابل‌صدور مهر شده است (`hamresan_hidden_tags_v1`).
- کپی فایل‌ها نشان می‌دهد **چند** شماره مخفی است، نه کدام‌ها. فضای شماره‌ها برای هش بدون کلید قابل حدس است، نه برای این.
- بدون برچسب، `isHidden` بدون رفتن سراغ Keystore `false` می‌دهد. پس نسخهٔ تجاری هزینه‌ای نمی‌دهد.
- استانداردسازی `BlockedNumbers.normalizeToThreadId` است، پس `+98…`، `0098…` و `09…` یکسان‌اند.

رکوردهای مهرشده (`smscrypto/SealedBox.kt`)

مهر یک‌طرفه به زوج‌کلید خود بخش (ML-KEM-768 و X25519):

```
FORMAT ‖ X25519 متن ‖ (۳۲) موقت ML-KEM (۱۰۸۸) ‖ AES-256-GCM
و کلید nonce = HKDF(ssKem ‖ X25519, salt = SHA-256(سرآیند ‖ گیرنده ‖ برچسب))
```

- سربار ۱۱۳۷ بایت است و `SealedBoxTest` قالب را ثابت می‌کند.
- راز در `secure_meta` با کلید `sealing_identity` (`SealingKey`) است و با اولین باز شدن ساخته می‌شود. نیمهٔ عمومی با هر باز شدن به Kotlin می‌رود.

Kotlin می‌تواند مهر کند ولی نمی‌تواند باز کند.

`sealed_queue`

جدول در پایگاه‌دادهٔ اصلی (`hidden/SealedInbox.kt` و `DatabaseHelper._createSealedQueueTable`) با ستون‌های `id, kind, blob, created_at`. فقط `kind` (`call` یا `sms`) آشکار است.

```
{"k":"sms", "address":"…", "body":"…", "timestamp":0, "subscriptionId":0}
{"k":"call", "number":"…", "type":0, "date":0, "duration":0, "account":"…"}
```

Dart با `openSealed` آن‌ها را باز می‌کند. رکوردی که باز نشود (مهرشده پیش از بازنشانی) دور انداخته می‌شود.

۲۵.۲ تماس‌ها

Telecom هر تماس اپراتوری را خودش در `CallLog.Calls` می‌نویسد و تلفن پیش‌فرض نمی‌تواند جلویش را بگیرد. پس `HiddenCallLog` ردیف را **جابه‌جا می‌کند**: اول مهر، بعد حذف. حذف ناموفق دفعهٔ بعد دوباره مهر می‌شود و `hc_calls` با `UNIQUE(phone, timestamp, call_type)` تکراری را حذف می‌کند.

زمان جاروب	محدوده
بعد از تماس مخفی (`onCallRemoved`)	حدود ۱۵ ثانیه تلاش، چون ردیف کمی بعد می‌آید
هر `MainActivity.onResume` و پیش از خواندن آینه	تدریجی، ۲ روز عقب
تغییر مجموعهٔ مخفی	کل دفتر تماس: تماس‌های گذشته هم می‌آیند

- `CallLogSyncHandler` شماره‌های مخفی را از `callLogEntries` و فهرست شناسه‌ها **فیلتر می‌کند**.
- `CallInCallService.lookupContactName` نام حافظه (`HiddenNumbers.setNames`) یا `null` می‌دهد، هرگز دفترچهٔ گوشی، و عکس ندارد.
- تماس بی‌پاسخ: «تماس بی‌پاسخ» خالی با شناسهٔ `0x5EC1`، بدون شماره و دکمه، و `markMissedCall` نمی‌شود چون آن شماره را آشکار ذخیره می‌کند. launch action برابر `{type: hiddenCalls}` است.

۲۵.۳ پیامک

- `SmsDeliverReceiver` برای فرستندهٔ مخفی provider را رد می‌کند.
- `HiddenSms.receive` در هر دو گیرنده، پیش از شاخهٔ `#E:`، کل پیام را مهر می‌کند و **همان اعلانی** را می‌دهد که `SecureSmsInbox.arrival` تصمیم می‌گیرد.
- `SecureMessenger.drain` اول این‌ها را می‌گیرد (`HiddenSmsSource`). `#E:` به `sm_inbox` می‌رود و ساده به‌عنوان پیام ساده ذخیره می‌شود.
- `querySms` و خانواده‌اش آدرس‌های مخفی را فیلتر می‌کنند.
- `SmsService.sendSms` برای شمارهٔ مخفی `HIDDEN_NUMBER` می‌دهد: «این شماره در دفترچه مخفی است؛ از بخش امن پیام دهید».

رمز و ساده هرگز یک گفتگو نیستند

گفتگوی ساده زیر `SecureMessenger.plainThread(phone)` برابر `p:<phone>` است (`isPlainThread`، `numberOf`).

- `sendPlain` فقط روی `p:` کار می‌کند، با `private: true` می‌فرستد و دست‌دهی و «حذف پس از دیدن» ندارد.
- سربرگ «پیامک رمزنشده» می‌گوید و بنر جای گفتگوی رمز را نشان می‌دهد.
- فهرست گفتگوها `p:` را کنار می‌گذارد. ردیف «پیامک‌های رمزنشدهٔ مخفی» با تعداد نخوانده `SecureInboxScreen(plain:` `true)` را باز می‌کند.
- `rename` هر دو را عوض می‌کند.

قبلاً یک گفتگو برای هر شماره بود و یک INIT گفتگوی ساده را رمز می‌کرد.

۲۵.۴ مخفی کردن شماره، تاریخچه را می‌برد

`HiddenBloc._onSave` به این ترتیب:

۱. شماره‌های استاندارد (`canonicalPhone`). هر شماره فقط مال یک مخاطب مخفی است.

۲. نوشتن در `hc_contacts` و `hc_numbers`.

۳. برچسب و نام به Kotlin و جاروب کامل.

۴. تخلیهٔ تماس‌های مهرشده.

۵. برای هر شماره‌ای که **تازه** مخفی شده: `HiddenHistoryMover.smsHistory` و `SecureImportHistory` که فقط بعد از ذخیره تمام می‌شود.

۶. **فقط بعد از آن** `purgeSms` و `purgeCalls`.

`purgeSms` گفتگو را از provider حذف می‌کند (بدون نقش پیامک کاری نمی‌کند و فیلتر بومی کار را نگه می‌دارد) و همهٔ ردیف‌های محلی را **سخت** حذف می‌کند، با tombstoneها، `message_search` و نشان‌های سنجاق، بایگانی، سیم و نخوانده.

«از دفترچه تلفن گوشی» فرم را از یک مخاطب پر می‌کند و بعد از ذخیره حذف آن را پیشنهاد می‌دهد، حتی مخاطب سیم‌کارت. حذف مخاطب مخفی مخفی بودن را تمام می‌کند و آنچه جابه‌جا شده می‌ماند.

۲۵.۵ حفره‌های بسته‌شده

- **زمان‌بندی برای شمارهٔ مخفی رد می‌شود.** شماره‌ای که **بعد از** زمان‌بندی مخفی شود بدون ردیف provider فرستاده و با `{outgoing: true}` مهر می‌شود (`HiddenSms.sendPrivately`، هر دو تحویل‌دهنده).
- **پیامک اپراتور با نام شمارهٔ مخفی** («تماس از 0912...») در گفتگوی همان مخاطب مهر می‌شود (`about`) و از آینه حذف می‌شود.
- **بدون نقش پیامک،** ذخیرهٔ مخاطب مخفی گرفتن نقش را پیشنهاد می‌دهد.

۲۵.۶ ذخیره و برنامه

- جدول‌ها: `hc_contacts`، `hc_numbers` و `hc_calls`.
- `HiddenBloc` **lazy نیست** و بعد از `MessageBloc`، `CallLogBloc` و `SecureMessagesBloc` می‌آید: با باز شدن کلید مهر را می‌سازد، برچسب و نام را می‌فرستد، جاروب می‌کند و تماس‌ها را تخلیه می‌کند. با قفل نام‌ها را پاک می‌کند.
- `SecureStore.reset` برای از بین بردن برچسب، کلید برچسب و کلید مهر `HiddenBridge.forget` را صدا می‌زند: بدون دفترچه چیزی نباید مخفی بماند.
- کانال `hidden`: `setNumbers`، `setNames`، `clearNames`، `setSealingKey`، `sweep`، `isHidden`، `forget`، `clearMissedNotice`. متدهای `security.*` هم از همین کانال می‌روند.
- صفحه‌ها (`lib/features/hidden/`): `HiddenContactsEntry`، `HiddenCallsEntry`، `HiddenContactsScreen`، `HiddenContactEditScreen`، `HiddenContactScreen` و `HiddenCallsScreen`. ردیف‌ها بالای فهرست‌ها هستند تا offset پرش سریع مخاطبین عوض نشود.

۲۵.۷ محدودیت‌ها

بدون نقش پیامک، پیامک‌های جابه‌جاشده در provider سیستم می‌مانند و فقط از این برنامه پنهان‌اند. جزئیات آزمون روی دو گوشی در `docs/FINAL_DEVICE_TESTS.md` است.

۲۵.۸ آزمون

- `SealedBoxTest` : رفت‌وبرگشت، کلید اشتباه، هر بایت تغییرکرده، قالب و مقدار طلایی.
- Dart: repository (ترتیب، جایگزینی، مالکان و تکرار تماس)، `HiddenBloc` (باز شدن، رد شدن، جابه‌جایی تاریخچه فقط بعد از رسیدن به بخش، تخلیه، حذف، قفل) و messenger.

بخش ۲۶. امکانات امنیتی دیگر

۲۶.۱ پیام زمان‌دار

«پیام زمان‌دار» یعنی **عمر محدود**، نه ارسال زمان‌بندی‌شده.

- پرچم `FLAG_TIMED` (`0x20`) و بعد از شناسهٔ گروه، ثانیه به شکل varint، حداکثر یک هفته (`TimedPayloadTest`).
- **پیام خودمان** `ttl` بعد از فرستادن و **پیام طرف** `ttl` بعد از اولین نمایش می‌رود.
- ستون `ttl` در `sm_messages` و `sg_messages` است. `purgeExpired` با باز شدن و با تایمری از `nextExpiry` اجرا می‌شود.
- دکمهٔ زمان composer «پس از دیدن» یا یک عمر را انتخاب می‌کند.

۲۶.۲ متن پوششی (فقط نسخهٔ سازمانی)

بسته‌های پیام و رسید **به شکل کلمات عادی** می‌روند تا از دید فیلتر پیامک متن رمز نباشند.

`kt/smscrypto/CoverText.kt`، قالب ۲:

```
یک بیت ۱ ‖ بیت‌های ۰ تا مضرب ۱۲ ‖ (بسته "hamresan.cover.v2" از SHA-256) بسته ‖ ۲ بایت چک
هر ۱۲ بیت → یک کلمه از فهرست منجمد ۴۰۹۶ کلمه‌ای
```

نکته	توضیح
واژه‌ها	`CoverWords.kt` تولیدشده است و هرگز دستی ویرایش نمی‌شود
بدون رمز جدا	نشست خودش رمز و احراز می‌کند و گیرنده باید پوشش را بدون کلید بشناسد
واژه‌های ممنوع	`CoverBlocked.kt` (در پوشهٔ آزمون) فهرست را فقط به شکل SHA-256 بریده‌شده دارد و `CoverTextTest` بررسی می‌کند. شامل ناسزا، جنسی، توهین، مقدس، سیاسی، نظامی، مواد، مرگ و کلمات فیشینگ
هزینه	بستهٔ ۴۰ بایتی ۲۸ کلمه است: حدود ۱۷۰ نویسه (۳ بخش فارسی) یا ۱۹۰ نویسه (۲ بخش انگلیسی)
ظاهر	جمله‌های ۵ تا ۱۲ کلمه که پایانشان را مقدار کلمه تعیین می‌کند

- **هر گیرنده در هر نسخه** اول `CoverText.uncover` را اجرا می‌کند، پس بقیهٔ مسیر فقط `#E:` می‌بیند. به همین دلیل گیرندهٔ تجاری یا بین‌سازمانی هم می‌تواند بخواند.
- **دست‌دهی هرگز پوشانده نمی‌شود.**
- تنظیم در منوی «پیام‌های رمز» و `secure_meta` کلید `cover_text`.

> این متن‌ها **فقط از نظر ظاهر** طبیعی هستند و هزینهٔ پیامک بیشتری دارند. متن پوششی هنوز بین دو گوشی واقعی فرستاده نشده است و در فهرست آزمون‌های نهایی قرار دارد.

۲۶.۳ کنترل امنیت سامانه

تنظیمات ← امنیت. یک گزارش از وضعیت گوشی.

- `kt/security/DeviceSecurity.kt` **فقط واقعیت‌ها** را گزارش می‌کند.

- `security/models/device_security_report.dart` آن‌ها را وزن‌دهی و به فارسی بیان می‌کند و آزمون دارد.

بررسی	معنی
روت	نشانه‌های شناخته‌شده
قلاب	چارچوب‌های hook در همین فرایند
امضاکننده	غیر از کلید انتشار
debuggable و شبیه‌ساز	ساخت غیرواقعی
قفل صفحه، اشکال‌زدایی USB	تنظیم گوشی
سرویس‌های دسترس‌پذیری	با `<queries>` نام برده می‌شوند
سن وصلهٔ امنیتی، رمزگذاری حافظه	به‌روز بودن
جای کلید vault	StrongBox، TEE یا نرم‌افزار
نقش‌های پیش‌فرض	پیامک و تلفن

۲۶.۴ عکس ورود ناموفق

خاموش به‌طور پیش‌فرض. روشن کردن مجوز `CAMERA` را از `PermissionGate` می‌خواهد.

- **هر سومین PIN اشتباه پیاپی برنامه:** یک JPEG از هر دوربین (Camera2، بدون پیش‌نمایش و صدا).
- **مهر با** `SealedBox` به کلید بخش در `noBackupFilesDir/intruder/` .
- `IntruderRepository` آن‌ها را بعد از باز شدن بخش به `ip_photos` می‌برد.
- `IntruderReporter` پنجرهٔ گزارش را در PIN درست بعدی نشان می‌دهد.
- PIN اشتباه بخش امن هم شمرده می‌شود.
- **محدودیت:** نشانگر دوربین اندروید ۱۲+ روشن می‌شود و نمی‌شود خاموشش کرد.

`uses-feature` برای دوربین `required=false` است تا گوشی بدون دوربین از فروشگاه حذف نشود.

۲۶.۵ امکانات حذف‌شده

برای اینکه کسی دوباره سراغشان نرود:

ردیف	امکان	چرا
۱۲	رمز کردن کل پایگاه‌دادهٔ اصلی	Kotlin در حالت مرده و بدون کلید آن را باز می‌کند، همهٔ رازها در SQLCipher هستند و پیامک عادی در provider سیستم هم هست
۲۵	مکان و پاک‌سازی با پیامک خاص	نوشته و بعد از ساخت حذف شد و برنمی‌گردد
۴۱	حالت‌های کاری	ساخته نشد
۳۶	ردیف «مخاطبین رمز»	با حذف فهرست اعضا فقط در دیگری به «گفتگوی رمز» بود

بخش ۲۷. ابزار و پنل ویندوزی

پوشهٔ `tools/keybank/` دو ابزار دارد که کد فعال‌سازی و فایل کلید می‌سازند.

ابزار	برای چه کسی	چطور اجرا می‌شود
پنل صدور	مسئول سازمان	فایل نصب `HamresanPanel-1.5.0.msi`. جاوا لازم ندارد
ابزار خط فرمان	تیم فنی	`hamresan-keybank.cmd`. جاوا ۱۱ یا بالاتر لازم دارد

> این ابزارها سورس رمزنگاری را **از خود برنامه** کامپایل می‌کنند (`smscrypto/` منهای سه فایل اندرویدی). پس هر فایلی که بسازند، برنامه هم می‌خواند. **این کد را هرگز کپی نکنید.**

۲۷.۱ ساختار کد

```
tools/keybank/
  build.gradle.kts        سورس smscrypto را از مسیر برنامه می‌گیرد
  hamresan-keybank.cmd    اجرای jar
  dev/                    مرجع آزمایشی، خارج از git
  src/main/kotlin/
    Main.kt               فرمان‌ها
    Issuance.kt           صدور و AuthorityFolder، مشترک خط فرمان و پنل
    IssuanceLog.kt        دفتر صدور
    Activation.kt         کد فعال‌سازی، سمت صادرکننده
  src/test/kotlin/        ActivationTest، AuthorityFolderTest
  panel/
    build.gradle.kts      Compose Desktop و ساخت MSI
    rules.pro             ProGuard
    src/main/kotlin/      صفحه‌های پنل
    src/test/kotlin/      PreviewRender
```

۲۷.۲ ساخت

`JAVA_HOME` باید روی JDK 21 باشد.

```
android/gradlew -p tools/keybank test :panel:test
android/gradlew -p tools/keybank jar
android/gradlew -p tools/keybank :panel:packageReleaseMsi
```

خروجی	مسیر
خط فرمان	`tools/keybank/build/libs/hamresan-keybank.jar`
پنل	`tools/keybank/panel/build/compose/binaries/main-release/msi/`
تصویرهای راهنما	`tools/keybank/panel/build/previews/`

اگر پنل تغییر کرد، `packageVersion` را در `panel/build.gradle.kts` بالا ببرید و راهنمای پنل را به‌روز کنید. فقط `:test` را برای آزمون ابزار بزنید، چون `test` بدون دو نقطه آزمون پنل را هم اجرا می‌کند.

۲۷.۳ فرمان‌های خط فرمان

فرمان	کار
`init authority.hka`	ساختن مرجع، فقط یک بار. رمز حداقل ۱۲ نویسه می‌خواهد
`public authority.hka`	چاپ کلید عمومی برای `trust_anchors.dart`
`issue authority.hka roster.csv out`	ساختن فایل‌های کلید و `passwords.csv`
`inspect file.hkb authority-public.txt`	بررسی یک فایل
`activation <device> [--organization]`	کد فعال‌سازی، بدون نیاز به فایل مرجع

برای اجرای خودکار، رمز مرجع در متغیر `HAMRESAN_KEYBANK_PASSWORD` قرار می‌گیرد.

فهرست اعضا

```
organization,نام سازمان
name,phones,generation
نام عضو اول,شماره همراه;شماره دوم
نام عضو دوم,شماره همراه
```

- ذخیره با UTF-8. `passwords.csv` هم UTF-8 با BOM است تا Excel فارسی را درست نشان دهد.
- کلید از **اولین شمارهٔ همراه** ساخته می‌شود، پس ترتیب شماره‌ها را عوض نکنید.
- `generation` اختیاری است و پیش‌فرض ۰ است.
- اجرای دوباره کلید و رمز هیچ عضو قبلی را عوض نمی‌کند.

۲۷.۴ کد مشترک

کلاس	کار
`Issuance`	همه یا هیچ: اول در `issued/.issuing/` می‌نویسد و بعد جابه‌جا می‌کند
`AuthorityFolder`	پوشهٔ مرجع: `authority.hka`، `roster.csv`، `issued/`، `backups/`
`addMember`، `editMember`، `removeMember`، `newKey`	تغییر اعضا. `prepare*` اول اعتبارسنجی می‌کند و `issueOne` / `removeOne` فقط فایل همان عضو را عوض می‌کنند
`nextGeneration`	نسل بعدی برای گوشی گم‌شده
`Activation`	سومین پیاده‌سازی مشتق‌گیری، با `Blake3Digest` از Bouncy Castle
`IssuanceLog`	`issuance-log.csv`، با وضعیت‌های `NOT_RECORDED`، `NEW_KEY` و `CURRENT`

`ActivationTest` سه‌تایی‌هایی را ثابت می‌کند که با برنامه یکسان‌اند. پس اگر کسی یک طرف را عوض کند، آزمون می‌شکند.

۲۷.۵ پنل صدور

Compose Desktop 1.12.1 و Kotlin 2.4.0. نامش «پنل صدور هم‌رسان» است.

منو	کار
«فعال‌سازی بین‌سازمانی»	کد ۶ نویسه‌ای دستگاه و کد فعال‌سازی
«فعال‌سازی سازمانی»	همان برای APK سازمانی

منو	کار
«فایل کلید اعضا»	باز شدن با فایل مرجع و رمز آن

در «فایل کلید اعضا» این کارها ممکن است:

- **«عضو جدید»:** کلید و رمز تازه. برگهٔ تحویل با `DeliverySheet` چاپ می‌شود.
- **«ویرایش»:** نام یا شماره‌ها.
- **«گوشی گم شد»:** `generation` یکی زیاد می‌شود و رمز تازه می‌آید.
- **«حذف از سازمان»:** فایل حذف می‌شود، ولی **چیزی لغو نمی‌شود**.

فایل پنل	کار
`PanelApp.kt`	پنجره، ریل و رنگ‌ها
`Common.kt`	تکه‌های مشترک مثل متن LTR
`ActivationScreen.kt`	دو صفحهٔ فعال‌سازی
`KeysScreen.kt` ، `KeysState.kt` ، `MembersUi.kt`	بخش کلید و حالت آن
`DeliverySheet.kt`	صفحهٔ چاپی تحویل
`Platform.kt`	تکه‌های ویندوزی
`SelfTest.kt`	آزمایش نسخهٔ نصب‌شده

قانون‌های پنل

- **رمز مرجع هیچ‌جا ذخیره نمی‌شود.** بخش کلید با «قفل»، بستن برنامه و بعد از ۱۰ دقیقه کار نکردن قفل می‌شود.
- **تغییرها روی `KeysState.launch` اجرا می‌شوند،** نه scope پنجره. بستن پنجره نباید کار نیمه‌تمام بگذارد.
- **متن آمیخته در `Text` های جدا:** شماره و کد را از متن فارسی جدا نگه دارید، وگرنه bidi ترتیب را به هم می‌زند.
- **اولین شماره باید همراه یا خارجی باشد.**

بسته‌سازی

- **jpackage روی JDK 21:** MSI برای هر کاربر، بدون نیاز به مدیر، با `upgradeUuid` ثابت تا نسخهٔ جدید روی قبلی نصب شود.
- **ProGuard فقط کوچک می‌کند** و مبهم‌سازی ندارد.
- **Bouncy Castle بدون امضا:** `unsignedBouncyCastle` امضای jar را برمی‌دارد، چون ProGuard آن را می‌شکند.
- **androidx از مسیر JetBrains** می‌آید (redirector).
- `<پوشه> --self-test` نسخهٔ **نصب‌شده** را بدون باز کردن پنجره بررسی می‌کند و نتیجه را در `self-test.txt` می‌نویسد. این برای پیدا کردن کلاسی است که ProGuard اشتباهی حذف کرده باشد.

`PreviewRender`

آزمون‌های `:panel:test` صفحه‌ها را با فونت واقعی به PNG می‌کشند. راهنمای پنل از همین تصویرها استفاده می‌کند. پس بعد از تغییر ظاهر، تصویرهای راهنما هم با دوباره اجرا کردن آزمون به‌روز می‌شوند.

۲۷.۶ جای داده‌های واقعی

مرجع واقعی **بیرون از مخزن** روی رایانهٔ مسئول نگه داشته می‌شود. هر کس آن فایل و رمزش را داشته باشد، برای کل سازمان کلید می‌سازد. آن را در git، سند یا پیام نگذارید.

بخش ۲۸. ظاهر، RTL و ویجت‌های مشترک

۲۸.۱ سیستم طراحی

فایل	کار
`core/theme/app_colors.dart`	رنگ‌های برند. آبی اصلی `#00A6ED`، خطر `#EC0206`، متن `#334A52`
`core/theme/app_dimensions.dart`	فاصله، گردی، اندازه‌ها. **به‌جای عدد از این‌ها استفاده کنید**
`core/theme/app_theme.dart`	تم روشن و تاریک Material 3 با فونت Vazirmatn
`core/theme/surface_roles.dart`	نقش‌های سطح
`core/theme/theme_bloc.dart`	روشن، تاریک یا سیستم

۲۸.۲ سه سطح

`SurfaceRoles` روی `ColorScheme` سه لایه تعریف می‌کند، چون نقش‌های M3 در روشن و تاریک فرق دارند:

نقش	کجا	روشن	تاریک
`pageBackground`	زمینهٔ صفحه	`surfaceContainer`	`surface`
`cardSurface`	کارت‌ها و گروه‌ها	`surfaceBright`	`surfaceContainerHigh`
`raisedSurface`	صفحه‌کلید، منو، پنجره	`surfaceContainerHigh`	`surfaceContainerHigh`

این‌ها هم تعریف شده‌اند: `keySurface`، `tonalRow`، `chipUnselected`، `bubbleIncoming` و `bubbleOutgoing`.

`bubbleIncoming` همیشه یک پله با `cardSurface` فاصله دارد، وگرنه پیام دریافتی در حالت تاریک حباب نداشت.

`surface_roles_test.dart` این را ثابت می‌کند.

۲۸.۳ RTL و فارسی

- **همهٔ متن کاربر فارسی است** و لایهٔ ترجمه نداریم (K7).
- **هر صفحه یا پنجرهٔ تازه** ریشه‌اش را در `Directionality(textDirection: TextDirection.rtl, ...)` می‌پیچد یا از `RtlAppBar` استفاده می‌کند.
- **ارقام نمایشی** از `PersianUtils` می‌گذرند. رقم لاتین روی صفحه خطاست.
- **شماره تلفن** از `PhoneNormalizer` می‌گذرد. خودتان رقم جدا نکنید.
- **جهت پیام** با `ScriptDirection` از روی حروف تعیین می‌شود (جداگانه برای هر پیام).
- **`MaterialLocalizations` فارسی نداریم،** پس برچسب‌های چارچوب مثل نوار انتخاب متن دستی داده می‌شوند.
- **تاریخ:** `DateFormatter` و `JalaliDate` روی تقویم انتخابی (`CalendarType`). `jalali_date_picker.dart` انتخابگر مشترک است.

۲۸.۴ لوگو

آیکون برنامه **تولید می‌شود، نه دستی کشیده می‌شود.** `Hamresan-Logo.svg` با ابعاد ۵۱۲×۵۱۲ و زمینهٔ سرمه‌ای `#023066` به دو چیز تبدیل می‌شود:

- mipmap مربعی برای نسخه‌های قدیمی،
- پیش‌زمینهٔ adaptive شفاف با اندازهٔ ۶۸dp داخل بوم ۱۰۸dp، که امن داخل ناحیهٔ ۶۶dp قرار می‌گیرد.

`@color/ic_launcher_background` زمینهٔ خود لوگوست. هر دو را با هم از SVG تولید کنید. تصویر مربعی تمام‌صفحه به‌عنوان پیش‌زمینه، با هر ماسک گردی گوشه‌هایش بریده می‌شود.

۲۸.۵ ویجت‌های مشترک

همه در `core/widgets/` هستند. پیش از ساختن ویجت تازه این فهرست را ببینید.

ویجت	کار
`GroupedList`، `SliverGroupedList`، `SectionLabel`	فهرست کارت‌دار به سبک گوگل. **دادهٔ کاربر همیشه** `SliverGroupedList`
`InfiniteScroll`، `LoadMoreIndicator`	صفحهٔ بعدی، حتی وقتی صفحهٔ اول کوتاه است
`AvatarWidget`	دایره با حرف اول
`LazyContactAvatar`	عکس مخاطب با LRU دویست‌تایی
`PhoneContactAvatar`	آواتار از روی **شماره**
`ContactNumbersLine`، `HighlightedPhone`	خط شماره زیر نام
`ContactIndexList` و خانواده‌اش	بخش‌بندی الفبایی و نوار پرش سریع
`HomeSearchHeader`	نوار جستجوی بالای تب‌ها
`SelectionAppBar`، `SelectionCheck`	سربرگ حالت انتخاب
`SliverTwoColumnBoard`	تختهٔ دوستونی پیش‌نویس‌ها
`showUndoSnack`	نوار تأیید با واگرد
`CaretLineFix`	جای درست نشانگر در متن آمیخته
`RtlAppBar`	سربرگ راست‌چین
`PermissionGate`، `DefaultAppGate`، `AppLockWrapper`	دروازه‌های شروع

۲۸.۶ فهرست‌های بلند

فهرستی که از دادهٔ گوشی پر می‌شود (تماس، پیامک، نتیجهٔ جستجو، بایگانی، گفتگوی رمز، تماس مخفی) **صفحه‌به‌صفحه** خوانده می‌شود و با `ListView.builder` یا sliver ساخته می‌شود.

- محرک `InfiniteScroll` است که برای صفحهٔ اول کوتاه هم فعال می‌شود. listener اسکرول این حالت را نمی‌بیند.
- BLoC پشت آن یک پرچم in-flight دارد، چون محرک برای هر صفحه چند بار فعال می‌شود.
- فهرست‌های دستی (پیش‌نویس، قالب، زمان‌بندی، موردعلاقه، مسدود) و دفترچهٔ تلفن کامل خوانده می‌شوند، چون «انتخاب همه»، پرش سریع و فیلترها به کل نیاز دارند. ولی ردیف‌هایشان باز هم lazy ساخته می‌شوند.

۲۸.۷ نوار تأیید و واگرد

`showUndoSnack` (`core/widgets/undo_snack_bar.dart`) تنها snack bar دکمه‌دار برنامه است.

`SnackBar` **دکمه‌دار تضمین نمی‌کند که بسته شود:** `ScaffoldMessengerState` وقتی `MediaQuery.accessibleNavigation` فعال است تایمر را شروع نمی‌کند. پس با هر سرویس دسترس‌پذیری، «گفتگو بایگانی شد» روی صندوق می‌ماند. این تابع `Timer` خودش را دارد و مهلت ۳ ثانیه‌ای را با یک حلقهٔ کوچک‌شونده دور «واگرد» نشان می‌دهد.

برای دکمه‌ای که میانبر است و واگرد نیست (مثل «تنظیمات») از `showCountdown: false` استفاده کنید.

۲۸.۸ لمس طولانی و انتخاب

لمس طولانی **همه‌جا یک حرکت است** و `HapticFeedback.mediumImpact()` دارد.

کجا	نتیجه
حباب گفتگو	بلند شدن و انتخاب متن (بخش ۱۰)
ردیف گفتگو	انتخاب چندتایی با `MessagesSelectionAppBar`. ردیف فقط‌پیش‌نویس فقط آن را دور می‌اندازد
ردیف مخاطب	انتخاب چندتایی: ستاره، اشتراک، حذف، انتخاب همه
کارت پیش‌نویس و دسته	انتخاب چندتایی با `SelectionAppBar`

- **ردیف‌های مجازی «همه» و «بدون دسته‌بندی» قابل انتخاب نیستند،** چون ردیفی پشتشان در جدول نیست.
- **صندوق سربرگ sliver خودش را دارد،** چون سربرگش جمع می‌شود.
- **حالت انتخاب در `pickMode` پیش‌نویس خاموش است.**
- **حذف مخاطب از دفترچهٔ گوشی انجام می‌شود** و بعد cacheها باطل می‌شوند و `RefreshContacts` می‌آید.

۲۸.۹ لمس شماره در پیام

`showPhoneActionSheet` باز می‌شود: تماس، پیامک، مشاهده یا افزودن مخاطب، کپی. **هرگز `tel:` اجرا نمی‌شود:** چون خود برنامه تلفن پیش‌فرض است، intent به همین فرایند برمی‌گردد و رابط تا ۹ ثانیه یخ می‌زد. جستجوی مخاطب بعد از باز شدن شیت و در try/catch است.

۲۸.۱۰ نوار «بازگشت به تماس»

`ReturnToCallBar` در `core/navigation/` نوار سبز بالای برنامه است و در بخش ۱۳ توضیح داده شده است.

بخش ۲۹. سرعت

بیشتر مسیرهای داغ برنامه با دادهٔ زیاد گوشی سروکار دارند: هزاران پیامک، تماس و مخاطب روی isolate اصلی. این جدول کارهایی را نشان می‌دهد که نباید خراب شوند.

۲۹.۱ بهینه‌سازی‌های موجود

قسمت	روش	کجا
نگاشت تماس	`Isolate.run` برای نگاشت، بعد افزودن نام روی isolate اصلی	`CallLogService.getCallLogs`
ورود پیامک	صفحه‌های ۲۰۰تایی، بدون سقف	`SmsService.syncDeviceMessages`
تکرار پیامک	`ConflictAlgorithm.ignore` روی ایندکس یکتا، بدون خواندن پیش از نوشتن	`MessageRepository`
ذخیرهٔ تماس	یک تراکنش batch	`CallLogRepository.saveCallLogsBatch`
نام مخاطب	نقشهٔ شماره به نام، یک بار در هر جلسه	`MessageBloc._cachedPhoneToName`
جابه‌جایی تب	`_FadeIndexedStack` همهٔ تب‌ها را نگه می‌دارد	`MainNavigation`
نشان نخوانده	`buildWhen` فقط با تغییر جمع	`MainNavigation`
فیلتر شماره‌گیر	۳۰۰ debounce میلی‌ثانیه	`DialerBloc`
فیلتر مخاطب	۲۰۰ debounce میلی‌ثانیه، نتیجه با (عبارت، منبع) نگه داشته می‌شود	`ContactsListScreen`
ایندکس‌ها	`thread_id` ، `timestamp DESC` ، `is_read` و ایندکس یکتا	`DatabaseHelper._createDB`
«اخیر»	هرگز کل `call_logs` را نمی‌خواند. `hasAnyCallLogs()` با `LIMIT 1`	`CallLogService`
آواتار	fetch تکی LRU، lazy، دویست‌تایی، cache منفی، حذف درخواست تکراری	`LazyContactAvatar`
فهرست‌ها	virtualized با `SliverGroupedList`	`core/widgets/google_list.dart`

۲۹.۲ بودجهٔ بازگشت به برنامه

برگشت به برنامه قبلاً در **هر بار** این کارها را می‌کرد: خواندن و نوشتن تماس‌های ۳۶۵ روز، دو بار خواندن کل دفترچه و پاک کردن cache تصویر. حالا:

محرک	چه چیزی مجاز است
بازگشت به برنامه	sync تدریجی تماس (ردیف‌های تازه‌تر)، آینهٔ پیامک هر ۲ دقیقه، دفترچه **حداکثر هر ۱۰ دقیقه**
تغییر واقعی دفترچه	به‌روزرسانی فوری و پاک کردن cache تصویر
ContentObserver دفتر تماس	sync تدریجی، تفاوت حذف‌ها حداکثر هر ۵ دقیقه
کشیدن برای تازه‌سازی	همه‌چیز، بدون محدودیت

- تشخیص حذف **فقط شناسه‌ها** را بومی می‌خواند (`callLogIdsSince`).

- ورود اول تماس‌ها برش‌های ۳۰ روزه با وقفه است و چیزی در حافظه جمع نمی‌کند.
- `reconcileDeviceRows` با یک `IN (…)` برای کل دسته پاسخ می‌دهد.
- `DeviceSyncQueue` **خواندن‌های سنگین را سریال می‌کند.** شروع سرد سه payload بزرگ کانال را همزمان نگه نمی‌دارد. همین اوج، برنامه را روی گوشی کم‌حافظه می‌کشت.
- `getContactByPhoneNumber` یک map است و جستجوی خطی نیست.

۲۹.۳ تب پیام‌ها

سه مشکل ساختاری بود که هیچ‌کدام به حجم داده ربط نداشت:

۱. **sync پیش از اولین نقاشی منتظر می‌ماند.** حالا آینهٔ محلی اول نقاشی می‌شود و sync جدا اجرا می‌شود (`_startBackgroundSync` و `DeviceSyncFinished`). `listenToIncomingSms` پیش از sync اجرا می‌شود تا پیامک این فاصله گم نشود.

۲. **صفحه‌بندی پرچم in-flight نداشت** و یک fling ده‌ها پرس‌وجو می‌ساخت. `_loadingMoreThreads` و `_loadingMoreMessages` این را حل کردند.

۳. `getAllThreads` **کار هر گفتگو را پیش از** `LIMIT` **می‌کرد.** حالا در یک CTE به نام `page` صفحه‌بندی می‌کند و زیرپرس‌وجوها فقط روی ردیف‌های همان صفحه اجرا می‌شوند.

`removeRowsMissingFromDevice` هم تفاوت را داخل SQLite و با جدول موقت شناسه‌ها حساب می‌کند.

۲۹.۴ دام‌ها

- **پایگاه‌داده روی isolate اصلی است.** صفحه‌بندی را نگه دارید.
- **محدودیت Binder حدود ۱ مگابایت است.** آینه با فهرست شناسه و صفحه‌های شناسهٔ غایب کار می‌کند. آن را صفحه‌دار نگه دارید.
- `getAllContacts` **کند است.** آن را در `itemBuilder` یا هر رویداد صدا نزنید.
- **هرگز تعداد ردیف وابسته به داده را در** `GroupedList` **یا** `ListView(children:)` **نگذارید.** هر دو همهٔ ردیف‌ها را می‌سازند و هر ردیف یک خواندن آواتار دارد. یک رقم در شماره‌گیر هزاران ویجت و کانال می‌سازد.
- `setState` **در هر کلید composer** برای شمارندهٔ پیامک فعلاً ارزان است. اگر composer بزرگ شد، آن را به `ValueListenableBuilder` محدود کنید.

۲۹.۵ قانون بازسازی

- از `const` استفاده کنید.
- در لیست‌ها و پوستهٔ ناوبری `buildWhen` و `listenWhen` بگذارید.
- ویجت‌های برگ بدون حالت و `const` باشند.
- وضعیت Equatable است، پس emit یکسان نقاشی ندارد.

۲۹.۶ چطور اندازه بگیریم

```
flutter run --profile
```

بعد DevTools و Performance را باز کنید. **اولین باز شدن «اخیر»** و **اولین ورود «پیام‌ها»** را ببینید و برای فهرست پرآواتار، raster thread را بررسی کنید.

۲۹.۷ شروع برنامه

روی گوشی آزمون (Samsung A33): «اخیر» با نام، حدود یک ثانیه بعد از `am start` ، بدون فریم افتاده. دو دروازهٔ اول صفحهٔ خالی می‌کشند و `hasAllRequiredPermissions` خواندن‌هایش را موازی اجرا می‌کند. **اسپینر سر راه نگذارید:** هر بار شروع برنامه را شبیه به بارگذاری می‌کند.

۲۹.۸ ابزارهای تشخیص

این‌ها در نسخهٔ فروشگاه خاموش‌اند:

```
flutter build apk --release --dart-define=CARET_TRACE=true
adb logcat -s flutter | grep -E '\[caret\]|\[redial\]'
```

`[caret]` هر حرکت نشانگر و `[redial]` هر تصمیم «تماس مجدد خودکار» را چاپ می‌کند.

بخش ۳۰. شبکه و گزارش خطا

۳۰.۱ لایهٔ شبکه وجود ندارد

برنامه **هیچ سرور، REST یا WebSocket ندارد.** همهٔ داده روی گوشی است و تنها «شبکه»، پیامک و تماس اپراتور است که اندروید واسطه‌اش است. پس به‌جای لایهٔ شبکه، **مرزها** مهم هستند.

```
BLoC ─▶ Repository و Service
          │        │        │          │
          ▼        ▼        ▼          ▼
        SQLite   کلید و مقدار  پلاگین‌ها   کانال‌ها
```

مرز	چه چیزی	قانون
SQLite	`communication_app.db` و `secure.db`	بخش ۶
کانال‌ها	Kotlin	بخش ۷. کدهای خطا قرارداد هستند
پلاگین‌ها	`flutter_contacts`، `permission_handler` و بقیه	هیچ پلاگینی مجوز نمی‌خواهد
کلید و مقدار	`SharedPreferences`، `flutter_secure_storage`	PIN و کد فعال‌سازی فقط در دومی

کدهای خطای ارسال پیامک قرارداد هستند و هر دو طرف با هم تغییر می‌کنند: `NO_SIM_CARD`، `NO_SERVICE`، `PERMISSION_DENIED`، `SMS_SEND_FAILED`، `HIDDEN_NUMBER` و `NUMBER_NOT_ALLOWED`.

۳۰.۲ تنها اتصال‌های خروجی

اتصال	کجا	چه زمانی
گزارش خطا	`CrashReporting`	با DSN کامپایل‌شده **و** روشن کردن کاربر
پیش‌نمایش لینک	`LinkPreviewService`	صفحهٔ لینک داخل پیام، مطابق تنظیم (بخش ۱۰)
جستجوی وب	`url_launcher`	انتخاب کاربر، با مرورگر

نسخهٔ سازمانی با این قول تحویل داده می‌شود که هیچ چیز روی آن با سرور شخص ثالث حرف نمی‌زند. پس پیش از اضافه کردن هر اتصال تازه، به نسخهٔ سازمانی فکر کنید.

۳۰.۳ تاب‌آوری

- شروع برنامه منتظر مرز خراب نمی‌ماند. اعلان بهترین تلاش است و دروازه‌ها صفحهٔ خالی می‌کشند.
- مجوز ردشده بقیهٔ برنامه را از کار نمی‌اندازد.
- پیامک تکراری با ایندکس یکتا جمع می‌شود.
- کار سنگین بومی (ICC و خواندن دفتر تماس) روی executor خودش است.

۳۰.۴ اگر روزی سرور اضافه شد

یک لایهٔ `core/network/` بسازید (client، نتیجهٔ تایپ‌دار، نگاشت DTO به مدل)، آن را به repository تزریق کنید و BLoC را عوض نکنید. ویجت هرگز شبکه را صدا نمی‌زند.

۳۰.۵ گزارش خطا

`core/services/crash_reporting.dart` کل `main` را می‌پیچد.

- **خاموش است** مگر DSN با `--dart-define=SENTRY_DSN=…` کامپایل شده باشد **و** کاربر در تنظیمات «تشخیص خطا» را روشن کند (پیش‌فرض خاموش، اثر از اجرای بعد).
- **در نسخهٔ سازمانی کامپایل نمی‌شود** (`CrashReporting.isAvailable`).
- `sentry` **خالص Dart است، نه** `sentry_flutter`**،** چون آن یک پلاگین Gradle با AGP 7.4.2 دارد که این پروژه نمی‌تواند resolve کند و `assembleDebug` می‌شکند. نتیجه این است که خطای Kotlin گزارش نمی‌شود (K5).
  `FlutterError.onError` و `PlatformDispatcher.onError` دستی وصل شده‌اند.

چیزی از پیام بیرون نمی‌رود

> این برنامهٔ پیش‌فرض پیامک است. هیچ‌کدام از این‌ها را شل نکنید.

- `sendDefaultPii` خاموش است.
- عکس صفحه و سلسله‌مراتب نما فرستاده نمی‌شود.
- breadcrumbهای `console`، `http` و `query` کامل حذف می‌شوند، چون `debugPrint` در این برنامه متن پیام را هم چاپ می‌کند.
- `data` در breadcrumb خالی می‌شود.
- هر رشتهٔ باقی‌مانده از یک پاک‌کننده می‌گذرد که هر رشتهٔ ۴ رقم یا بیشتر را حذف می‌کند.

بخش ۳۱. آزمون‌ها

۳۱.۱ چهار لایه

لایه	کجا	تعداد تقریبی	چطور اجرا می‌شود
Dart: منطق و BLoC	`test/unit/` (۵۷ فایل)	حدود ۹۰۰ آزمون با لایهٔ بعدی	`flutter test`
Dart: ویجت	`test/widget/` (۳۲ فایل)		`flutter test`
JVM: کد Kotlin خالص	`android/app/src/test/`	۱۴۲	`cd android && ./gradlew :app:testCommercialDebugUnitTest`
ابزار و پنل	`tools/keybank/`	۲۵ و ۱۳	`android/gradlew -p tools/keybank test :panel:test`
دو گوشی واقعی	`docs/FINAL_DEVICE_TESTS.md`	چک‌لیست	دستی

یک فایل جدا:

```
flutter test test/unit/phone_normalizer_test.dart
```

> روی ویندوز `gradlew` اگر `java` در PATH نباشد به `JAVA_HOME` نیاز دارد، مثلاً JBR همراه Android Studio.

۳۱.۲ الگوهای آزمون Dart

- **BLoC:** `bloc_test` و `mocktail`. BLoCهایی که وابستگی می‌گیرند با mock آزموده می‌شوند. `SettingsBloc` با `SharedPreferences.setMockInitialValues({})`.
- **Repository با طرح واقعی روی پایگاه‌دادهٔ حافظه:**

```
sqfliteFfiInit();
databaseFactory = databaseFactoryFfi;
DatabaseHelper.databasePathOverride = inMemoryDatabasePath;
DatabaseHelper.resetForTesting(); // بین آزمون‌ها
```

نمونهٔ کامل: `test/unit/repositories_test.dart`.

- **ویجت:** برای ویجت‌های استخراج‌شده. فونت تست پیش‌فرض، فارسی را مربع می‌کشد.
- **موتور پیام رمز:** `secure_messenger_test.dart` سه موتور را روی یک رادیوی شبیه‌سازی‌شده اجرا می‌کند. بهترین جا برای دیدن کل پروتکل است.

۳۱.۳ آزمون‌های قفل‌کننده

این آزمون‌ها رفتار نمی‌سنجند، بلکه **یک قرارداد را قفل می‌کنند**. شکستن‌شان یعنی چیزی عوض شده که نباید می‌شد.

آزمون	چه چیزی را قفل می‌کند	اگر شکست
`GoldenVectorTest`	کل پروتکل رمز، بایت به بایت	`Wire.VERSION` تازه، **نه** مقدار تازه
`SealedBoxTest`	قالب مهر	همان
`KeyBankTest`	`Canon`، مشتق‌گیری کلید عضو، قالب فایل	تغییر، کلید همهٔ اعضا را عوض می‌کند
`PayloadTest`	جدول فارسی	نوع تازه
`blake3_test.dart`	۱۹ بردار BLAKE3	پیاده‌سازی خراب است
`ActivationTest`، `activation_code_test.dart`	کدهای فعال‌سازی، با ابزار یکی	هر دو طرف با هم
`VaultCryptoTest`، `BioSealTest`	قالب فایل vault	
`KnownAnswerTest`، `KeyBankKnownAnswerTest`	بردارهای NIST و RFC	کتابخانه خراب یا اشتباه استفاده شده
`CoverTextTest`	واژه‌نامه و واژه‌های ممنوع	واژه‌نامه را دستی ویرایش نکنید
`surface_roles_test.dart`	رابطهٔ سطح‌ها	
`app_bloc_providers_test.dart`	ترتیب ثبت BLoC	
`call_over_app_lock_test.dart`	تماس و پیوند عمیق روی قفل	

۳۱.۴ چه چیزی را خودکار نمی‌شود آزمود

مسیر تماس (Telecom، نقش تلفن، صفحهٔ قفل) و رفتار اپراتور با آزمون خودکار قابل بررسی نیست. پس:

- منطق خالص تا جایی که بشود در Kotlin خالص استخراج می‌شود: `CallRules`، `NumberListRules`، `ReminderTiming`، `SimRegistry` با `IccMatchTest` و `SecureSmsInboxArrivalTest`.
- بقیه روی گوشی امتحان می‌شود.

۳۱.۵ فهرست آزمون دو گوشی

هر قابلیتی که فقط با **دو گوشی واقعی و شبکهٔ واقعی** قابل امتحان است، در همان کامیت به `docs/FINAL_DEVICE_TESTS.md` اضافه می‌شود. مورد قبول‌شده تیک و تاریخ می‌خورد و مورد رد شده به `KNOWN_ISSUES.md` می‌رود. شبیه‌ساز رفتار اپراتور، دو سیم‌کارت، پیامک چندبخشی و برنامه‌های سازندهٔ گوشی را نشان نمی‌دهد.

هر پیامک واقعی هزینه دارد. پیش از آزمون، با مسئول پروژه هماهنگ کنید که از کدام شماره استفاده کنید.

۳۱.۶ آزمون شکننده

`pin_error_text_test.dart` در اجرای کامل گاهی می‌شکند و تنها قبول می‌شود (K9). اگر فقط همین یکی شکست، جدا اجرایش کنید.

۳۱.۷ پیش از کامیت

```
flutter analyze      # باید صفر مشکل باشد
dart format lib test
flutter test
```

بخش ۳۲. ساخت و انتشار

۳۲.۱ ساخت نسخهٔ نهایی

```
scripts/release_build.sh                       # تجاری، برای فروشگاه
scripts/release_build.sh --flavor organization  # سازمانی، فقط تحویل مستقیم
```

در PowerShell: `.\scripts\release_build.ps1 [-Flavor organization]`. خروجی در `build/app/outputs/flutter-apk/app-<flavor>-release.apk` است.

> APK سازمانی **هرگز** در فروشگاهی آپلود نمی‌شود.

۳۲.۲ چرا اسکریپت لازم است

`pubspec.yaml` می‌گوید `version: 1.0.0+1`. آن `+1` فقط برای توسعهٔ محلی است. هر آپلود versionCode بزرگ‌تری می‌خواهد و عدد دستی یک بار فراموش می‌شود. پس:

مقدار	منبع
`versionName`	تگ روی HEAD (`v1.2.3` به `1.2.3`)، وگرنه از `pubspec` با هشدار
`versionCode`	`git rev-list --count HEAD`

شمارش کامیت روی لپ‌تاپ و runner یکسان است، حالت جدا نمی‌خواهد و فقط زیاد می‌شود.

۳۲.۳ بررسی‌های اسکریپت

بررسی	چه کاری
درخت کثیف	هشدار، چون APK با هیچ کامیتی نمی‌خواند
manifest	`aapt2 dump badging`: بسته `ir.hamrasan.app` و versionCode و versionName دقیقاً مطابق فلیور
مرجع آزمایشی	اگر `libapp.so` کلید مرجع آزمایشی را داشته باشد، رد می‌شود
امضا	`apksigner verify --print-certs`: اگر `Android Debug` باشد یا `certificate DN:` نباشد، رد می‌شود

این بررسی‌ها از دل خطا آمده‌اند. `apksigner` بدون java فقط یک پیام خطا چاپ می‌کرد و اسکریپت در آن دنبال `Android Debug` می‌گشت و چون پیدا نمی‌کرد، امضا را **تأیید** می‌کرد. حالا نبودن DN خطاست.

`APPLICATION_ID` و `ORGANIZATION_VERSION_CODE_BASE` در اسکریپت **آینهٔ** `android/app/build.gradle.kts` هستند.

۳۲.۴ امضا

- `android/upload-keystore.jks` و `android/key.properties` در git نیستند.
- `enableV1Signing=false`، و V2 و V3 روشن است.
- `build.gradle.kts` اگر `key.properties` نباشد به کلید debug برمی‌گردد و همین را اسکریپت می‌گیرد.

> بازار و مایکت معادل Play App Signing ندارند. **از دست رفتن keystore یعنی دیگر نمی‌شود برای این بسته به‌روزرسانی داد.** تنها راه، بستهٔ جدید و از دست دادن همهٔ نصب‌هاست.

پشتیبان

`scripts/backup_keystore.{ps1,sh}` آرشیو AES-256-CBC با PBKDF2 ششصدهزار دور می‌سازد.

- پیش از بسته‌بندی، keystore با رمزها باز می‌شود.
- بعد از بسته‌بندی، آرشیو باز و بایت‌به‌بایت مقایسه می‌شود.
- بازیابی: `scripts/backup_keystore.sh --restore` .
- رمز آرشیو نه در مخزن است و نه در خود آرشیو.

۳۲.۵ حجم و معماری

ABIهای دیگر عمداً حذف شده‌اند. دایرکتوری خالی `armeabi-v7a` باعث می‌شد نصب روی گوشی ۳۲ بیتی موفق باشد و با `UnsatisfiedLinkError` در اولین اجرا بمیرد.

`isMinifyEnabled` و `isShrinkResources` روشن است و keep-ruleهای اختصاصی دارد. `mapping.txt` را نگه دارید، چون بدون آن stack trace خوانده نمی‌شود.

۳۲.۶ حریم خصوصی

- **`allowBackup=false`** و `dataExtractionRules` .
- **`PrivacyPolicyPage`** متن مرجع سیاست حریم خصوصی است و یک کارت برای هر گروه مجوز دارد.
- **نسخهٔ وب** در `docs/privacy/index.html` است و از GitHub Pages سرو می‌شود.
- **مجوز تازه در manifest، در همان کامیت کارت می‌گیرد.** این اولین چیزی است که بازبین می‌بیند.

۳۲.۷ بازبینی فروشگاه

ریسک اصلی رد شدن اولین ارسال **فهرست مجوزهاست، نه باگ.** در توضیحات فروشگاه باید **صریح** نوشته شود که برنامه جایگزین پیامک و تلفن پیش‌فرض است. بدون آن، این فهرست شبیه جاسوس‌افزار خوانده می‌شود. جدول کامل مجوزها و توجیهشان در `docs/publishing/store-release.md` است.

۳۲.۸ مسیر انتشار

بازار (`pishkhan.cafebazaar.ir`): ثبت‌نام و احراز هویت، ساخت برنامه با `ir.hamrasan.app` ، آپلود APK، اطلاعات فروشگاهی و لینک حریم خصوصی، و ارسال برای بازبینی.

مایکت (`developer.myket.ir`): ثبت‌نام، رزرو نام بسته، آپلود با **همان** keystore، اطلاعات فروشگاهی و بازبینی.

دارایی‌ها: آیکون ۵۱۲×۵۱۲ از `Hamresan-Logo.svg` ، حداقل ۴ تصویر، توضیح کوتاه و بلند فارسی، دسته‌بندی، ردهٔ سنی و لینک حریم خصوصی.

۳۲.۹ وضعیت زنده

`docs/publishing/store-release.md` **زنده است.** هر تغییر در وضعیت انتشار در همان کامیت آنجا ثبت می‌شود. طرح secret، CI/CDها و endpointها هم آنجاست. هنوز حسابی ساخته نشده و چیزی منتشر نشده است.

۳۲.۱۰ ثبت نسخهٔ سازمانی

هر APK سازمانی که تحویل داده می‌شود در `docs/architecture/editions.md` (جدول «Organization APK history») ثبت می‌شود.

۳۲.۱۱ پوشهٔ تحویل

`docs/mentor/Hamresan/` در git نیست و این ساختار را دارد:

```
1-App-Android/      دو APK
2-Panel-Windows/    MSI پنل
3-Hamresan-Authority/  مرجع و فایل‌های کلید
4-Docs/             راهنماها به شکل PDF
```

keystore و `key.properties` هرگز در این پوشه قرار نمی‌گیرند.

۳۲.۱۲ مجوزهای باز

موضوع	وضعیت
`PROCESS_OUTGOING_CALLS`	استفاده‌ای ندارد و باید بعد از تأیید روی گوشی حذف شود
`armeabi-v7a`	تصمیم محصولی: پذیرفتن محدودیت یا APK دوم
کپی USB از پشتیبان keystore	هنوز انجام نشده است

بخش ۳۳. قوانین طلایی و چک‌لیست تغییر

اگر فقط یک بخش از این سند را حفظ می‌کنید، این بخش باشد. هر قانون از یک خطای واقعی آمده است.

۳۳.۱ قوانین سراسری

۱. **هیچ پلاگینی مجوز runtime نمی‌خواهد.** همه از `PermissionGate` می‌گذرد. پلاگین‌هایی که یک `MethodChannel.Result` نگه می‌دارند با دو فراخوانی همزمان برنامه را می‌کشند (`Reply already submitted`). به همین دلیل `call_log`، `another_telephony`، `image_picker` و `image_cropper` نداریم.

۲. **هیچ رازی در پایگاه‌دادهٔ اصلی نوشته نمی‌شود.** Kotlin آن را بدون کلید در حالت مرده باز می‌کند. راز فقط در `secure.db` است و `sqflite_sqlcipher` فقط در `lib/features/secure/` import می‌شود.

۳. **پیامک رمز (`#E:`) هرگز وارد `content://sms` یا `messages` نمی‌شود.** متن رمز در `secure_queue` پارک می‌شود و ارسال با `private: true` است.

۴. **شمارهٔ مخفی بیرون از بخش امن ردی نمی‌گذارد.**

۵. **هر batch به contacts provider زیر ۵۰۰ عملیات تکه می‌شود.** بالاتر از سقف هیچ‌چیز نوشته نمی‌شود و از راه `flutter_contacts` فرایند را می‌کشد.

۶. **BLoC فقط در `AppBlocProviders` ساخته می‌شود.** داخل صفحه `BlocProvider` نسازید.

۷. **ویجت به SQLite، پلاگین یا کانال دست نمی‌زند.** جهت: Repository ← BLoC ← Widget مرز.

۸. **هر شماره از `PhoneNormalizer` می‌گذرد.** خودتان رقم جدا نکنید. این خطا مسدودی را کاملاً از کار انداخته بود.

۹. **`messages` با `ConflictAlgorithm.ignore` درج می‌شود.**

۱۰. **هر صفحهٔ تازه RTL است.**

۱۱. **برنامه `tel:` اجرا نمی‌کند،** چون خودش تلفن پیش‌فرض است.

۱۲. **Migration نداریم.** هنوز چیزی منتشر نشده است. طرح را در `_createDB` یا `_onCreate` عوض کنید و گوشی آزمون را دوباره نصب کنید. اولین تغییر بعد از انتشار، `databaseVersion` را بالا می‌برد و `onUpgrade` می‌سازد.

۱۳. **لینک عمیق روی قفل باز نمی‌شود** (`AppLock.whenDown`).

۱۴. **رفتن به برنامهٔ دیگر برای نتیجه داخل `AppHandoff.run` است.**

۱۵. **اسپینر سر راه شروع برنامه نمی‌گذاریم.**

۱۶. **فهرست داده‌محور lazy است** و صفحه‌به‌صفحه خوانده می‌شود.

۱۷. **چیزی از پیام به گزارش خطا نمی‌رود.**

۱۸. **نگهبان‌های state را به‌عنوان اضافه حذف نکنید** (بخش ۵).

۳۳.۲ آینه‌ها

این فایل‌ها باید **با هم** عوض شوند. ناهماهنگی با خطا نمی‌ترکد، بلکه بی‌صدا رفتار دیگری می‌کند.

طرف اول	طرف دوم
`ScheduledSmsWorker.kt`	`scheduled_message_model.dart`

طرف اول	طرف دوم
`TemplateWire.kt`	`message_template_model.dart`، `template_wire.dart`، `TinyImage` (`sigil`، شکل `isWire`، `label`)
`BlockedNumbers.kt`	`PhoneNormalizer`
`SecureSmsInbox.kt`	`DatabaseHelper._createSecureQueueTable`، `AppConstants.secureQueueTable`
`hidden/SealedInbox.kt`	`DatabaseHelper._createSealedQueueTable`، `AppConstants.sealedQueueTable`
`SmsNotificationActionReceiver.chosenThreadSim`	`ThreadSimRepository`
`SimRegistry.setAppDefaults`، `APP_ASK`	`SimService.setAppDefault`، `appAsk`
`SimRegistry.routeFor`	`SimService.callRoute` فقط می‌پرسد
`SystemBlockList.kt`	`system_block_list.dart`. `remove` با پاسخ `-1` یعنی «اجازهٔ نوشتن نداریم»
نام فلیورها و پایهٔ versionCode در `build.gradle.kts`	`AppEdition`، `scripts/release_build.{sh,ps1}`
`ActivationCode`، `ActivationTarget`	تولیدکنندهٔ موجود ویندوزی، `tools/keybank/src/main/kotlin/Activation.kt`
`Wire.PREFIX`	`SmsCryptoService.wirePrefix`
`CallRules.normalizePrefix`	`CallBlockPrefixes.normalize`
`NumberListRules.normalizePattern`	`NumberLists.normalizePattern` و کلیدهای کانال و حالت
`NotificationPrivacy.PIN_KEY`	`AppConstants.pinKey`
`MemberCert.keyIdOf`	`certKeyIdFor` در `secure_identities.dart`، هر دو با یک بردار

۳۳.۳ چک‌لیست هر تغییر

پیش از شروع:

- [ ] یادداشت همان قسمت در `docs/architecture/` را بخوان. این یادداشت‌ها اختیاری نیستند.
- [ ] بخش همان قسمت در همین سند را بخوان.

پیش از کامیت:

- [ ] `flutter analyze`: صفر مشکل.
- [ ] `dart format lib test`: بدون تفاوت.
- [ ] تغییر رفتار، تا جایی که می‌شود، آزمون دارد.
- [ ] تغییر منطق کسب‌وکار، یادداشت در `DESIGN_DECISIONS.md` یا توضیح PR دارد.
- [ ] یادداشت قسمت در **همان کامیت** به‌روز است.
- [ ] کاری که فقط با دو گوشی معلوم می‌شود در `FINAL_DEVICE_TESTS.md` آمده است.
- [ ] مجوز تازه کارت در `PrivacyPolicyPage` و ردیف در `store-release.md` دارد.

- [ ] آینه‌ها با هم عوض شده‌اند.

۳۳.۴ کد و فایل

- یک امکان یعنی یک پوشه زیر `lib/features/` با `bloc/`، `models/`، `repositories/`، `screens/` و در صورت نیاز `services/`.
- فایل بزرگ‌تر از حدود ۴۰۰ خط بو می‌دهد. ویجت نمایشی را به `widgets/` کنار صفحه ببرید.
- مدل‌ها `Equatable` و تغییرناپذیرند، با `copyWith`.
- لایهٔ انتزاع اضافه (use-case، DI container) نسازید. سادگی از خلوص مهم‌تر است.
- وابستگی را با سازنده تزریق کنید تا آزمون‌پذیر باشد.
- بازسازی با استخراج امن ویجت انجام می‌شود، نه بازنویسی (D10).

۳۳.۵ کامیت

- کامیت‌های کوچک و متمرکز، با پیشوندهای `feat:`، `fix:`، `refactor:`، `docs:` و `chore:`.
- تغییرهای پرریسک (مسیر پیامک، طرح پایگاه‌داده، ارتقای وابستگی) PR خودشان و یادداشت آزمون دستی دارند.
- PR بازسازی می‌گوید چه چیزی جابه‌جا شده، رفتار تغییر نکرده و نتیجهٔ `flutter analyze` چیست.
- هر تغییر versionCode را زیاد می‌کند، پس لازم نیست عددی را دستی عوض کنید.

۳۳.۶ اگر چیزی شکست

نشانه	اول کجا را ببینید
`Reply already submitted`	پلاگینی که مجوز خواسته است یا `Result` دوم
پیامک هست ولی در صندوق نیست	آینه و tombstone (بخش ۸)
شماره با فرمت دیگر پیدا نمی‌شود	`PhoneNormalizer` و ستون `normalized`
نام مخاطب تازه به همه‌جا نمی‌رسد	سیگنال revision (بخش ۱۶)
صفحهٔ تماس روی قفل نمی‌آید	`CallUiCoordinator` و دو flag پنجره (بخش ۱۳)
«پیام رمز» باز نمی‌شود	`sm_inbox` و نوع نشست (بخش ۲۴)
APK نسخهٔ دیگر نصب نمی‌شود	بازهٔ versionCode (بخش ۲۰)
`INSTALL_FAILED_UPDATE_INCOMPATIBLE`	امضای متفاوت. نصب debug را با release قاطی نکنید

بخش ۳۴. مشکلات شناخته‌شده و کارهای باقی

وضعیت به تاریخ ۱۴۰۵/۰۸/۱۳. منبع زندهٔ این بخش `docs/KNOWN_ISSUES.md` است.

۳۴.۱ مشکلات باز

شناسه	موضوع	جزئیات
K1	**تأییدنشده بین دو گوشی واقعی**	متن پوششی، کل APK سازمانی، تولیدکنندهٔ ویندوزی موجود برای کد فعال‌سازی، ردیف‌های ساخته‌شده در ۱۴۰۵/۰۸/۱۱ تا ۱۳ (۵، ۱۰، ۲۳، ۳۴، ۳۶، ۳۹، ۴۲) و عکس مخاطب از دوربین. ذخیره در حساب Google و Google Drive هم امتحان نشده است
K2	**ردیف‌های ماتریس که انجام نشده‌اند**	جدول پایین
K3	**پیامک چندبخشی بلند ممکن است گم شود**	MIUI یک بار درخواست بلند را انداخت (`checkSmsCmd … smsBlocked`). درخواست گم‌شده با تکرار ۱۵ دقیقه‌ای و قانون INIT همزمان برمی‌گردد، ولی بخش گم‌شدهٔ پیام برنمی‌گردد
K4	**ساخت کد فعال‌سازی از APK ممکن است**	مشتق‌گیری و شناسه‌ها در APK هستند و سرور و لغو نداریم. به‌خاطر سازگاری پذیرفته شده است
K5	**خطای Kotlin گزارش نمی‌شود**	`sentry` خالص Dart
K6	**شمارهٔ غیرایرانی ضعیف نرمال می‌شود**	دو فرمت یک شمارهٔ خارجی شاید یکی نشوند
K7	**لایهٔ ترجمه نداریم**	متن‌ها مستقیم فارسی هستند
K8	**وابستگی‌های عقب‌مانده**	`flutter_secure_storage` 9، `flutter_local_notifications` 18، `flutter_contacts` 1. نسخهٔ ۲ مورد آخر کل API را عوض می‌کند و شاخه و آزمون جدا می‌خواهد
K9	**یک آزمون حساس به زمان**	`pin_error_text_test.dart`

۳۴.۲ پذیرفته‌شده در طراحی

- **متن پوششی کلمه است، نه جمله.** گرامر واقعی هزینهٔ پیامک را چند برابر می‌کند.
- **دست‌دهی پوشانده نمی‌شود.**
- **مسیر `ConnectionService` خفته است** (`call/CallConnectionService.kt`). داربست VoIP است و تماس اپراتور از `CallInCallService` می‌رود.
- **حذف عضو چیزی را لغو نمی‌کند.** لغو در طراحی نیست.
- **دو مرجع روی یک گوشی:** با اولی جواب داده می‌شود.
- **بدون نقش پیامک،** تاریخچهٔ مخفی‌شده در provider سیستم می‌ماند.

۳۴.۳ ردیف‌های ماتریس

ردیف	موضوع	وضعیت
۲	ورود با الگو	کنار گذاشته شد
۴	میانبر با کلید فیزیکی	لغو: برنامهٔ عادی به کلید سخت‌افزاری دسترسی ندارد
۱۲	رمز کردن کل پایگاه‌دادهٔ اصلی	لغو (بخش ۲۶)
۲۴	ستاره روی پیام رمز	کنار گذاشته شد
۲۵	مکان و پاک‌سازی با پیامک	لغو
۳۷	پروفایل مخاطب و گروه	کنار گذاشته شد
۳۸	گفتن نام تماس‌گیرنده	لغو: صدای فارسی روی گوشی‌ها نبود و صدای اردو و عربی قابل فهم نبود
۴۱	حالت‌های کاری	کنار گذاشته شد
۷، ۸، ۹، ۴۳	پشتیبان، انتقال پشتیبان، هدایت خودکار پیامک، تاریخچهٔ دائمی مخاطب	ساخته نشده و تصمیم نگرفته

۳۴.۴ کارهای باقی

کار	توضیح
آزمون دو گوشی	چک‌لیست `docs/FINAL_DEVICE_TESTS.md`
راهنمای کاربر	بعد از آزمون‌ها به‌روز می‌شود
فروشگاه	حساب‌ها، تصمیم `PROCESS_OUTGOING_CALLS` و `armeabi-v7a`، ارسال اول
CI/CD	طرح در `store-release.md` آماده است و نوشته نشده
پشتیبان keystore	کپی روی USB

پیوست الف: فهرست فایل‌به‌فایل کد

همهٔ فایل‌های Dart و Kotlin پروژه در این پیوست آمده‌اند. ستون «خط» اندازهٔ فایل را نشان می‌دهد تا بدانید با چه چیزی روبه‌رو هستید. مسیرها نسبت به ریشهٔ هر جدول هستند.

نقطهٔ شروع

فایل	خط	کار
`main.dart`	111	نقطهٔ شروع: `verifyBuildFlavor`، گزارش خطا و بالا آوردن برنامه

lib/core: هستهٔ مشترک

ریشه: `lib/core`

فایل	خط	کار
`bloc_providers/app_bloc_providers.dart`	171	ساخت همهٔ BLoCها با ترتیب درست
`constants/app_constants.dart`	156	نام جدول‌ها، کلیدها و عددهای ثابت
`database/database_helper.dart`	648	طرح کامل پایگاه‌دادهٔ اصلی
`edition/activation_code.dart`	166	مشتق‌گیری و بررسی کد فعال‌سازی
`edition/app_edition.dart`	79	نسخهٔ زمان کامپایل
`navigation/app_route_observer.dart`	8	خبر تغییر route به صفحه‌ها
`navigation/call_ui_coordinator.dart`	439	آوردن صفحهٔ تماس روی هر چیزی
`navigation/main_navigation.dart`	592	پوستهٔ چهار تب و انتخاب تب شروع
`navigation/return_to_call_bar.dart`	200	نوار سبز «بازگشت به تماس»
`navigation/widgets/message_nav_icon.dart`	22	نماد پیام‌ها با شمارهٔ نخوانده
`services/app_handoff.dart`	78	معافیت رفتن به برنامهٔ دیگر برای نتیجه
`services/composer_draft_store.dart`	127	متن نیمه‌تمام composer
`services/crash_reporting.dart`	180	گزارش خطا بدون دادهٔ پیام
`services/deep_link_service.dart`	203	اجرای intent و اعلان، پشت قفل
`services/device_sync_queue.dart`	40	سریال کردن خواندن‌های سنگین گوشی
`services/image_picker_service.dart`	105	انتخاب عکس بدون پلاگین مجوزخواه
`services/location_service.dart`	116	یک بار خواندن مکان برای پیام
`services/permission_service.dart`	74	مدیریت مجوزها
`sim/sim_bloc.dart`	84	وضعیت سیم‌کارت‌ها
`sim/sim_call.dart`	172	تنها راه برقراری تماس در برنامه
`sim/sim_card.dart`	159	مدل سیم و پیش‌فرض‌ها

فایل	خط	کار
`sim/sim_service.dart`	432	پل به `SimRegistry`
`sim/widgets/call_icon_button.dart`	54	دکمهٔ تماس با انتخاب سیم با لمس طولانی
`sim/widgets/sim_picker.dart`	386	نشان، چیپ و انتخابگر سیم
`theme/app_colors.dart`	66	رنگ‌های برند
`theme/app_dimensions.dart`	45	فاصله، گردی و اندازه
`theme/app_theme.dart`	397	تم روشن و تاریک
`theme/surface_roles.dart`	78	نقش‌های سطح و گردی گروه
`theme/theme_bloc.dart`	79	انتخاب حالت تم
`utils/blake3.dart`	299	BLAKE3 خالص Dart
`utils/calendar_type.dart`	5	جلالی یا میلادی
`utils/contact_name_style.dart`	84	«قالب نام» و ترتیب
`utils/date_formatter.dart`	122	قالب تاریخ با رقم فارسی
`utils/jalali_date.dart`	142	تبدیل به جلالی بدون وابستگی
`utils/message_text_scale.dart`	93	«اندازه متن پیام»
`utils/persian_alphabet.dart`	130	ترتیب الفبای فارسی
`utils/persian_utils.dart`	89	رقم فارسی و قالب نمایش شماره
`utils/phone_normalizer.dart`	136	شکل استاندارد شماره و شناسهٔ گفتگو
`utils/script_direction.dart`	96	جهت متن از روی حروف
`utils/search_text.dart`	395	جستجوی تاشده، شکل‌های شماره و T9
`utils/sms_address.dart`	67	آیا می‌شود به این فرستنده پاسخ داد
`utils/ussd_code.dart`	136	پیدا کردن کد USSD در پیام
`widgets/app_lock_wrapper.dart`	386	«قفل خودکار»، `AppLock` و `RelockScreen`
`widgets/avatar_widget.dart`	116	آواتار با حرف اول
`widgets/caret_line_fix.dart`	557	جای درست نشانگر در متن آمیخته
`widgets/contact_index_list.dart`	262	بخش‌بندی الفبایی و نوار پرش
`widgets/contact_numbers_line.dart`	80	خط شماره زیر نام
`widgets/default_app_gate.dart`	572	دروازهٔ نقش پیامک و تلفن
`widgets/google_list.dart`	449	فهرست کارت‌دار و نسخهٔ lazy
`widgets/highlighted_phone.dart`	74	شماره با بخش منطبق برجسته
`widgets/home_search_header.dart`	89	نوار جستجوی تب‌ها
`widgets/infinite_scroll.dart`	62	محرک صفحهٔ بعدی
`widgets/jalali_date_picker.dart`	199	انتخاب تاریخ روی تقویم انتخابی

فایل	خط	کار
`widgets/lazy_contact_avatar.dart`	135	عکس مخاطب با LRU
`widgets/permission_gate.dart`	285	تنها درخواست‌کنندهٔ مجوز پیش از برنامه
`widgets/phone_contact_avatar.dart`	98	آواتار از روی شماره
`widgets/rtl_app_bar.dart`	87	سربرگ راست‌چین
`widgets/selection_app_bar.dart`	78	سربرگ و تیک حالت انتخاب
`widgets/two_column_board.dart`	78	تختهٔ دوستونی lazy
`widgets/undo_snack_bar.dart`	194	نوار تأیید با واگرد و شمارش معکوس

lib/features/authentication

ریشه: `lib/features/authentication`

فایل	خط	کار
`bloc/auth_bloc.dart`	217	وضعیت PIN برنامه
`bloc/auth_event.dart`	85	رویدادهای `auth_bloc`
`bloc/auth_state.dart`	72	حالت‌های `auth_bloc`
`models/auth_type.dart`	22	PIN یا بدون قفل
`models/pin_policy.dart`	15	طول PIN برنامه و بخش امن
`repositories/auth_repository.dart`	312	PIN، کد بازیابی، جلسهٔ باز
`repositories/pin_attempt_limiter.dart`	93	بودجهٔ تلاش و قفل زمانی
`screens/auth_choice_screen.dart`	88	ساختن PIN یا ادامه بدون آن
`screens/auth_wrapper_screen.dart`	66	مسیریابی از روی `AuthBloc`
`screens/pin_auth_screen.dart`	106	ورود PIN هنگام شروع
`screens/pin_setup_screen.dart`	145	ساختن و تغییر PIN
`screens/recovery_code_screen.dart`	270	نمایش و ورود کد بازیابی
`screens/verify_pin_screen.dart`	130	پرسیدن PIN فعلی
`screens/widgets/pin_pad.dart`	418	نقطه‌ها، صفحه‌کلید، خطا و دکمهٔ اثر انگشت
`services/app_lock_platform.dart`	169	پل کانال `app_lock`
`services/biometric_unlock.dart`	210	روشن کردن و باز کردن با اثر انگشت
`services/notification_privacy_sync.dart`	27	هماهنگی تنظیم پنهان بودن اعلان با Kotlin

lib/features/call_history

ریشه: `lib/features/call_history`

فایل	خط	کار
`bloc/call_log_bloc.dart`	335	حالت «اخیر»

فایل	خط	کار
`bloc/call_log_event.dart`	69	رویدادهای `call_log_bloc`
`bloc/call_log_state.dart`	37	حالت‌های `call_log_bloc`
`models/call_log_model.dart`	94	مدل یک تماس
`repositories/call_log_repository.dart`	158	جدول `call_logs`
`screens/call_history_screen.dart`	392	تب «اخیر»
`screens/widgets/call_detail_sheet.dart`	715	شیت جزئیات تماس
`screens/widgets/call_log_tile.dart`	647	ردیف تماس
`screens/widgets/call_reminder_sheet.dart`	245	ساختن «یادآور» و ورودی آن
`services/call_log_service.dart`	471	خواندن، sync و نام‌گذاری
`services/call_reminders.dart`	116	پل یادآورها
`services/native_call_log_service.dart`	145	پل کانال `call_log`

lib/features/contacts

ریشه: `lib/features/contacts`

فایل	خط	کار
`bloc/contact_bloc.dart`	91	حالت دفترچه
`bloc/contact_event.dart`	35	رویدادهای `contact_bloc`
`bloc/contact_state.dart`	36	حالت‌های `contact_bloc`
`models/contact_form_entries.dart`	35	ردیف‌های فرم و برچسب‌هایشان
`models/contact_model.dart`	178	مدل مخاطب و محل ذخیره
`models/contact_save_target.dart`	57	گوشی، سیم یا حساب
`models/phone_match.dart`	61	شمارهٔ منطبق با جستجو
`models/vcf_file.dart`	203	خواندن و نوشتن `.vcf`
`repositories/contact_name_cache.dart`	190	جدول `contact_name_cache`
`repositories/contact_repository.dart`	506	دفترچهٔ گوشی، cache و سیگنال revision
`screens/add_edit_contact_screen.dart`	811	فرم مخاطب
`screens/contact_import_screen.dart`	259	انتخاب از فایل `.vcf`
`screens/contact_labels_screen.dart`	388	«برچسب‌ها»
`screens/contacts_list_screen.dart`	1219	تب «مخاطبین»
`screens/device_contact_detail_screen.dart`	1467	صفحهٔ جزئیات مخاطب
`screens/duplicate_contacts_screen.dart`	209	«مخاطب‌های تکراری»
`screens/photo_crop_screen.dart`	359	«برش عکس»

فایل	خط	کار
`screens/widgets/contact_form_fields.dart`	253	فیلدهای فرم مخاطب
`services/contact_extras_service.dart`	377	تنظیمات هر مخاطب: زنگ، پیام‌رسان و …
`services/contact_file_service.dart`	249	صدور و ورود فایل مخاطب
`services/contact_filters.dart`	191	چیپ‌های برچسب، پیام‌رسان و «رمز»
`services/contact_groups_service.dart`	282	برچسب‌ها و قوانین حساب
`services/contact_link_service.dart`	81	ادغام و جدا کردن
`services/pinned_contacts.dart`	78	مخاطب‌های سنجاق‌شده
`services/sim_contacts_service.dart`	117	دفترچهٔ سیم‌کارت
`widgets/contact_filter_bar.dart`	161	ردیف چیپ‌ها
`widgets/contact_photo_actions.dart`	140	شیت عکس مخاطب
`widgets/contact_picker_sheet.dart`	175	انتخاب مخاطب و شماره
`widgets/group_picker_sheet.dart`	215	تیک زدن برچسب‌ها
`widgets/merge_contacts_dialog.dart`	126	پنجرهٔ «ادغام»
`widgets/phone_number_picker.dart`	183	انتخاب یکی از شماره‌ها
`widgets/save_number_actions.dart`	59	ذخیرهٔ شماره در مخاطب تازه یا موجود
`widgets/save_target_sheet.dart`	68	انتخاب محل ذخیره

lib/features/dialer

ریشه: `lib/features/dialer`

فایل	خط	کار
`bloc/dialer_bloc.dart`	858	صفحه‌کلید، چرخهٔ تماس و تماس مجدد
`bloc/dialer_event.dart`	180	رویدادهای `dialer_bloc`
`bloc/dialer_state.dart`	289	حالت‌های `dialer_bloc`
`models/speed_dial_entry.dart`	61	یک کلید شماره‌گیری سریع
`repositories/speed_dial_repository.dart`	46	جدول `speed_dial`
`screens/dialer_screen.dart`	396	صفحه‌کلید
`screens/in_call_screen.dart`	1073	صفحهٔ حین تماس
`screens/incoming_call_screen.dart`	491	صفحهٔ تماس ورودی
`screens/speed_dial_screen.dart`	159	«شماره‌گیری سریع»
`services/auto_redial_policy.dart`	38	تنظیمات «تماس مجدد خودکار»
`services/call_block_prefixes.dart`	55	پیشوندهای «تماس‌های تبلیغاتی»
`services/native_call_service.dart`	515	پل کانال `call` و رویدادها

فایل	خط	کار
`services/speed_dial_service.dart`	94	cache شماره‌گیری سریع
`services/voicemail.dart`	57	تماس با پیغام‌گیر سیم
`widgets/dialer_bottom_sheet.dart`	85	باز کردن صفحه‌کلید به شکل شیت
`widgets/dialer_widgets.dart`	1152	نمایش شماره، کلیدها، دکمهٔ تماس و پیشنهادها

lib/features/edition

ریشه: `lib/features/edition`

فایل	خط	کار
`bloc/edition_bloc.dart`	227	نسخهٔ مؤثر و فعال‌سازی
`repositories/activation_repository.dart`	89	ذخیره و بررسی دوبارهٔ کد
`screens/inter_org_activation_screen.dart`	121	«نسخه بین‌سازمانی»
`screens/organization_activation_gate.dart`	87	دروازهٔ APK سازمانی
`services/device_identity_service.dart`	36	پل `ANDROID_ID`
`widgets/activation_code_form.dart`	228	فرم کد و کارت کد دستگاه

lib/features/favorites

ریشه: `lib/features/favorites`

فایل	خط	کار
`bloc/favorites_bloc.dart`	62	حالت موردعلاقه‌ها
`bloc/favorites_event.dart`	33	رویدادهای `favorites_bloc`
`bloc/favorites_state.dart`	36	حالت‌های `favorites_bloc`
`models/favorite_model.dart`	80	مدل با کلید `normalized`
`repositories/favorites_repository.dart`	60	جدول `favorites`
`screens/favorites_screen.dart`	512	تب «موردعلاقه‌ها»

lib/features/hidden

ریشه: `lib/features/hidden`

فایل	خط	کار
`bloc/hidden_bloc.dart`	452	دفترچه، برچسب‌ها، جابه‌جایی تاریخچه
`repositories/hidden_contacts_repository.dart`	320	جدول‌های `hc_`
`screens/hidden_calls_screen.dart`	235	«تماس‌های مخفی»
`screens/hidden_contact_edit_screen.dart`	317	ساختن و ویرایش مخاطب مخفی
`screens/hidden_contact_screen.dart`	359	صفحهٔ یک مخاطب مخفی

فایل	خط	کار
`screens/hidden_contacts_screen.dart`	213	«دفترچه مخفی»
`services/hidden_history_mover.dart`	108	بیرون بردن تاریخچه از مسیر عادی
`services/sealed_inbox.dart`	78	خواندن `sealed_queue`
`services/sealing_key.dart`	62	زوج‌کلید مهر بخش
`widgets/hidden_entries.dart`	119	ردیف‌های ورود بالای فهرست‌ها

lib/features/keybank

ریشه: `lib/features/keybank`

فایل	خط	کار
`bloc/key_bank_bloc.dart`	243	فایل کلید و کلیدهای خودمان
`screens/key_bank_screen.dart`	364	«بانک کلید»
`services/key_file_service.dart`	43	پل انتخابگر فایل
`services/trust_anchors.dart`	177	کلید عمومی مراجع مورد اعتماد

lib/features/messages

ریشه: `lib/features/messages`

فایل	خط	کار
`bloc/draft_bloc.dart`	195	پیش‌نویس و دسته
`bloc/draft_event.dart`	113	رویدادهای `draft_bloc`
`bloc/draft_state.dart`	59	حالت‌های `draft_bloc`
`bloc/message_bloc.dart`	1173	صندوق، گفتگو و ارسال
`bloc/message_event.dart`	267	رویدادهای `message_bloc`
`bloc/message_state.dart`	88	حالت‌های `message_bloc`
`bloc/scheduled_bloc.dart`	208	زمان‌بندی‌ها
`bloc/scheduled_event.dart`	102	رویدادهای `scheduled_bloc`
`bloc/scheduled_state.dart`	53	حالت‌های `scheduled_bloc`
`bloc/template_bloc.dart`	81	قالب‌ها
`bloc/template_event.dart`	48	رویدادهای `template_bloc`
`bloc/template_state.dart`	36	حالت‌های `template_bloc`
`models/built_in_templates.dart`	148	قالب‌های داخل برنامه
`models/draft_model.dart`	62	مدل پیش‌نویس
`models/message_category_model.dart`	51	مدل دسته
`models/message_group.dart`	302	گروه و شناسهٔ `g:`

فایل	خط	کار
`models/message_model.dart`	283	مدل پیام و گفتگو
`models/message_search_query.dart`	194	جستجوی متن کامپایل‌شده
`models/message_template_model.dart`	283	مدل قالب و فیلدها
`models/one_time_code.dart`	168	پیدا کردن رمز یک‌بارمصرف
`models/scheduled_message_model.dart`	476	مدل زمان‌بندی، تکرار، jitter
`models/template_wire.dart`	227	قالب پیامک قالب‌های داخلی
`models/tiny_image.dart`	237	رمز کردن و باز کردن «تصویر کوچک»
`repositories/draft_repository.dart`	188	جدول‌های پیش‌نویس و دسته
`repositories/group_repository.dart`	409	جدول‌های پیام گروهی
`repositories/message_repository.dart`	994	جدول `messages` ، صندوق، جستجو و آینه
`repositories/scheduled_message_repository.dart`	181	جدول `scheduled_messages`
`repositories/template_repository.dart`	68	جدول `message_templates`
`repositories/thread_sim_repository.dart`	109	جدول `thread_sim`
`screens/archived_threads_screen.dart`	153	بایگانی
`screens/contact_selector_screen.dart`	1336	«گفتگوی جدید» و انتخاب گیرنده
`screens/conversation_details_screen.dart`	500	«جزئیات» گفتگو
`screens/conversation_screen.dart`	2114	صفحهٔ گفتگو
`screens/draft_editor_screen.dart`	153	ساخت پیش‌نویس
`screens/drafts_list_screen.dart`	543	تختهٔ پیش‌نویس‌ها
`screens/group_details_screen.dart`	414	جزئیات گروه
`screens/message_categories_screen.dart`	362	«دسته‌بندی‌ها»
`screens/messages_list_screen.dart`	1169	تب «پیام‌ها»
`screens/scheduled_messages_screen.dart`	236	فهرست زمان‌بندی‌ها
`screens/spam_and_blocked_screen.dart`	629	«هرزنامه و مسدودشده»
`screens/starred_messages_screen.dart`	213	«ستاره‌دار»
`screens/template_editor_screen.dart`	182	ساخت قالب
`screens/template_fill_screen.dart`	402	پر کردن قالب
`screens/templates_list_screen.dart`	462	انتخاب قالب
`screens/widgets/category_name_dialog.dart`	47	پنجرهٔ نام دسته
`screens/widgets/conversation_app_bars.dart`	238	سربرگ‌های گفتگو
`screens/widgets/conversation_sheets.dart`	201	شیت‌های گفتگو
`screens/widgets/default_sms_banner.dart`	70	بنر پیشنهاد نقش پیامک

فایل	خط	کار
`screens/widgets/emoji_panel.dart`	1946	پنل ایموجی
`screens/widgets/link_preview_card.dart`	152	کارت پیش‌نمایش لینک
`screens/widgets/linkified_text.dart`	288	متن با لینک، شماره و USSD
`screens/widgets/message_action_overlay.dart`	387	بلند شدن حباب و منو
`screens/widgets/message_bubble.dart`	713	حباب، کد یک‌بارمصرف و متن قابل انتخاب
`screens/widgets/message_composer.dart`	468	composer
`screens/widgets/message_list_states.dart`	172	حالت‌های خالی و در حال ورود
`screens/widgets/messages_app_bars.dart`	247	سربرگ‌های sliver صندوق
`screens/widgets/phone_action_sheet.dart`	207	شیت شمارهٔ داخل پیام
`screens/widgets/pinch_text_scale.dart`	353	بزرگ‌نمایی متن با دو انگشت
`screens/widgets/schedule_send_sheet.dart`	720	شیت «زمان‌بندی ارسال»
`screens/widgets/scheduled_bubble.dart`	164	حباب پیام زمان‌بندی‌شده
`screens/widgets/thread_sim_choice.dart`	39	انتخاب سیم برای گفتگو
`screens/widgets/thread_tile.dart`	282	ردیف گفتگو
`screens/widgets/tiny_image_sheet.dart`	159	ساختن «تصویر کوچک»
`screens/widgets/tiny_image_view.dart`	82	کشیدن تصویر کوچک
`screens/widgets/ussd_action_sheet.dart`	104	شیت کد USSD
`services/link_preview_service.dart`	232	خواندن Open Graph
`services/native_scheduled_sms_service.dart`	59	پل آلارم زمان‌بندی
`services/native_sms_service.dart`	449	پل کانال `sms`
`services/notification_service.dart`	74	اعلان‌های سمت Dart
`services/scheduled_delivery_service.dart`	137	تحویل‌دهندهٔ Dart
`services/sms_service.dart`	1055	ارسال، دریافت، آینه و کدهای خطا
`services/spam_prompt_store.dart`	74	پرسش هرزنامه‌ای که رد شده

lib/features/search

ریشه: `lib/features/search`

فایل	خط	کار
`bloc/search_bloc.dart`	157	جستجوی یکپارچه
`bloc/search_event.dart`	26	رویدادهای `search_bloc`
`bloc/search_state.dart`	82	حالت‌های `search_bloc`
`screens/search_screen.dart`	415	صفحهٔ جستجو

lib/features/secure

ریشه: `lib/features/secure`

فایل	خط	کار
`bloc/secure_session_bloc.dart`	704	وضعیت قفل بخش امن
`repositories/intruder_repository.dart`	145	عکس‌های ورود ناموفق
`repositories/key_bank_repository.dart`	189	جدول `kb_own_keys`
`repositories/secure_group_store.dart`	830	جدول‌های `sg_`
`repositories/secure_message_store.dart`	1061	جدول‌های `sm_`
`repositories/secure_store.dart`	431	تنها بازکنندهٔ `secure.db` و طرح آن
`screens/secure_pin_entry_screen.dart`	164	گرفتن PIN برای یک مرحله
`screens/secure_unlock_screen.dart`	291	ساخت و باز کردن بخش
`services/hidden_bridge.dart`	74	پل کانال `hidden`
`services/secure_vault_service.dart`	171	پل کانال `secure_vault`
`services/sms_crypto_service.dart`	673	پل کانال `sms_crypto`
`widgets/secure_lock_button.dart`	63	نماد قفل و `openSecureSection`
`widgets/secure_locked_view.dart`	37	نمای بخش بسته
`widgets/secure_session_guard.dart`	63	قفل با خروج از برنامه

lib/features/secure_sms

ریشه: `lib/features/secure_sms`

فایل	خط	کار
`bloc/secure_messages_bloc.dart`	1062	حالت پیام‌ها و گروه‌های رمز
`repositories/secure_queue_repository.dart`	54	جدول `secure_queue`
`screens/secure_conversation_screen.dart`	716	گفتگوی رمز، composer و بنر
`screens/secure_group_edit_screen.dart`	273	ساخت و ویرایش گروه
`screens/secure_group_screen.dart`	550	گفتگوی گروه رمز
`screens/secure_inbox_entry.dart`	77	ردیف ورود بالای صندوق
`screens/secure_inbox_screen.dart`	593	«پیام‌های رمز»
`screens/secure_new_conversation_screen.dart`	280	«گفتگوی رمز»
`services/hidden_sms_source.dart`	45	پیامک‌های مخفی برای موتور
`services/secure_identities.dart`	144	کلید خودمان و طرف مقابل
`services/secure_messenger.dart`	1719	موتور اصلی
`services/secure_sms_sender.dart`	42	رابط ارسال، برای آزمون جایگزین می‌شود

lib/features/security

ریشه: `lib/features/security`

فایل	خط	کار
`models/device_security_report.dart`	261	وزن‌دهی و متن کنترل امنیت
`screens/device_security_screen.dart`	199	«کنترل امنیت سامانه»
`screens/intruder_photos_screen.dart`	222	«عکس‌های ورود ناموفق»
`services/intruder_reporter.dart`	73	گزارش در PIN درست بعدی
`services/security_bridge.dart`	71	پل متدهای `security.*`

lib/features/settings

ریشه: `lib/features/settings`

فایل	خط	کار
`bloc/blocked_numbers_bloc.dart`	201	فهرست مسدودها و هرزنامه
`bloc/settings_bloc.dart`	315	همهٔ تنظیمات جز تم
`bloc/settings_event.dart`	71	رویدادهای `settings_bloc`
`bloc/settings_state.dart`	306	حالت‌های `settings_bloc`
`models/blocked_number_model.dart`	97	مدل با `normalized` استاندارد
`models/number_lists.dart`	200	فهرست سیاه و سفید
`repositories/blocked_numbers_repository.dart`	291	جدول `blocked_numbers`
`screens/call_block_rules_page.dart`	221	«مسدود کردن تماس»
`screens/number_lists_page.dart`	307	«فهرست سیاه و سفید»
`screens/privacy_policy_page.dart`	476	«حریم خصوصی و دسترسی‌ها»
`screens/secure_section_settings_page.dart`	248	تنظیمات «بخش امن»
`screens/settings_screen.dart`	1121	صفحهٔ اصلی تنظیمات
`screens/settings_subpages.dart`	437	زیرصفحه‌های تنظیمات
`screens/sim_settings_page.dart`	215	«سیم‌کارت‌ها»
`screens/widgets/block_number_dialog.dart`	176	تأیید مسدود کردن
`services/number_lists_bridge.dart`	50	پل فهرست‌ها به Kotlin
`services/system_block_list.dart`	67	پل فهرست مسدود اندروید

کد Kotlin (`kt/`)

ریشه: `android/app/src/main/kotlin/com/example/communication_super_app`

فایل	خط	کار
`BlockedNumbers.kt`	51	خواندن `blocked_numbers` از Kotlin
`HeadlessSmsSendService.kt`	42	پاسخ سریع از صفحهٔ تماس
`IncomingSmsReceiver.kt`	158	گیرندهٔ `SMS_RECEIVED`
`LocationHandler.kt`	147	کانال `location`
`MainActivity.kt`	734	ثبت کانال‌ها، پرچم‌های پنجره و intentها
`MmsReceiver.kt`	22	گیرندهٔ خالی برای شرط نقش پیامک
`NotificationAvatars.kt`	99	عکس گرد اعلان
`NumberListRules.kt`	182	قوانین خالص فهرست سیاه و سفید
`NumberLists.kt`	128	سمت گوشی فهرست‌ها
`SmsDeliverReceiver.kt`	110	گیرندهٔ `SMS_DELIVER` و نوشتن در provider
`SmsHandler.kt`	1335	کانال `sms`
`SmsNotificationActionReceiver.kt`	358	دکمه‌های اعلان پیامک
`SmsNotifier.kt`	753	همهٔ اعلان‌های پیامک
`SystemBlockList.kt`	138	فهرست مسدود اندروید
`TemplateWire.kt`	259	آینهٔ قالب پیامک
`call/CallConnection.kt`	55	داربست VoIP
`call/CallConnectionService.kt`	60	داربست VoIP خفته
`call/CallEvent.kt`	11	مدل رویداد
`call/CallEventStreamHandler.kt`	37	فرستادن رویداد به Dart
`call/CallHandler.kt`	638	کانال `call`
`call/CallInCallService.kt`	1991	قلب تماس: گرفتن تماس از Telecom
`call/CallPrefs.kt`	113	تنظیمات خواندنی بدون Flutter
`call/CallRules.kt`	132	قوانین خالص مسدودی تماس
`call/MissedCallNotificationReceiver.kt`	119	جلوگیری از اعلان تکراری تماس بی‌پاسخ
`call/ProximityGate.kt`	90	حسگر نزدیکی
`calllog/CallLogSyncHandler.kt`	350	کانال `call_log`
`contacts/ContactExtrasHandler.kt`	921	کانال `contact_extras`
`contacts/ContactFileHandler.kt`	274	کانال `contact_file`
`contacts/ContactGroupsHandler.kt`	480	کانال `contact_groups`
`contacts/ContactLinkHandler.kt`	213	کانال `contact_link`
`contacts/SimContactsHandler.kt`	206	کانال `sim_contacts`
`edition/DeviceIdentityHandler.kt`	48	کانال `device_identity`

فایل	خط	کار
`hidden/HiddenCallLog.kt`	121	جاروب تماس مخفی
`hidden/HiddenHandler.kt`	138	کانال `hidden`
`hidden/HiddenNumbers.kt`	209	برچسب‌های شمارهٔ مخفی
`hidden/HiddenSms.kt`	86	مهر پیامک مخفی و ارسال خصوصی
`hidden/SealedInbox.kt`	103	جدول `sealed_queue`
`media/PhotoHandler.kt`	440	کانال `media` : انتخاب و برش عکس
`reminders/CallReminderHandler.kt`	78	کانال `call_reminders`
`reminders/CallReminderReceiver.kt`	31	آلارم یادآور
`reminders/CallReminders.kt`	393	ذخیره، زمان‌بندی و اعلان یادآور
`scheduled/BootReceiver.kt`	23	چیدن دوبارهٔ آلارم‌ها بعد از روشن شدن
`scheduled/ScheduledSmsAlarmReceiver.kt`	43	آلارم زمان‌بندی
`scheduled/ScheduledSmsChannel.kt`	75	تحویل به Dart وقتی برنامه باز است
`scheduled/ScheduledSmsScheduler.kt`	110	چیدن یک آلارم برای نزدیک‌ترین پیام
`scheduled/ScheduledSmsWorker.kt`	602	تحویل بی‌سر
`secure/KeystoreKeyWrapper.kt`	133	کلید Keystore
`secure/SecureBiometric.kt`	120	نسخهٔ اثر انگشتی کلید بخش
`secure/SecureVaultHandler.kt`	334	کانال `secure_vault`
`secure/VaultCrypto.kt`	172	قالب vault
`security/AppLockHandler.kt`	91	کانال `app_lock`
`security/AppLockSession.kt`	136	آیا برنامه الان قفل است
`security/BiometricGate.kt`	335	پنجرهٔ اثر انگشت
`security/DeviceSecurity.kt`	197	واقعیت‌های امنیتی گوشی
`security/IntruderCamera.kt`	287	عکس از ورود ناموفق
`security/NotificationPrivacy.kt`	134	پنهان کردن محتوای اعلان
`security/SecurityHandler.kt`	83	متدهای `security.*`
`sim/SimHandler.kt`	182	کانال‌های `sim`
`sim/SimRegistry.kt`	642	فهرست سیم‌ها و قانون سیم تماس
`smscrypto/CertHandshake.kt`	254	دست‌دهی گواهی
`smscrypto/CoverText.kt`	164	متن پوششی
`smscrypto/CoverWords.kt`	93	واژه‌نامهٔ منجمد
`smscrypto/Identity.kt`	137	کلید بلندمدت
`smscrypto/Kdf.kt`	39	HKDF و SHA-256، HMAC

فایل	خط	کار
`smscrypto/KeyFilePicker.kt`	99	انتخابگر فایل کلید
`smscrypto/Payload.kt`	441	قالب محتوا و جدول فارسی
`smscrypto/SealedBox.kt`	105	مهر یک‌طرفه
`smscrypto/SecureSmsInbox.kt`	225	پارک کردن پیام رمز و تصمیم اعلان
`smscrypto/Session.kt`	202	ratchet و رمز هر پیام
`smscrypto/SmsCryptoException.kt`	72	کدهای خطا
`smscrypto/SmsCryptoHandler.kt`	411	کانال `sms_crypto`
`smscrypto/Upgrade.kt`	110	ارتقای پساکوانتومی
`smscrypto/Wire.kt`	224	قالب روی سیم
`smscrypto/keybank/Authority.kt`	128	کلید مرجع
`smscrypto/keybank/Bin.kt`	106	خواندن و نوشتن دودویی
`smscrypto/keybank/Canon.kt`	63	شکل استاندارد منجمد
`smscrypto/keybank/Issuer.kt`	147	سمت مرجع و فایل `.hka`
`smscrypto/keybank/KeyFile.kt`	78	فایل `.hkb`
`smscrypto/keybank/MemberCert.kt`	131	گواهی عضو
`smscrypto/keybank/PasswordKdf.kt`	48	Argon2id
`smscrypto/keybank/SealedFile.kt`	84	فایل رمزدار

ابزار خط فرمان

ریشه: `tools/keybank/src`

فایل	خط	کار
`main/kotlin/Activation.kt`	91	کد فعال‌سازی، سمت صادرکننده
`main/kotlin/Issuance.kt`	566	صدور و پوشهٔ مرجع
`main/kotlin/IssuanceLog.kt`	129	دفتر صدور
`main/kotlin/Main.kt`	175	فرمان‌های خط فرمان
`test/kotlin/ActivationTest.kt`	84	آزمون کد فعال‌سازی
`test/kotlin/AuthorityFolderTest.kt`	314	آزمون پوشهٔ مرجع

پنل صدور

ریشه: `tools/keybank/panel/src`

فایل	خط	کار
`main/kotlin/ActivationScreen.kt`	116	صفحه‌های فعال‌سازی
`main/kotlin/Common.kt`	126	تکه‌های مشترک

فایل	خط	کار
`main/kotlin/DeliverySheet.kt`	114	برگهٔ چاپی تحویل
`main/kotlin/KeysScreen.kt`	354	بخش کلید
`main/kotlin/KeysState.kt`	420	حالت بخش کلید
`main/kotlin/MembersUi.kt`	389	فهرست و فرم اعضا
`main/kotlin/PanelApp.kt`	194	پنجره و ریل
`main/kotlin/Platform.kt`	49	تکه‌های ویندوزی
`main/kotlin/SelfTest.kt`	140	آزمایش نسخهٔ نصب‌شده
`test/kotlin/PreviewRender.kt`	356	تصویرهای صفحه‌ها

پیوست ب: تصمیم‌های طراحی

این‌ها تصمیم‌هایی هستند که از روی کد معلوم نیستند. بدون فهمیدن دلیلشان آن‌ها را برنگردانید.

شناسه	تصمیم	چرا	هزینهٔ پذیرفته‌شده
D1	BLoC سراسری، بدون provider در صفحه	تب‌ها در `IndexedStack` زنده می‌مانند و یک حالت را مشترک می‌خواهند	همهٔ BLoCها تمام جلسه زنده‌اند
D2	شمارهٔ نرمال‌شده کلید پیوند است	مخاطب از گوشی خوانده می‌شود و شناسهٔ پایدار ندارد و یک شماره چند فرمت دارد	شمارهٔ خارجی ضعیف‌تر (K6)
D3	برنامه تلفن پیش‌فرض است	پاسخ روی قفل، انتظار تماس و سیم هر تماس	شکننده‌ترین کد، با آزمون دستی
D3a	تماس مجدد فقط برای تماسی که وصل نشده	از داخل برنامه پیام اپراتور با پاسخ آدم فرق ندارد	پیام اپراتور متصل پوشش داده نمی‌شود
D4	حذف تکرار پیامک در دو سطح	پیامک زنده و ورودی از گوشی شناسهٔ متفاوت دارند	دو پیام یکسان در یک ثانیه یکی می‌شوند
D5	صندوق آینهٔ گوشی با به‌روزرسانی ارزان	خواندن کامل چند ثانیه طول می‌کشد	تغییر برنامهٔ دیگر کمی دیر می‌رسد
D6	مجوزها پیش از پوسته	درخواست همزمان تب‌ها برنامه را می‌کشت	یک صفحهٔ کوتاه در شروع اول
D7	نگاشت سنگین در isolate	هزاران تماس رابط را یخ می‌زد	یک کپی از مرز isolate
D8	RTL و فارسی همه‌جا	محصول فارسی است	لایهٔ ترجمه نداریم (K7)
D9	دو پایگاه‌داده، بدون migration	هنوز چیزی منتشر نشده	بعد از انتشار `onUpgrade` لازم است
D10	بازسازی با استخراج ویجت	رفتار روی گوشی سخت‌ترین بخش است	چند `State` بزرگ می‌مانند
D11	دو APK، یک شناسه، یک کلید	فروشگاه با نام بسته تصمیم می‌گیرد	تجاری بدون حذف روی سازمانی نصب نمی‌شود
D12	راز فقط در بخش امن Kotlin	پایگاه‌دادهٔ اصلی را بدون کلید باز می‌کند	پیام رمز فقط پشت PIN خود بخش
D13	دست‌دهی گواهی، بعد ارتقای پساکوانتومی	دست‌دهی کامل پساکوانتومی حدود ده بخش پیامک است	پیام‌های اول تا ارتقا کلاسیک‌اند
D14	فقط `PermissionGate` مجوز می‌خواهد	`Reply already submitted`	چهار پلاگین با کد بومی جایگزین شدند
D15	جابه‌جایی مخاطب با انتخابگر سیستم	به گوشی، کارت و فضای ابری می‌رسد و مجوز حافظه نمی‌خواهد	کاربر هر بار جا را انتخاب می‌کند
D16	تصویر کوچک به شکل متن پیامک	MMS نداریم و متن به هر گوشی می‌رسد	تصویر کوچک است و چند بخش هزینه دارد
D17	یادآورها مال Kotlin هستند	آلارم باید با برنامهٔ بسته کار کند	با SQL نمی‌شود پرس‌وجو کرد

تصمیم‌های ساختاری کامل‌تر در `docs/DESIGN_DECISIONS.md` هستند.

پیوست پ: واژه‌نامه

واژه	معنی
AAD	دادهٔ همراهی که رمز نمی‌شود ولی دستکاری‌اش در GCM کشف می‌شود
AES-256-GCM	رمز متقارن با تشخیص دستکاری
Argon2id	تبدیل گذرواژه به کلید، که عمداً حافظهٔ زیاد می‌خواهد
BCD	نوشتن هر رقم در ۴ بیت
BLAKE3	تابع هش سریع، در کد فعال‌سازی
BLoC	الگوی مدیریت حالت با رویداد ورودی و حالت خروجی
Content provider	رابط اندروید برای دادهٔ مشترک، مثل `content://sms`
CTE	زیرپرس‌وجوی نام‌دار در SQL با `WITH`
Ed25519	امضای دیجیتال کلاسیک، برای گواهی عضو
FLAG_SECURE	جلوگیری از عکس صفحه و تصویر recents
Flavor	گونهٔ ساخت Gradle، اینجا `commercial` و `organization`
FTS	جستجوی متن کامل در SQLite
Golden vector	خروجی ثابت‌شدهٔ بایت‌به‌بایت برای قفل کردن قالب
GSM-7	الفبای ۷ بیتی پیامک، ۱۶۰ نویسه در یک پیامک
HKDF	مشتق کردن چند کلید از یک راز
HMAC	هش کلیددار
Isolate	رشتهٔ اجرای جدا در Dart
Keystore	نگهداری کلید سخت‌افزاری اندروید که بیرون نمی‌آید
LEB128	نوشتن عدد با طول متغیر
ML-DSA-65	امضای پساکوانتومی (FIPS 204)
ML-KEM-768	توافق کلید پساکوانتومی (FIPS 203)
Mirror-sync	به‌روز نگه داشتن پایگاه‌دادهٔ برنامه از provider گوشی
Nonce	عدد یک‌بارمصرف همراه رمز
Noise-XX	الگوی دست‌دهی با احراز دوطرفه
PBKDF2	تبدیل PIN به کلید با تکرار زیاد
Post-quantum / پساکوانتومی	مقاوم در برابر رایانهٔ کوانتومی
Ratchet	زنجیرهٔ یک‌طرفهٔ کلید. کلید قدیمی از جدید به دست نمی‌آید
ROLE_DIALER، ROLE_SMS	نقش تلفن و پیامک پیش‌فرض اندروید
Salt / نمک	مقدار تصادفی برای یکتا کردن هش
Soft delete / tombstone	ردیفی که پاک‌شده علامت می‌خورد تا آینه آن را برنگرداند

واژه	معنی
SQLCipher	SQLite رمزشده
StrongBox	تراشهٔ امن جدا در برخی گوشی‌ها
T9	جستجوی نام با حروف روی کلیدها
TEE	محیط اجرای امن پردازنده
Telecom	چارچوب تماس اندروید
X25519	توافق کلید کلاسیک
مرجع	سازمانی که برای اعضا کلید می‌سازد
نسل (`generation`)	شمارندهٔ کلید عضو، با گوشی گم‌شده زیاد می‌شود
نشست	حالت مشترک رمز دو طرف
دست‌دهی	چند پیام که دو طرف با آن به نشست می‌رسند
گواهی عضو	سند ۱۰۳ بایتی امضای مرجع: این کلید مال این شماره است
فایل کلید	فایل `.hkb` عضو
مهر کردن	رمز یک‌طرفه‌ای که نویسنده نمی‌تواند باز کند
ماتریس	جدول امکانات نسخه‌ها در `docs/پیوست فنی.docx`

پیوست ت: برای هر تغییر سراغ کجا بروید

مسیرهای `kt/` نسبت به پوشهٔ Kotlin و بقیه نسبت به `lib/features/` یا `lib/` هستند.

می‌خواهید	سراغ این فایل‌ها	بخش
متن یا ظاهر حباب پیام	`messages/screens/widgets/message_bubble.dart`	۱۰
composer	`messages/screens/widgets/message_composer.dart`	۱۰
ردیف صندوق	`messages/screens/widgets/thread_tile.dart`	۹
جستجوی صندوق	`MessageRepository`، `message_search_query.dart`	۹
اعلان پیامک	`kt/SmsNotifier.kt`	۸
مسیر رسیدن پیامک	`kt/SmsDeliverReceiver.kt`، `kt/IncomingSmsReceiver.kt`	۸
ارسال پیامک	`messages/services/sms_service.dart`، `kt/SmsHandler.kt`	۸
زمان‌بندی	`scheduled_message_model.dart`، `kt/scheduled/ScheduledSmsWorker.kt`	۱۲
قالب و پیش‌نویس	`messages/models/`، `messages/screens/`	۱۱
صفحهٔ تماس	`dialer/screens/in_call_screen.dart`، `incoming_call_screen.dart`	۱۳
رفتار تماس	`kt/call/CallInCallService.kt`، `core/navigation/call_ui_coordinator.dart`	۱۳
صفحه‌کلید	`dialer/widgets/dialer_widgets.dart`، `dialer/bloc/dialer_bloc.dart`	۱۳
«اخیر»	`call_history/`	۱۴
هر چیزی با سیم‌کارت	`core/sim/`، `kt/sim/SimRegistry.kt`	۱۵
مخاطب	`contacts/repositories/contact_repository.dart`، `kt/contacts/`	۱۶
جستجو و T9	`core/utils/search_text.dart`	۱۷
تنظیم تازه	`settings_state.dart`، `settings_bloc.dart`، `settings_screen.dart`	۱۸
تنظیمی که Kotlin باید بخواند	همان، به‌علاوهٔ `kt/call/CallPrefs.kt`	۱۸
PIN و قفل	`authentication/`، `core/widgets/app_lock_wrapper.dart`	۱۹
امکان فقط برای یک نسخه	`state.hasSecureFeatures` یا `kOrganizationBuild`	۲۰
قالب کد فعال‌سازی	**تغییر نکنید.** `core/edition/activation_code.dart` و `Activation.kt`	۲۰
جدول `secure.db`	`secure/repositories/secure_store.dart`	۲۱
قالب رمز	`Wire.VERSION` **تازه.** `kt/smscrypto/`	۲۲
گواهی و فایل کلید	`kt/smscrypto/keybank/`	۲۳
رفتار پیام رمز	`secure_sms/services/secure_messenger.dart`	۲۴
صفحه‌های پیام رمز	`secure_sms/screens/`	۲۴
گروه رمز	`secure/repositories/secure_group_store.dart`، `secure_messenger.dart`	۲۴

می‌خواهید	سراغ این فایل‌ها	بخش
دفترچهٔ مخفی	`hidden/`، `kt/hidden/`	۲۵
متن پوششی	`kt/smscrypto/CoverText.kt`	۲۶
کنترل امنیت	`kt/security/DeviceSecurity.kt`، `device_security_report.dart`	۲۶
پنل	`tools/keybank/panel/src/main/kotlin/`	۲۷
رنگ و فاصله	`core/theme/`	۲۸
جدول پایگاه‌داده	`core/database/database_helper.dart`، `core/constants/app_constants.dart`	۶
کانال تازه	`kt/MainActivity.kt` و سرویس Dart	۷
مجوز تازه	`AndroidManifest.xml`، `PermissionService`، `PrivacyPolicyPage`، `store-release.md`	۳۲
BLoC تازه	`core/bloc_providers/app_bloc_providers.dart`	۵
مراحل شروع برنامه	`main.dart`، `permission_gate.dart`، `default_app_gate.dart`	۴
ساخت و انتشار	`scripts/release_build.sh`، `android/app/build.gradle.kts`	۳۲

سندهای کنار کد

سند	چه چیزی دارد
`docs/DEVELOPER_GUIDE.md`	قوانین سراسری به انگلیسی
`docs/architecture/*.md`	جزئیات هر قسمت، با تاریخچهٔ خطاها
`docs/KNOWN_ISSUES.md`	مشکلات باز
`docs/DESIGN_DECISIONS.md`	تصمیم‌ها
`docs/FINAL_DEVICE_TESTS.md`	آزمون دو گوشی
`docs/publishing/store-release.md`	وضعیت زندهٔ انتشار
`tools/keybank/README.md`	ابزارها

هر تغییری یادداشت همان قسمت را **در همان کامیت** به‌روز می‌کند. پس اگر این راهنما با آن یادداشت‌ها یا با کد فرق داشت، کد درست است و بعد از آن یادداشت‌ها.

پایان سند. هر تغییر در کد، در همان کامیت در یادداشت همان بخش زیر `docs/architecture/` هم نوشته می‌شود.